

IBPS SO IT OFFICER 2026

DATA STRUCTURES & ALGORITHMS + OBJECT-ORIENTED PROGRAMMING

COMPLETE PROFESSIONAL KNOWLEDGE MASTER NOTES

Zero to Exam Ready | Detailed Concepts | PYQ Analysis | 200 MCQs

PREPARED BY

Ways to Officer: Dpksanu

Copyright & Content Protection Notice

This study material has been prepared for educational and examination preparation purposes under “Ways to Officer: Dpksanu”. Unauthorized copying, reproduction, redistribution, resale, modification, publication, or commercial use of this PDF or any substantial portion of its content is strictly prohibited without prior written permission.

Copying and unauthorized redistribution of this PDF are strictly prohibited.

© Ways to Officer: Dpksanu. All rights reserved.

Disclaimer

This study material is an independent educational resource prepared for IBPS SO IT Officer 2026 Professional Knowledge preparation. It is not officially affiliated with, endorsed by, or published by the Institute of Banking Personnel Selection (IBPS) or any participating bank.

Previous-year questions identified as memory-based or pattern-based are reconstructed from publicly available exam analyses, educational sources, and candidate recollections. Exact wording may differ from the original examination. No question in this book is falsely claimed to be an actual reproduced previous-year question.

Exam patterns and syllabus emphasis may change. Candidates should refer to the latest official IBPS notification for official examination information.

Table of Contents

PART A Data Structures & Algorithms (Ch 1-13)

1 Introduction · 2 Complexity · 3 Arrays · 4 Strings · 5 Linked Lists · 6 Stacks · 7 Queues

8 Recursion · 9 Trees · 10 Balanced/Special Trees · 11 Graphs · 12 Hashing · 13 Searching & Sorting

PART B Object-Oriented Programming (Ch 14-23)

14 Foundations · 15 Four Pillars · 16 Encapsulation/Abstraction · 17 Inheritance · 18 Polymorphism

19 Abstract Classes/Interfaces · 20 Constructors · 21 Keywords/Modifiers · 22 Relationships · 23 Java vs Python

PART C Last-Minute Revision (Notes, Top-100 Facts ×2, Master Tables, Top-50 Traps ×2, Confusing Concepts)

PART D 200 IBPS SO IT MCQs (Q1-100 Easy, Q101-200 Moderate-Hard)

PART E Detailed Answer Key & Explanations (Q1-200)

PART F Sources & References

PART A — DATA STRUCTURES & ALGORITHMS

CHAPTER 1 — INTRODUCTION TO DATA STRUCTURES

Welcome. This is the very first step of your journey. We assume you know **nothing** about computers or programming, and by the end of this chapter you will understand what a data structure is, why it exists, and the exact vocabulary IBPS uses to ask questions about it.

1.1 Data vs Information

Simple Meaning

Data is a raw fact that has no meaning on its own. **Information** is data that has been processed and now carries meaning.

Real-Life Analogy

Imagine someone hands you a slip of paper that just says "**90**". - On its own, "90" is just a number — this is **data**. - If the slip says "**Rahul scored 90 marks in Maths**", now it means something — this is **information**.

Technical Explanation

Data are individual, unorganised values (numbers, characters, symbols). When we organise, process, or give context to that data, it becomes information that supports a decision.

Point	Data	Information
Meaning	Raw, no meaning alone	Processed, meaningful
Depends on	Nothing	Depends on data
Example	90, "A", 25	"Rahul scored 90 marks"
Usefulness	Low by itself	High for decisions

★ IBPS Exam Point

Data → (processing) → Information. Information is always **processed data**.

◆ Memory Trick

Data = **D**ull (no meaning). **Information** = **I**mportant (has meaning).

1.2 What is a Data Structure?

Simple Meaning

A **data structure** is a particular way of **storing and organising data** in a computer so that it can be used efficiently.

Why We Need It

A computer can hold millions of data items. If they are stored carelessly, finding or updating even one item becomes slow. A good data structure lets us store data so that operations like **searching, inserting, deleting, and sorting** happen quickly and use less memory.

Real-Life Analogy

Think of a **library**. If books were dumped randomly on the floor, finding one book would take hours. Instead, books are arranged on labelled shelves by subject. That arrangement is a "data structure" for books — it makes searching fast.

Technical Explanation

A data structure defines: 1. How data is arranged in memory, and 2. The set of operations allowed on that data (insert, delete, search, traverse, update).

★ IBPS Exam Point

A data structure is about **organisation of data + operations on it**, not just storage.

1.3 What is an Algorithm?

Simple Meaning

An **algorithm** is a **finite, step-by-step set of instructions** to solve a problem.

Real-Life Analogy

A **cooking recipe** is an algorithm: a clear, ordered list of steps that, if followed correctly, always produces the dish.

Key Properties of a Good Algorithm

- **Input** — takes zero or more inputs.
- **Output** — produces at least one output.
- **Finiteness** — must end after a limited number of steps (it cannot run forever).
- **Definiteness** — each step is clear and unambiguous.
- **Effectiveness** — each step is basic enough to be carried out.

The Core Relationship (very important)

Data → Data Structure → Algorithm → Output

Raw **data** is placed into a **data structure** (organised form). An **algorithm** then processes that organised data to produce the desired **output**. Data structures and algorithms are two sides of the same coin — you choose a data structure to make your algorithm efficient.

★ IBPS Exam Point

Finiteness is the property that separates an algorithm from an ordinary process. An algorithm **must terminate**.

⚠ Examiner's Trap

A process that never ends is **not** an algorithm. Watch for options that describe an "infinite loop" as an algorithm — that is wrong.

1.4 Abstract Data Type (ADT)

Simple Meaning

An **ADT** describes **what** a data structure does (its operations and behaviour) **without** saying **how** it is built internally.

Real-Life Analogy

Think of a **TV remote**. You know it has buttons for "volume up", "channel change", etc. (what it does). You don't need to know the circuit inside (how it works). The remote's list of buttons is like an ADT.

Technical Explanation

An ADT is a **logical/mathematical model**. For example, a **Stack ADT** promises operations `push`, `pop`, `peek` with LIFO behaviour. Whether it is implemented using an array or a linked list is hidden — that is the actual **data structure** implementing the ADT.

Term	Focus	Example
ADT	What (behaviour, operations)	Stack ADT: push/pop/peek
Data Structure	How (actual implementation)	Stack using array / linked list

★ IBPS Exam Point

ADT = specification (logical view). Data structure = implementation (physical view). ADT **hides implementation details** (this is abstraction).

◆ Memory Trick

ADT = Abstract = Asks only "what", not "how".

1.5 Classification of Data Structures

This classification is a **very frequent** source of IBPS MCQs. Learn every branch.

(A) Primitive vs Non-Primitive

Primitive data structures are the basic building blocks provided directly by the language. **Non-primitive** structures are built using primitive types.

Type	Meaning	Examples
Primitive	Basic, predefined data types	int, float, char, boolean, pointer
Non-Primitive	Built from primitives	Array, Stack, Queue, Linked List, Tree, Graph

(B) Linear vs Non-Linear (most important split)

Linear: elements are arranged in a **sequence**, one after another. Each element has one predecessor and one successor (except the ends).

Non-Linear: elements are arranged in a **hierarchical or connected** manner; an element may connect to many others.

Type	Arrangement	Examples
Linear	Sequential (one level)	Array, Stack, Queue, Linked List
Non-Linear	Hierarchical / networked	Tree, Graph

(C) Static vs Dynamic

Static: size is **fixed** at compile time and cannot change during execution.

Dynamic: size can **grow or shrink** during execution (run time).

Type	Size	Memory	Example
Static	Fixed	Allocated at compile time	Array
Dynamic	Flexible	Allocated at run time	Linked List

(D) Homogeneous vs Non-Homogeneous

Homogeneous: all elements are of the **same** data type (e.g., an array of integers). **Non-Homogeneous (heterogeneous):** elements may be of **different** data types (e.g., a structure/record holding a name (string) + age (int) + marks (float)).

★ IBPS Exam Point

- Array = Linear + Static + Homogeneous.
- Linked List = Linear + Dynamic.
- Tree and Graph = Non-Linear.

⚠ Examiner's Trap

A **Stack** and a **Queue** are **linear** structures even though they are often drawn vertically or built using arrays/linked lists. Do not call them non-linear. Also: a **tree** is non-linear, but a **traversal output** of a tree is linear — don't confuse the structure with its traversal.

◆ Memory Trick

"**A-S-Q-L are Linear**" → **A**rray, **S**tack, **Q**ueue, **L**inked list are linear. **T**ree and **G**raph ("**TG**") are non-linear.

1.6 Common Operations on Data Structures

Every data structure supports some of these operations. IBPS often asks which operation a structure supports and how costly it is.

Operation	Meaning
Traversal	Visiting each element once
Insertion	Adding a new element
Deletion	Removing an element
Searching	Finding an element
Sorting	Arranging elements in order
Merging	Combining two structures
Updating	Changing an existing element's value

★ IBPS Exam Point

"Traversal" means visiting **every** element **exactly once**. Searching may stop early once the item is found.

1.7 Chapter 1 — Quick Revision Box

- **Data** = raw facts; **Information** = processed, meaningful data.
- **Data structure** = organised storage + allowed operations.
- **Algorithm** = finite, definite, effective steps with input & output; **must terminate**.
- Flow: **Data** → **Data Structure** → **Algorithm** → **Output**.
- **ADT** = what (logical); **Data structure** = how (physical).

- Linear: Array, Stack, Queue, Linked List. Non-linear: Tree, Graph.
- Array = Static + Homogeneous; Linked List = Dynamic.

CHAPTER 2 — ALGORITHM ANALYSIS AND COMPLEXITY

This is the single highest-priority chapter in Part A. IBPS SO IT almost always includes complexity/Big-O questions. Master it completely.

We build this from absolute zero.

2.1 Why Do We Analyse Algorithms?

Simple Meaning

Usually there is **more than one** algorithm to solve the same problem. **Analysis** helps us decide which one is **better** — faster and lighter on memory.

Real-Life Analogy

To travel from home to office you could walk, cycle, or drive. All three "solve" the problem, but they differ in **time** and **fuel (resources)**. Choosing the best route = analysing algorithms.

The Two Resources We Measure

1. **Time Complexity** — how much **time** the algorithm takes as input grows.
2. **Space Complexity** — how much **memory** it needs as input grows.

⚠ Why Not Just Use a Stopwatch?

Actual running time depends on the computer's speed, the language, and other programs running. That is unreliable. Instead we count **how the number of basic operations grows** with input size — this is machine-independent.

2.2 The Building Blocks (zero-knowledge foundation)

- **Input:** the data given to an algorithm (e.g., a list of numbers to sort).
- **Input size (n):** how much data there is (e.g., number of elements). Complexity is always expressed **in terms of n**.
- **Operation:** one basic step — a comparison, an addition, an assignment.

- **Execution growth:** how the operation count rises as **n** rises. This growth is what we care about.

★ IBPS Exam Point

Complexity describes **growth with respect to input size n**, not the exact time in seconds.

2.3 Best, Average, and Worst Case

For the same algorithm, difficulty can depend on the specific input.

- **Best case:** the input that makes the algorithm finish fastest (least operations).
- **Worst case:** the input that makes it take the most operations.
- **Average case:** the expected number of operations over all typical inputs.

Example — Linear Search (checking each element one by one for a target): - Best case: target is the **first** element → **O(1)**. - Worst case: target is the **last** element or **absent** → **O(n)**. - Average case: target is somewhere in the middle → **O(n)**.

★ IBPS Exam Point

Worst case is the most commonly asked and the most important guarantee, because it tells us the maximum time the algorithm can ever take.

2.4 Asymptotic Notations — Big O, Omega, Theta

"Asymptotic" simply means: **behaviour when n becomes very large**. We ignore small inputs and constant factors and look at the dominant growth.

Notation	Symbol	Bounds	Meaning (plain English)
Big O	O	Upper bound	The maximum growth — "no worse than". Worst case.
Omega	Ω	Lower bound	The minimum growth — "at least this fast". Best case.
Theta	Θ	Tight bound	Upper and lower bound match — the exact growth.

Real-Life Analogy

If a shopkeeper says a delivery will take **at most 5 days** → that is Big O (upper bound). "At least 2 days" → Omega. "Exactly around 3 days, no more no less" → Theta.

★ IBPS Exam Point

- Big O → **upper** bound (worst case) — asked most often.
- Omega (Ω) → **lower** bound (best case).
- Theta (Θ) → **tight** bound (both).

⚠ Examiner's Trap

Big O is an **upper** bound, NOT the exact time. Options that say "Big O gives the exact running time" are wrong. Also don't swap Ω and O.

◆ Memory Trick

Omega looks like the last letter → think "**at least / lowest**" (lower bound). **Big O** → "**O**verhead / **O**n the top" (upper bound). **Theta** → **T**ight, both.

2.5 The Growth Order (memorise this line)

From **fastest/best** (grows slowest) to **slowest/worst** (grows fastest):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Complexity	Name	Feel
$O(1)$	Constant	Best — same time regardless of n
$O(\log n)$	Logarithmic	Excellent — halves the work each step
$O(n)$	Linear	Good — grows in step with n
$O(n \log n)$	Linearithmic	Fair — best possible for comparison sorting
$O(n^2)$	Quadratic	Slow — nested loops
$O(n^3)$	Cubic	Slower — triple nested loops
$O(2^n)$	Exponential	Very bad — doubles each added input
$O(n!)$	Factorial	Worst — impractical for large n

Quick numeric feel (n = 10 vs n = 1000)

- $O(\log n)$: ~3 vs ~10 — barely grows.
- $O(n)$: 10 vs 1000.
- $O(n^2)$: 100 vs 10,00,000.
- $O(2^n)$: 1024 vs an astronomically huge number.

◆ Memory Trick

"1, log, n, n-log-n, square, cube, 2-power, factorial" — say it aloud in order until automatic. This exact ordering is a repeated MCQ.

⚠ Examiner's Trap

For large n, **$O(2^n)$ is worse than $O(n^3)$** but people wrongly assume "cube is the worst". Also **$O(\log n)$ is faster than $O(n)$** — a common confusion.

2.6 How to Calculate Complexity (the rules)

Rule 1 — Drop constants. $O(2n) \rightarrow O(n)$. $O(500) \rightarrow O(1)$. **Rule 2 — Keep only the highest-order term.** $O(n^2 + n + 5) \rightarrow O(n^2)$. **Rule 3 — A single simple statement** (one assignment, one comparison) = **$O(1)$** . **Rule 4 — A single loop** running n times = **$O(n)$** . **Rule 5 — Nested loops** = multiply. Two loops each $n \rightarrow O(n^2)$. **Rule 6 — Consecutive (separate) loops** = add, then keep the largest $\rightarrow$ still **$O(n)$** if both are $O(n)$. **Rule 7 — A loop that divides/multiplies the counter by 2** each time = **$O(\log n)$** .

2.7 Worked Complexity Examples (25 examples)

Read each and confirm you understand *why*.

1. Single statement `x = a + b;` $\rightarrow O(1)$ (runs once).

2. Simple single loop

```
for i = 1 to n: print(i)
```

Loop runs n times $\rightarrow O(n)$.

3. Loop with step 2

```
for i = 1 to n step 2: print(i)
```

Runs $n/2$ times $\rightarrow$ drop constant $\rightarrow O(n)$.

4. Two consecutive loops

```
for i = 1 to n: print(i)
for j = 1 to n: print(j)
```

$n + n = 2n \rightarrow O(n)$.

5. Nested loop (both n)

```
for i = 1 to n:
  for j = 1 to n: print(i,j)
```

$n \times n \rightarrow O(n^2)$.

6. Triple nested loop Three loops each $n \rightarrow O(n^3)$.

7. Nested loop, inner depends on outer

```
for i = 1 to n:
  for j = 1 to i: print
```

$1+2+\dots+n = n(n+1)/2 \rightarrow O(n^2)$.

8. Halving loop

```
i = n
while i > 1: i = i / 2
```

i: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. Number of steps = $\log_2 n \rightarrow \mathbf{O(\log n)}$.

9. Doubling loop

```
i = 1
while i < n: i = i * 2
```

i: $1 \rightarrow 2 \rightarrow 4 \rightarrow \dots \rightarrow n$. Steps = $\log_2 n \rightarrow \mathbf{O(\log n)}$.

10. Loop multiplying by 3 $i = i * 3$ until $i \geq n \rightarrow \mathbf{O(\log_3 n) = O(\log n)}$ (base ignored).

11. Outer n, inner log n

```
for i = 1 to n:
    j = 1
    while j < n: j = j * 2
```

$n \times \log n \rightarrow \mathbf{O(n \log n)}$.

12. Outer n, inner halving of j from n Same as above $\rightarrow \mathbf{O(n \log n)}$.

13. Constant loop

```
for i = 1 to 100: print(i)
```

Runs a fixed 100 times, independent of $n \rightarrow \mathbf{O(1)}$.

14. Loop to $\sqrt{n}$

```
for i = 1; i*i <= n; i++
```

Runs $\sqrt{n}$ times $\rightarrow \mathbf{O(\sqrt{n})}$.

15. Linear search $\rightarrow$ best $O(1)$, worst/average $\mathbf{O(n)}$.

16. Binary search (on sorted array, halves each step) $\rightarrow \mathbf{O(\log n)}$.

17. Bubble sort (nested passes) $\rightarrow \mathbf{O(n^2)}$ worst/average, $\mathbf{O(n)}$ best (already sorted, optimised).

18. Selection sort $\rightarrow \mathbf{O(n^2)}$ in all cases.

19. Insertion sort $\rightarrow$ worst/avg $\mathbf{O(n^2)}$, best $\mathbf{O(n)}$ (already sorted).

20. Merge sort $\rightarrow \mathbf{O(n \log n)}$ in all cases.

21. Quick sort $\rightarrow$ best/avg $\mathbf{O(n \log n)}$, worst $\mathbf{O(n^2)}$ (bad pivot).

22. Heap sort $\rightarrow \mathbf{O(n \log n)}$ in all cases.

23. Traversing a matrix $n \times n$ Two nested loops over rows and columns $\rightarrow \mathbf{O(n^2)}$.

24. Simple recursion — factorial $\text{fact}(n) = n * \text{fact}(n-1)$ makes n calls $\rightarrow \mathbf{O(n)}$ time.

25. Recursion — naive Fibonacci $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ branches into two calls each time $\rightarrow \mathbf{O(2^n)}$ (exponential — classic trap).

★ IBPS Exam Point

A loop where the counter is **multiplied or divided by a constant** each iteration is **logarithmic — $O(\log n)$** . This is one of the most repeated concepts.

⚠ Examiner's Trap

Naive recursive Fibonacci is **$O(2^n)$** , not $O(n)$. The two recursive calls per step cause exponential blow-up.

2.8 Space Complexity (short but tested)

Space complexity = total memory an algorithm needs relative to n . It includes the input space plus any **auxiliary (extra) space** the algorithm uses.

- A loop that uses a few variables → **$O(1)$** auxiliary space (in-place).
- Merge sort needs an extra array → **$O(n)$** auxiliary space.
- Recursion uses the **call stack**; depth of recursion = extra space (e.g., factorial recursion → $O(n)$ stack space).

★ IBPS Exam Point

Merge sort is $O(n \log n)$ time but needs **$O(n)$** extra space. Quick sort is in-place with **$O(\log n)$** average stack space. This time-vs-space trade-off is a favourite MCQ.

2.9 MASTER TIME COMPLEXITY TABLE

Data Structure operations (average case):

Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Doubly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Hash Table	N/A	$O(1)$	$O(1)$	$O(1)$
Binary Search Tree (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary Search Tree (worst/skewed)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

* $O(1)$ when inserting/deleting at a **known** position such as the head; $O(n)$ if you must first search/traverse to reach the position.

Sorting algorithms:

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Searching algorithms:

Algorithm	Best	Average	Worst	Requirement
Linear Search	$O(1)$	$O(n)$	$O(n)$	None
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	Sorted data

⚠ Examiner's Trap

- **Hash Table** has no meaningful "access by index"; its strength is **$O(1)$ search on average**, but worst case (many collisions) is **$O(n)$** .
- **BST** is $O(\log n)$ only when reasonably **balanced**; a skewed BST degrades to **$O(n)$** .
- Quick sort's **worst** case is **$O(n^2)$** , even though its average is $O(n \log n)$.

2.10 Chapter 2 — Quick Revision Box

- Big O = upper (worst), Ω = lower (best), Θ = tight.
- Growth: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$
- Multiply/divide counter by constant $\rightarrow O(\log n)$.
- Nested loops multiply; consecutive loops add (keep largest).
- Comparison-based sorting can't beat $O(n \log n)$.
- Merge sort: $O(n \log n)$ time, $O(n)$ space, stable. Quick sort worst $O(n^2)$.
- Naive Fibonacci recursion = $O(2^n)$.

CHAPTER 3 — ARRAYS

The array is the **first and most fundamental** data structure. Many later structures (stacks, queues, heaps, hash tables) are built on arrays.

3.1 What is an Array?

Simple Meaning

An **array** is a collection of elements of the **same data type**, stored in **contiguous (side-by-side) memory locations**, and accessed using an **index**.

Real-Life Analogy

Think of a **row of lockers** in a school corridor, numbered 0, 1, 2, 3... Each locker holds one item, and you can jump straight to locker number 5 without opening lockers 0–4. The locker number is the **index**.

Basic Terminology

- **Element:** one value stored in the array.
- **Index:** the position number of an element. In most languages indexing starts at **0** (so the first element is at index 0).
- **Contiguous memory:** all elements sit next to each other in memory, with no gaps.
- **Size/Length:** total number of elements the array can hold.

Example

Index:	0	1	2	3	4
Value:	10	20	30	40	50

Here $A[0] = 10$, $A[2] = 30$, $A[4] = 50$. The array length is 5.

★ IBPS Exam Point

Array = **linear + static + homogeneous**, stored in **contiguous** memory. Indexing typically starts at **0**.

◆ Memory Trick

A-R-R-A-Y → **A**ll elements in a **R**ow, **R**eached by index, **A**ll same type, **Y**ou get instant access.

3.2 Why Arrays Are Fast at Access — $O(1)$

Because elements are contiguous and the same size, the computer can **calculate** the exact memory address of any index directly (formula in 3.6). It does not need to walk through earlier elements. This is why **access by index is $O(1)$** — constant time.

3.3 Types of Arrays

- **One-dimensional (1D):** a simple list — $A[i]$.

- **Two-dimensional (2D):** a table/matrix with rows and columns — $A[i][j]$.
- **Multidimensional:** three or more dimensions — $A[i][j][k]$, etc.

2D array visual (3 rows × 3 columns):

	Col0	Col1	Col2
Row0	1	2	3
Row1	4	5	6
Row2	7	8	9

$A[1][2] = 6$ (row 1, column 2).

3.4 Array Operations and Their Complexity

Operation	What happens	Time Complexity
Access (by index)	Direct address calculation	$O(1)$
Traversal	Visit all n elements	$O(n)$
Search (unsorted)	Linear scan	$O(n)$
Search (sorted)	Binary search	$O(\log n)$
Insertion (at end, space available)	Place at last free slot	$O(1)$
Insertion (at start/middle)	Shift elements right	$O(n)$
Deletion (at end)	Remove last	$O(1)$
Deletion (at start/middle)	Shift elements left	$O(n)$
Update (by index)	Overwrite value	$O(1)$

Why Insertion/Deletion in the Middle is $O(n)$

To insert at the front, **every** existing element must move one place to the right to make room. To delete from the front, every later element shifts one place left to fill the gap. In the worst case that is n shifts → $O(n)$.

Step-by-Step: Insert 99 at index 1 in $[10, 20, 30, 40]$

```

Start:      10  20  30  40
Shift right: 10  __  20  30  40  (20,30,40 moved right)
Insert 99:  10  99  20  30  40
  
```

Elements shifted = 3 → grows with n → $O(n)$.

★ IBPS Exam Point

- Array **access/update by index** = $O(1)$.
- Array **insertion/deletion in the middle** = $O(n)$ because of shifting.

⚠ Examiner's Trap

Do not answer "insertion in an array is $O(1)$ " blindly. It is $O(1)$ **only at the end** (with space); anywhere else it is **$O(n)$** due to shifting. This nuance is a classic trap.

3.5 Advantages and Limitations

Advantages - Instant $O(1)$ access to any element by index. - Simple and memory-efficient (no extra pointers). - Good cache performance because memory is contiguous.

Limitations - **Fixed size** (static) — you must know the size in advance; resizing is costly. - Insertion/deletion in the middle is slow ($O(n)$ shifting). - Wasted memory if the array is larger than needed; overflow if too small.

◆ Memory Trick

Arrays are "**fast to read, slow to edit (in the middle)**".

3.6 Address Calculation (numericals — IBPS loves these)

Because memory is contiguous, we can compute the address of any element. Let: - **B** = Base address (address of the first element / start of array) - **w** = size of one element in bytes (e.g., int often 2 or 4 bytes — the question will state it) - **LB** = lower bound (the starting index — usually 0, sometimes 1)

One-Dimensional Array

$$\text{Address of } A[i] = B + (i - LB) \times w$$

Example: Base address $B = 1000$, each element $w = 4$ bytes, $LB = 0$. Find address of $A[5]$.

$$\text{Address} = 1000 + (5 - 0) \times 4 = 1000 + 20 = 1020$$

Two-Dimensional Array

A 2D array is actually stored in a **single line** of memory. There are two ways to flatten it:

Row-Major Order — store **row by row** (row 0 fully, then row 1, ...). Used by **C, C++, Java, Python**.

$$\text{Address of } A[i][j] = B + [(i - LB_{\text{row}}) \times N + (j - LB_{\text{col}})] \times w \text{ where } N = \text{number of columns.}$$

Column-Major Order — store **column by column** (column 0 fully, then column 1, ...). Used by **FORTRAN, MATLAB, R**.

$$\text{Address of } A[i][j] = B + [(j - LB_{\text{col}}) \times M + (i - LB_{\text{row}})] \times w \text{ where } M = \text{number of rows.}$$

Worked Numerical (row-major)

An array `A[3][4]` (3 rows, 4 columns), base $B = 2000$, $w = 2$ bytes, $LB = 0$, row-major. Find address of `A[2][3]`.

```
N (columns) = 4
Address = 2000 + [ (2 - 0) × 4 + (3 - 0) ] × 2
          = 2000 + [ 8 + 3 ] × 2
          = 2000 + 11 × 2
          = 2000 + 22 = 2022
```

Worked Numerical (column-major, same array)

Find address of `A[2][3]` in **column-major** order.

```
M (rows) = 3
Address = 2000 + [ (3 - 0) × 3 + (2 - 0) ] × 2
          = 2000 + [ 9 + 2 ] × 2
          = 2000 + 11 × 2
          = 2000 + 22 = 2022
```

(They match here only by coincidence of numbers; usually they differ. Always apply the correct formula.)

★ IBPS Exam Point

- **Row-major:** multiply the row difference by the **number of columns (N)**.
- **Column-major:** multiply the column difference by the **number of rows (M)**.
- C/C++/Java/Python use **row-major**.

⚠ Examiner's Trap

The most common mistake is using the wrong dimension: in row-major you multiply by **columns**, not rows. Also check whether **LB is 0 or 1** — if the array starts at index 1, subtract 1.

◆ Memory Trick

"Row-major → Columns count." The order you're NOT going along (across a row you cross columns) is the number you multiply by. Row-major fills rows, so you count how many columns you skip per row.

3.7 Chapter 3 — Quick Revision Box

- Array = same type, contiguous memory, index-accessed; index starts at 0.
- Access/Update = $O(1)$; Search (unsorted) = $O(n)$; Insert/Delete in middle = $O(n)$.
- Static structure — fixed size.
- 1D: $\text{Addr} = B + (i - LB) \times w$.
- 2D Row-major: $B + [(i - LB_r) \times N + (j - LB_c)] \times w$ — multiply row diff by **columns**.
- 2D Column-major: $B + [(j - LB_c) \times M + (i - LB_r)] \times w$ — multiply col diff by **rows**.

- C/C++/Java/Python = row-major; FORTRAN/MATLAB = column-major.

CHAPTER 4 — STRINGS

4.1 What is a String?

Simple Meaning

A **string** is a **sequence of characters** treated as a single unit — for example, "BANK" or "IBPS2026".

Real-Life Analogy

A string is like a **word made of alphabet beads** on a thread. Each bead is one character; the whole thread is the string.

Basic Terminology

- **Character:** a single letter, digit, or symbol ('A', '7', '#').
- **Character array:** the underlying way strings are stored — an array of characters.
- **Length:** number of characters in the string.
- **Null character '\0':** in **C**, a string is a character array ending with a special null terminator '\0' that marks where the string stops.

Example

```
String: "BANK"
Index:  0   1   2   3   (4 in C = '\0')
Char:   B   A   N   K
Length = 4
```

★ IBPS Exam Point

In **C**, a string is a **null-terminated character array**; the '\0' marks the end. So "BANK" actually needs **5** bytes of storage (4 letters + '\0').

4.2 String Storage: Character Array vs String Object

- In **C**, strings are raw character arrays ending in '\0'.
- In **Java**, `String` is a **class/object**, and Java strings are **immutable** — once created, their contents cannot be changed. Any "modification" creates a **new** string object.
- In **Python**, strings are also **immutable** sequences of characters.

★ IBPS Exam Point

Java and Python strings are immutable. Operations that appear to change a string actually return a **new** string. For changeable text in Java, use **StringBuilder** / **StringBuffer**.

⚠ Examiner's Trap

"Immutable" does **not** mean you cannot reassign the variable — it means the **existing object's characters** cannot be altered in place. Reassigning `s = s + "x"` makes a new object; the old one is unchanged.

4.3 Common String Operations

Operation	Meaning	Example
Length	Count of characters	<code>len("BANK") = 4</code>
Concatenation	Joining two strings	<code>"BANK" + "ING" = "BANKING"</code>
Comparison	Checking equality / order	<code>"abc" vs "abd"</code>
Substring	Extracting part of a string	<code>substr("BANKING",0,4)="BANK"</code>
Reversal	Reversing character order	<code>"BANK" → "KNAB"</code>
Traversal	Visiting each character	<code>O(n)</code>

Complexity

- Length (if stored), access by index: **$O(1)$** (character array).
- Traversal, comparison, concatenation (building a new string of length n): **$O(n)$** .

⚠ Examiner's Trap

String **comparison** in Java: `==` compares **references** (are they the same object?), while `.equals()` compares **actual contents**. Using `==` to compare string contents is a classic bug and a favourite trick question.

4.4 Pattern Matching (basics)

Pattern matching = finding whether/where a small string (**pattern**) occurs inside a larger string (**text**).

- **Naive (brute-force) approach:** slide the pattern across the text one position at a time and compare. Worst case **$O(m \times n)$** where n = text length, m = pattern length.
- **Efficient algorithms** (names only, for awareness): **KMP (Knuth-Morris-Pratt)** and **Rabin-Karp** improve on naive matching. KMP achieves **$O(n + m)$** .

★ IBPS Exam Point

Naive pattern matching is $O(m \times n)$; KMP improves it to $O(n+m)$. Rabin-Karp uses **hashing** to compare substrings.

4.5 Chapter 4 — Quick Revision Box

- String = sequence of characters; C strings end with `'\0'`.
- Java & Python strings are **immutable**; use `StringBuilder` for mutable text in Java.
- Java: `==` compares references, `.equals()` compares contents.
- Naive matching $O(m \times n)$; KMP $O(n+m)$; Rabin-Karp uses hashing.

CHAPTER 5 — LINKED LISTS

Before linked lists, we must understand a few memory concepts. Do not skip 5.1.

5.1 Prerequisites: Memory, Variable, Address, Pointer

- **Memory:** the computer's storage, divided into many small numbered cells (bytes). Each cell has a unique **address** (like a house number).
- **Variable:** a named box that holds a value and lives at some memory address.
- **Address:** the location number where a value is stored.
- **Pointer / Reference:** a special variable that **stores the address of another variable** — it "points to" where the data lives, instead of holding the data itself.

Real-Life Analogy

A **pointer** is like a **contact in your phone**. The contact isn't the person — it stores the **address/number** that leads you to them. Following the pointer = calling the number to reach the person.

★ IBPS Exam Point

A pointer/reference **stores a memory address**, not the actual data value.

5.2 What is a Linked List?

Simple Meaning

A **linked list** is a linear data structure where elements (**nodes**) are **not** stored contiguously. Each node holds its **data** and a **link (pointer)** to the next node.

Real-Life Analogy

A **treasure hunt**: each clue (node) contains a message (data) **and** directions to the next clue (link). You must follow the chain from the first clue; you cannot jump straight to clue #5.

Basic Terminology

- **Node**: one element = **data field** + **link field**.
- **Data field**: stores the actual value.
- **Link/next field**: stores the address of the next node.
- **Head**: pointer to the **first** node (entry point to the whole list).
- **NULL**: special value in the last node's link, meaning "no next node — list ends here".

Structure Diagram (Singly Linked List)

```
HEAD
|
v
+---+---+ +---+---+ +---+---+
| 10 | •->| 20 | •->| 30 | NULL |
+---+---+ +---+---+ +---+---+
data next  data next  data next(end)
```

★ IBPS Exam Point

The **head** gives access to the whole list. If you lose the head pointer, the entire list is lost.
The **last node** points to **NULL**.

5.3 Types of Linked Lists

Type	Description	Key feature
Singly Linked List	Each node points to the next only	One-way traversal
Doubly Linked List	Each node has prev and next pointers	Two-way traversal
Circular Linked List	Last node points back to the first (not NULL)	No NULL end; loops around

Doubly Linked List node:

```
+-----+-----+-----+
| prev | data | next |
+-----+-----+-----+
```

Circular Linked List: last node's **next** points to the head, forming a ring.

★ IBPS Exam Point

- Doubly linked list allows **backward** traversal (extra memory for the prev pointer).
- Circular linked list has **no NULL** at the end; the last node links back to the first.

5.4 Linked List Operations and Complexity

Operation	Time Complexity	Why
Access / Search by value	$O(n)$	Must traverse from head
Insertion at head	$O(1)$	Just re-point head
Insertion at end	$O(n)$ ($O(1)$ if tail pointer kept)	Must reach the last node
Insertion after a given node	$O(1)$	Re-point two links
Deletion at head	$O(1)$	Move head to next
Deletion at end	$O(n)$	Must reach second-last node

Step-by-Step: Insert 15 at the head of $10 \rightarrow 20 \rightarrow 30$

1. Create new node with data 15.
 2. New node's next = current head (points to 10).
 3. Head = new node.
- Result: $15 \rightarrow 10 \rightarrow 20 \rightarrow 30$

Only pointer changes, no shifting → **$O(1)$** .

★ IBPS Exam Point

Insertion/deletion at the **head** is **$O(1)$** — no element shifting (unlike arrays). This is the linked list's biggest advantage.

⚠ Examiner's Trap

Random access is NOT $O(1)$ in a linked list. To reach the i -th node you must traverse from the head → **$O(n)$** . Arrays give $O(1)$ index access; linked lists do not.

5.5 Array vs Linked List (very high-frequency comparison)

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous (scattered)
Size	Fixed (static)	Dynamic (grows/shrinks)
Access by index	$O(1)$	$O(n)$
Insertion/Deletion at start	$O(n)$ (shifting)	$O(1)$
Extra memory	None	Pointer(s) per node
Cache performance	Better	Poorer

◆ Memory Trick

Array = fast access, slow edit. Linked list = slow access, fast edit (at ends).

5.6 Chapter 5 — Quick Revision Box

- Node = data + link(next). Head = first node; last node's next = NULL.
- Pointer stores an **address**, not data.
- Singly (one-way), Doubly (two-way, prev+next), Circular (last→first, no NULL).
- Insert/delete at head = $O(1)$; access/search = $O(n)$.
- Array vs Linked List is a must-know comparison table.

CHAPTER 6 — STACKS

6.1 What is a Stack?

Simple Meaning

A **stack** is a linear data structure where the element added **last** is removed **first**.

Technical Concept

Stack follows **LIFO — Last In, First Out**.

Real-Life Analogy

A **stack of plates**: you add a plate on top, and you also remove from the top. The last plate placed is the first one taken.

Basic Operations

Operation	Meaning
PUSH	Insert an element on top
POP	Remove the top element
PEEK / TOP	View the top element without removing it
isEmpty	Check if stack has no elements
isFull	Check if stack is full (array implementation)

Example

Push 10, Push 20, Push 30:

TOP
v
30
20
10

POP removes 30 (the last one pushed).

Complexity

Push, Pop, Peek are all **O(1)**.

Overflow and Underflow

- **Stack Overflow:** trying to PUSH when the stack (array-based) is already full.
- **Stack Underflow:** trying to POP when the stack is empty.

★ IBPS Exam Point

Stack = **LIFO**. Push/Pop/Peek = **O(1)**. Overflow = push on full; Underflow = pop on empty.

⚠ Examiner's Trap

Queue follows **FIFO**, not LIFO. Do not confuse the two. Also, "the first element pushed is at the **bottom**" and is removed **last**.

◆ Memory Trick

Stack of Plates = Last Plate In, First Plate Out (LIFO).

6.2 Applications of Stacks

- **Function call management** — the "call stack" stores return addresses; the most recent call returns first.
- **Recursion** — each recursive call is stacked; unwinds in reverse.
- **Undo/Redo** in editors.
- **Expression conversion and evaluation** (infix/postfix/prefix — see 6.3).
- **Balancing parentheses / brackets.**
- **Backtracking** (e.g., maze solving), **browser back button.**

★ IBPS Exam Point

Recursion internally uses a **stack** (the call stack). Function calls are managed **LIFO**.

6.3 Expression Notations (high-yield IBPS topic)

The same expression can be written three ways depending on where the operator sits:

Notation	Operator position	Example for $A + B$
Infix	Between operands	$A + B$
Prefix (Polish)	Before operands	$+ A B$
Postfix (Reverse Polish)	After operands	$A B +$

Computers prefer **postfix** because it needs **no parentheses** and is evaluated easily using a **stack**.

Operator Precedence (high → low)

1. $^$ (exponent) — highest
2. $*$, $/$
3. $+$, $-$ — lowest

Infix → Postfix (worked example)

Convert $A + B * C$. Since $*$ has higher precedence than $+$:

```
Step 1:  $B * C \rightarrow B C *$   
Step 2:  $A + (B C *) \rightarrow A B C * +$   
Postfix =  $A B C * +$ 
```

Convert $(A + B) * C$ (brackets force $+$ first):

```
Step 1:  $(A + B) \rightarrow A B +$   
Step 2:  $(A B +) * C \rightarrow A B + C *$   
Postfix =  $A B + C *$ 
```

Convert $A + B * C - D$:

```
 $B * C \rightarrow B C *$   
 $A + (B C *) \rightarrow A B C * +$   
 $(...) - D \rightarrow A B C * + D -$   
Postfix =  $A B C * + D -$ 
```

Infix → Prefix (worked example)

Convert $A + B * C$:

```
 $B * C \rightarrow * B C$   
 $A + (* B C) \rightarrow + A * B C$   
Prefix =  $+ A * B C$ 
```

Postfix Evaluation using a Stack (worked example)

Evaluate $2\ 3\ 4\ * +$:

```
Read 2 → push      Stack: 2
Read 3 → push      Stack: 2 3
Read 4 → push      Stack: 2 3 4
Read * → pop 4,3; 3*4=12; push 12  Stack: 2 12
Read + → pop 12,2; 2+12=14; push 14 Stack: 14
Result = 14
```

Rule: on an **operand**, push it; on an **operator**, pop the top two, apply, push the result.

★ IBPS Exam Point

- Postfix and prefix need **no parentheses**.
- Postfix evaluation uses a **stack**: push operands, on an operator pop two and compute.
- Precedence order: $\wedge > * / > + -$.

⚠ Examiner's Trap

For a **binary operator** during postfix evaluation, the **first popped** element is the **right** operand and the **second popped** is the **left** operand. This matters for $-$ and $/$ where order changes the answer (e.g., $6\ 2\ - = 6 - 2 = 4$, not $2 - 6$).

◆ Memory Trick

Prefix → operator **pre**cedes (before). **Postfix** → operator comes **post** (after). **Infix** → operator **in** the middle.

6.4 Chapter 6 — Quick Revision Box

- Stack = LIFO; Push/Pop/Peek = $O(1)$.
- Overflow = push on full; Underflow = pop on empty.
- Recursion & function calls use the call **stack**.
- Infix ($A+B$), Prefix ($+AB$), Postfix ($AB+$).
- Postfix/prefix need no brackets; evaluated via stack.
- In postfix eval, first popped = right operand.

CHAPTER 7 — QUEUES

7.1 What is a Queue?

Simple Meaning

A **queue** is a linear data structure where the element added **first** is removed **first**.

Technical Concept

Queue follows **FIFO — First In, First Out**.

Real-Life Analogy

A **queue at a ticket counter**: the person who joins first is served first. New people join at the **rear**; served people leave from the **front**.

Basic Terminology & Operations

- **Front**: end from which elements are **removed**.
- **Rear**: end at which elements are **added**.

Operation	Meaning
ENQUEUE	Insert an element at the rear
DEQUEUE	Remove an element from the front
Front/Ppeek	View the front element
isEmpty / isFull	Status checks

Example

Enqueue 10, 20, 30:

Front → 10 20 30 ← Rear

Dequeue removes 10 (the first one added).

Complexity

Enqueue and Dequeue are **O(1)**.

★ IBPS Exam Point

Queue = **FIFO**. Add at **rear** (enqueue), remove at **front** (dequeue). Both O(1).

⚠ Examiner's Trap

Do not swap the ends: insertion is at the **rear**, deletion at the **front**. (In a stack both happen at the top — different!)

◆ Memory Trick

Queue = Q for "Queue at a shop" = First come, first served (FIFO).

7.2 Types of Queues

Type	Description
Linear Queue	Simple FIFO; once rear reaches the end, space at the front is wasted even if free
Circular Queue	Rear wraps around to the front, reusing empty front slots (no wastage)
Deque (Double-Ended Queue)	Insertion and deletion allowed at both ends
Priority Queue	Each element has a priority; highest priority is served first, not strictly FIFO

Circular Queue — Why It Exists

In a **linear** array queue, after several dequeues the front slots become empty but unusable, and enqueue may falsely report "full". A **circular queue** connects the rear back to the front, so those empty slots get reused. Full/empty are tracked using front and rear positions (often with modulo arithmetic).

Deque

A **deque** allows insert/delete at both front and rear. Special forms: - **Input-restricted deque**: insertion at one end only, deletion at both. - **Output-restricted deque**: deletion at one end only, insertion at both.

Priority Queue

Elements are dequeued by **priority**, not arrival order. Commonly implemented using a **heap** (Chapter 10). Used in CPU scheduling, Dijkstra's algorithm.

★ IBPS Exam Point

- **Circular queue** solves the memory-wastage problem of the linear queue.
- **Priority queue** serves by priority, not FIFO; usually built with a **heap**.
- **Deque** = insert/delete at both ends.

⚠ Examiner's Trap

A **priority queue** is **NOT strictly FIFO** — that's the whole point. Also, a deque can behave like **both** a stack and a queue.

7.3 Applications of Queues

- **CPU scheduling** and **disk scheduling** in operating systems.
- **Print spooling** (jobs served in order).
- **Breadth-First Search (BFS)** in graphs uses a queue.

- **Buffering** (e.g., IO buffers, data streaming).

★ IBPS Exam Point

BFS uses a queue; DFS uses a stack (or recursion). Remember this pairing.

7.4 Stack vs Queue (comparison)

Feature	Stack	Queue
Order	LIFO	FIFO
Insert	Push (top)	Enqueue (rear)
Remove	Pop (top)	Dequeue (front)
Ends used	One (top)	Two (front & rear)
Used in	Recursion, DFS, undo	BFS, scheduling, buffering

7.5 Chapter 7 — Quick Revision Box

- Queue = FIFO; enqueue at rear, dequeue at front; both $O(1)$.
 - Circular queue reuses empty slots (fixes linear-queue wastage).
 - Deque = both ends; Priority queue = by priority (heap-based).
 - BFS → queue; DFS → stack.
 - Stack (one end, LIFO) vs Queue (two ends, FIFO).
-

CHAPTER 8 — RECURSION

8.1 What is Recursion?

Simple Meaning

Recursion is when a function **calls itself** to solve a smaller version of the same problem, until it reaches a simple case it can answer directly.

Real-Life Analogy

Standing between two facing mirrors, you see an image inside an image inside an image... each reflection is a smaller copy of the same scene. Recursion is a function repeating itself on smaller inputs.

Two Essential Parts

1. **Base case:** the stopping condition — the simplest input that is answered **without** further recursion. **Without a base case, recursion never stops** (infinite recursion → crash).
2. **Recursive case:** the function calls itself on a **smaller** input, moving toward the base case.

Example — Factorial

```
factorial(n):  
    if n == 0 or n == 1:      # base case  
        return 1  
    else:  
        return n * factorial(n - 1)  # recursive case
```

Trace `factorial(4)`:

```
factorial(4) = 4 * factorial(3)  
             = 4 * (3 * factorial(2))  
             = 4 * (3 * (2 * factorial(1)))  
             = 4 * 3 * 2 * 1 = 24
```

★ IBPS Exam Point

Every recursion **must** have a **base case** to terminate. The recursive call must progress toward the base case.

⚠ Examiner's Trap

Missing or unreachable base case → **infinite recursion** → **stack overflow** (not an "infinite loop" in the normal sense — it exhausts the call stack).

8.2 Recursion Uses the Stack

Each recursive call is placed on the **call stack** with its own variables and return address. Calls stack up until the base case, then they are removed (unwound) in **LIFO** order.

★ IBPS Exam Point

Recursion depth = number of stacked calls = **O(depth)** extra space on the call stack. Deep recursion can cause **stack overflow**.

8.3 Types of Recursion (awareness level)

- **Direct recursion:** a function calls itself directly.
- **Indirect recursion:** function A calls B, and B calls A.

- **Tail recursion:** the recursive call is the **last** operation; some compilers optimise it to use constant stack space.

8.4 Recursion vs Iteration

Feature	Recursion	Iteration (loops)
Approach	Function calls itself	Repeats using a loop
Memory	Uses call stack (more)	Uses $O(1)$ extra (usually)
Speed	Overhead of calls	Generally faster
Readability	Cleaner for tree/divide problems	Cleaner for simple counting
Risk	Stack overflow	Infinite loop

★ IBPS Exam Point

Any recursion can be rewritten as iteration and vice versa. Recursion trades **extra memory (stack)** for **cleaner code**.

⚠ Examiner's Trap

Naive recursive **Fibonacci is $O(2^n)$** because each call spawns two calls. The iterative version is **$O(n)$** . (Repeated from Chapter 2 — it is asked often.)

8.5 Chapter 8 — Quick Revision Box

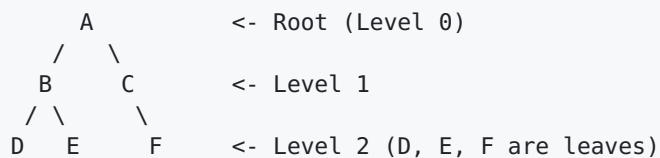
- Recursion = function calling itself; needs a **base case** + recursive case.
- Uses the **call stack**; deep recursion → stack overflow.
- Direct / indirect / tail recursion.
- Naive Fibonacci = $O(2^n)$; factorial recursion = $O(n)$ time, $O(n)$ stack space.

CHAPTER 9 — TREES

Trees are the first **non-linear** structure. Learn the terminology precisely — most IBPS tree questions test definitions and formulas.

9.1 Tree Terminology (learn every term)

Consider this tree:



Term	Meaning	In the example
Node	An element of the tree	A, B, C, D, E, F
Edge	A link connecting two nodes	A-B, B-D, etc.
Root	Topmost node (no parent)	A
Parent	Node with children	A is parent of B, C
Child	Node under a parent	B, C are children of A
Sibling	Nodes sharing the same parent	D and E (both children of B)
Leaf (external)	Node with no children	D, E, F
Internal node	Node with at least one child	A, B, C
Degree of a node	Number of children it has	Degree of A = 2
Level	Distance from root (root = level 0)	B is at level 1
Height of tree	Longest path (edges) from root to a leaf	2
Depth of a node	Edges from root to that node	Depth of D = 2
Subtree	A node with all its descendants	B, D, E form a subtree
Ancestor	Any node on the path up to the root	A is ancestor of D
Descendant	Any node below a given node	D is descendant of A

⚠ Examiner's Trap

- **Level** starts at **0** for the root in the standard convention (some books use 1 — read the question).
- **Height** is measured in **edges** (longest root-to-leaf path); a single-node tree has height **0**.
- **Degree of a tree** = the maximum degree among all its nodes.

9.2 Binary Tree and Its Types

A **binary tree** is a tree in which **each node has at most 2 children** (left child and right child).

Type	Definition
Full (strict) binary tree	Every node has either 0 or 2 children (never exactly 1)
Complete binary tree	All levels filled except possibly the last, which is filled left to right
Perfect binary tree	All internal nodes have 2 children and all leaves are at the same level
Skewed binary tree	Every node has only one child (left-skewed or right-skewed) — behaves like a linked list
Balanced binary tree	Height kept to about $\log n$ (e.g., AVL)

★ IBPS Exam Point (definitions)

- **Full** = 0 or 2 children per node.
- **Complete** = filled level by level, last level left-aligned.
- **Perfect** = full **and** all leaves on the same level.

9.3 Binary Tree Formulas (memorise — direct MCQs)

Using the convention **root at level 0**:

1. **Maximum nodes at level $l = 2^l$.**
2. **Maximum nodes in a binary tree of height $h = 2^{(h+1)} - 1$.**
3. **Minimum height** of a binary tree with n nodes = $\lfloor \log_2 n \rfloor$ (roughly).
4. In a **full binary tree**, if there are L leaf nodes and I internal nodes, then $L = I + 1$.
5. A binary tree with n nodes has exactly $n - 1$ edges.
6. Number of nodes in a **perfect** binary tree of height $h = 2^{(h+1)} - 1$; number of leaves = 2^h .

Worked Example

A perfect binary tree has height $h = 3$. How many total nodes and leaves?

Total nodes = $2^{(3+1)} - 1 = 16 - 1 = 15$
 Leaves = $2^3 = 8$

⚠ Examiner's Trap

Formula $2^{(h+1)} - 1$ uses the convention **height = edges** with root at level 0. If a question defines height as **number of levels**, adjust by one. Always check the convention stated.

9.4 Tree Traversals (very high-frequency)

Traversal = visiting every node exactly once in a defined order. There are two families:

Depth-First (Inorder, Preorder, Postorder) and **Breadth-First** (Level-order).

Use this tree for all examples:



Preorder (Root → Left → Right)

Visit node first, then left subtree, then right subtree.

Preorder = A B D E C F

Inorder (Left → Root → Right)

Left subtree, then node, then right subtree.

Inorder = D B E A C F

Postorder (Left → Right → Root)

Left subtree, then right subtree, then node.

Postorder = D E B F C A

Level-order (Breadth-First, uses a Queue)

Visit level by level, top to bottom, left to right.

Level-order = A B C D E F

◆ Memory Trick

The **position of "Root"** in the name tells you when to visit the node: - **Preorder** → Root **first** (Pre = before). - **Inorder** → Root **in the middle**. - **Postorder** → Root **last** (Post = after). Left is always visited before Right.

★ IBPS Exam Point

- Preorder = Root, Left, Right.
- Inorder = Left, Root, Right.
- Postorder = Left, Right, Root.
- Level-order uses a **queue** (BFS). DFS traversals can use a **stack**/recursion.

⚠ Examiner's Trap

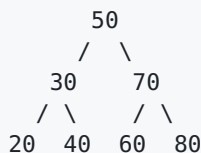
Inorder traversal of a Binary Search Tree gives elements in sorted (ascending) order — a favourite question (see 9.5). This is NOT true for a general binary tree, only for a BST.

9.5 Binary Search Tree (BST)

Definition

A **BST** is a binary tree where for **every** node: - All values in the **left subtree** are **less than** the node's value. - All values in the **right subtree** are **greater than** the node's value.

Example



Inorder of a BST = Sorted Order

Inorder = 20 30 40 50 60 70 80 (ascending!)

Operations & Complexity

Operation	Balanced BST	Skewed (worst) BST
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Search idea: compare target with node; go **left** if smaller, **right** if larger — like binary search on a tree.

★ IBPS Exam Point

- BST rule: **left < node < right**.
- **Inorder traversal of a BST is sorted ascending.**
- BST operations are $O(\log n)$ only if **balanced**; a **skewed** BST degrades to $O(n)$.

⚠ Examiner's Trap

Inserting **sorted data** into an ordinary BST creates a **skewed** tree (like a linked list) with $O(n)$ operations. This is exactly why balanced trees (AVL) exist — covered next.

9.6 Chapter 9 — Quick Revision Box

- Root = level 0; height in edges; leaf = no children; full = 0 or 2 children.
- Max nodes at level $l = 2^l$; max nodes for height $h = 2^{(h+1)} - 1$; n nodes $\rightarrow n-1$ edges.
- Full tree: leaves = internal + 1.
- Traversals: Pre (RLR-root first), In (LRootR), Post (root last), Level (queue/BFS).
- BST: left < node < right; **inorder = sorted**; balanced $O(\log n)$, skewed $O(n)$.

CHAPTER 10 — BALANCED & SPECIAL TREES

10.1 Why Balanced Trees?

A plain BST can become **skewed** ($O(n)$). **Balanced trees** automatically keep their height near **$\log n$** , guaranteeing $O(\log n)$ operations. The most common exam example is the **AVL tree**.

10.2 AVL Tree

Definition

An **AVL tree** is a **self-balancing BST** where, for every node, the **balance factor** is **-1, 0, or +1**.

$$\text{Balance Factor (BF)} = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$$

If any node's BF becomes **+2 or -2** after insertion/deletion, the tree performs a **rotation** to rebalance.

The Four Rotation Cases

Case	When it happens	Fix
LL (Left-Left)	Inserted into the left subtree of the left child (BF = +2, child BF = +1)	Single Right rotation
RR (Right-Right)	Inserted into the right subtree of the right child (BF = -2, child BF = -1)	Single Left rotation
LR (Left-Right)	Inserted into the right subtree of the left child	Left rotation on child, then Right rotation
RL (Right-Left)	Inserted into the left subtree of the right child	Right rotation on child, then Left rotation

Worked Example — RR case (single left rotation)

Insert 10, then 20, then 30:

```
Insert 10:  10
Insert 20:  10
              \
              20
Insert 30:  10              BF(10) = 0 - 2 = -2 → RR imbalance
              \
              20
               \
               30

Left rotation about 10:
      20
     /  \
    10   30    ✓ balanced (all BF = 0)
```

Worked Example — LL case (single right rotation)

Insert 30, 20, 10:

```
30          BF(30) = +2 → LL imbalance
/
20
/
10

Right rotation about 30:
  20
 /  \
10   30    ✓ balanced
```

Worked Example — LR case (double rotation)

Insert 30, 10, 20:

```
30          BF(30)=+2, but left child 10 is right-heavy → LR case
/
10
 \
  20

Step 1 - Left rotation on 10:      Step 2 - Right rotation on 30:
  30                                20
 /  \                              /  \
20   10                          10   30    ✓ balanced
/
10
```

★ IBPS Exam Point

- AVL balance factor $\in \{-1, 0, +1\}$.

- **LL → single right rotation; RR → single left rotation; LR → left then right; RL → right then left.**
- AVL guarantees **$O(\log n)$** search/insert/delete.

⚠ Examiner's Trap

- **LL is fixed by a RIGHT rotation** and **RR by a LEFT rotation** — the fix direction is **opposite** to the case name. This reversal is the most common mistake.
- Balance factor is **left height – right height** (not right – left) in the standard convention.

◆ Memory Trick

"Heavy on Left → rotate Right; heavy on Right → rotate Left." You rotate **away** from the heavy side.

10.3 B-Tree and B+ Tree (awareness level)

These are **multi-way** balanced trees used in **databases and file systems** (indexes), where a node can have **many** children — designed to minimise disk reads.

Feature	B-Tree	B+ Tree
Data stored	In internal and leaf nodes	Only in leaf nodes
Leaf linkage	Not linked	Leaves linked (fast range queries)
Use	General indexing	Database indexing (most common)

★ IBPS Exam Point

- **B-tree / B+ tree** are used for **database indexing** and file systems (they reduce disk I/O).
- In a **B+ tree**, all actual data/records are in the **leaf** nodes, and leaves are **linked** for range scans.

10.4 Heap

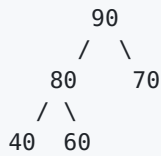
Definition

A **heap** is a **complete binary tree** that satisfies the **heap property**: - **Max-Heap**: every parent $\geq$ its children → the **root is the maximum**. - **Min-Heap**: every parent $\leq$ its children → the **root is the minimum**.

Array Representation

A heap is stored in an array. For a node at index **i** (0-based): - **Left child** = index **$2i + 1$** - **Right child** = index **$2i + 2$** - **Parent** = index **$\lfloor (i - 1) / 2 \rfloor$**

Example — Max-Heap



Array: [90, 80, 70, 40, 60]
Root (index 0) = 90 = maximum.

Operations & Complexity

Operation	Complexity
Insert	$O(\log n)$
Delete (extract root)	$O(\log n)$
Get max/min (peek root)	$O(1)$
Build heap from array	$O(n)$
Heap Sort	$O(n \log n)$

Applications

- **Priority queues** (Chapter 7).
- **Heap sort** (Chapter 13).
- **Dijkstra's shortest path** (uses a min-priority queue).

★ IBPS Exam Point

- **Max-heap root = largest; min-heap root = smallest.**
- A heap is a **complete binary tree** stored in an **array**; children of index i are at $2i+1$ and $2i+2$.
- Insert/delete = $O(\log n)$; peek = $O(1)$; build-heap = $O(n)$.

⚠ Examiner's Trap

A heap is **NOT** a sorted structure and is **NOT** a BST. It only guarantees the parent-child order along each path, not left-to-right order. Also note **build-heap is $O(n)$** , not $O(n \log n)$.

10.5 Chapter 10 — Quick Revision Box

- AVL = self-balancing BST; $BF \in \{-1, 0, +1\}$; $O(\log n)$ guaranteed.
- LL→right, RR→left, LR→left-then-right, RL→right-then-left. Rotate away from the heavy side.
- B-tree/B+ tree → database indexing; B+ stores data only in linked leaves.
- Heap = complete binary tree in an array; max-heap root = max; children at $2i+1$, $2i+2$.
- Heap: insert/delete $O(\log n)$, peek $O(1)$, build $O(n)$. A heap is not sorted and not a BST.

CHAPTER 11 — GRAPHS

11.1 What is a Graph?

Simple Meaning

A **graph** is a non-linear structure made of **vertices (nodes)** connected by **edges (links)**. It represents **relationships** between objects.

Real-Life Analogy

A **map of cities**: each city is a vertex, each road between two cities is an edge. Or a **social network**: people are vertices, friendships are edges.

Basic Terminology

- **Vertex (node)**: a point in the graph.
- **Edge**: a connection between two vertices.
- **Degree**: number of edges touching a vertex.
- **In-degree / Out-degree**: (directed graphs) edges coming in / going out.
- **Path**: a sequence of vertices connected by edges.
- **Cycle**: a path that starts and ends at the same vertex.

Notation

A graph **G = (V, E)** where V = set of vertices, E = set of edges.

11.2 Types of Graphs

Type	Meaning
Undirected	Edges have no direction (A-B same as B-A)
Directed (Digraph)	Edges have direction ($A \rightarrow B \neq B \rightarrow A$)
Weighted	Edges carry a value/cost (e.g., distance)
Unweighted	Edges have no weight
Cyclic	Contains at least one cycle
Acyclic	No cycles (e.g., a DAG — Directed Acyclic Graph)
Connected	A path exists between every pair of vertices
Disconnected	Some vertices are unreachable from others

★ IBPS Exam Point

- A **tree** is a special graph: **connected + acyclic** with **$n-1$ edges** for n vertices.
- **DAG** = Directed Acyclic Graph (used in scheduling, dependency resolution).

11.3 Graph Representation

(A) Adjacency Matrix

A 2D matrix of size $V \times V$. Cell $[i][j] = 1$ (or the weight) if there is an edge from i to j , else 0. - **Space:** $O(V^2)$ — wasteful for sparse graphs. - **Edge lookup:** $O(1)$ — instant "is there an edge?".

(B) Adjacency List

Each vertex stores a **list of its neighbours**. - **Space:** $O(V + E)$ — efficient for sparse graphs. - **Edge lookup:** $O(\text{degree})$ — must scan the list.

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Check edge (i,j)	$O(1)$	$O(\text{degree})$
Best for	Dense graphs	Sparse graphs

★ IBPS Exam Point

- **Adjacency matrix = $O(V^2)$ space**, $O(1)$ edge check — good for **dense** graphs.
- **Adjacency list = $O(V+E)$ space** — good for **sparse** graphs.

11.4 Graph Traversals

Breadth-First Search (BFS)

Explores **level by level** — all neighbours first, then their neighbours. **Uses a QUEUE**. - Time: $O(V + E)$ with adjacency list. - Finds the **shortest path in an unweighted graph**.

Depth-First Search (DFS)

Goes **as deep as possible** along one branch before backtracking. **Uses a STACK (or recursion)**. - Time: $O(V + E)$ with adjacency list. - Used for cycle detection, topological sorting, connected components.

★ IBPS Exam Point

- **BFS → Queue; DFS → Stack/recursion.**
- Both are $O(V + E)$ with an adjacency list.
- **BFS** gives the shortest path in an **unweighted** graph.

◆ Memory Trick

BFS = **B**road (wide, level by level, Queue). **DFS** = **D**eep (dive down, Stack).

11.5 Shortest Path & Minimum Spanning Tree (awareness)

Algorithm	Purpose	Note
Dijkstra's	Shortest path from one source (weighted, non-negative)	Uses a min-priority queue (heap)
Bellman-Ford	Shortest path, handles negative weights	Slower than Dijkstra
Prim's	Minimum Spanning Tree (MST)	Grows one tree
Kruskal's	Minimum Spanning Tree (MST)	Picks smallest edges, avoids cycles

MST = a tree connecting all vertices with the **minimum total edge weight** (has exactly **$V-1$** edges).

★ IBPS Exam Point

- **Dijkstra** = single-source shortest path (no negative weights).
- **Prim's & Kruskal's** build the **MST**; an MST has **$V-1$** edges.
- **Bellman-Ford** handles negative edge weights.

⚠ Examiner's Trap

Dijkstra's algorithm fails with negative edge weights — use **Bellman-Ford** instead.

11.6 Chapter 11 — Quick Revision Box

- Graph $G=(V,E)$; directed/undirected, weighted/unweighted, cyclic/acyclic.
- Tree = connected acyclic graph with $n-1$ edges.
- Adjacency matrix $O(V^2)$, $O(1)$ edge check; adjacency list $O(V+E)$.
- BFS→queue (shortest path unweighted); DFS→stack; both $O(V+E)$.
- Dijkstra (no negatives), Bellman-Ford (negatives), Prim/Kruskal (MST, $V-1$ edges).

CHAPTER 12 — HASHING

12.1 What is Hashing?

Simple Meaning

Hashing converts a key into an array index using a **hash function**, so we can store and find data in (on average) **O(1)** time.

Real-Life Analogy

A **library book locator**: a formula turns a book's ID into a shelf number instantly, so you go straight to the shelf instead of searching every book.

Basic Terminology

- **Hash table**: the array that stores the data.
- **Key**: the value we search by.
- **Hash function**: the formula that maps a key to an index (e.g., $\text{index} = \text{key} \% \text{table_size}$).
- **Bucket/slot**: a position in the hash table.
- **Collision**: when two different keys map to the **same** index.
- **Load factor (α)**: $= (\text{number of items}) / (\text{table size})$. Higher load factor $\rightarrow$ more collisions.

Example

Table size = 10, hash function = $\text{key} \% 10$.

```
key 25  $\rightarrow 25 \% 10 = 5$   $\rightarrow$  store at index 5  
key 33  $\rightarrow 33 \% 10 = 3$   $\rightarrow$  store at index 3  
key 15  $\rightarrow 15 \% 10 = 5$   $\rightarrow$  COLLISION (index 5 taken)
```

★ IBPS Exam Point

Hashing gives **O(1) average** search/insert/delete. A common hash function is **key mod table_size**. Load factor = items / table size.

12.2 Collision Resolution Techniques

Collisions are unavoidable, so we need strategies to handle them.

(A) Separate Chaining (Open Hashing)

Each slot holds a **linked list** of all keys that hash there. Colliding keys are appended to the list. - Simple; table never "fills up". - Search time depends on chain length.

(B) Open Addressing (Closed Hashing)

All keys are stored **inside the table itself**; on a collision, probe for the next free slot.

Method	Probe sequence
Linear Probing	Try index+1, index+2, ... (may cause primary clustering)
Quadratic Probing	Try index+1 ² , index+2 ² , index+3 ² , ... (reduces primary clustering)
Double Hashing	Use a second hash function to decide the step size (best distribution)

Linear Probing Example (continuing above)

key 15 → $15 \% 10 = 5$ → index 5 taken (25) → try 6 → free → store 15 at index 6

★ IBPS Exam Point

- **Separate chaining** uses **linked lists** at each slot (open hashing).
- **Open addressing** stores everything in the table (linear/quadratic probing, double hashing).
- **Linear probing** suffers from **primary clustering**; **double hashing** gives the best spread.

⚠ Examiner's Trap

Terminology is reversed and confusing: "**Open hashing**" = **separate chaining**, while "**Closed hashing**" = **open addressing**. Read carefully. Also, hashing worst case is **O(n)** (all keys collide into one chain), even though the **average** is O(1).

12.3 Chapter 12 — Quick Revision Box

- Hashing → key mapped to index via hash function; O(1) average, O(n) worst.
- Common hash function: $\text{key} \bmod \text{table_size}$. Load factor = items / size.
- Collision = two keys, same index.
- Separate chaining (linked lists) = open hashing.
- Open addressing = linear/quadratic probing, double hashing = closed hashing.
- Linear probing → primary clustering; double hashing → best distribution.

CHAPTER 13 — SEARCHING & SORTING

The last DSA chapter, and one of the most heavily tested. IBPS asks pass-by-pass questions, so every trace below is worked step by step.

13.1 Searching Algorithms

Linear Search

Check each element one by one until found or the list ends. - No sorting needed. Best $O(1)$, Average/Worst $O(n)$.

Binary Search

Works only on **sorted** data. Repeatedly compare with the **middle** element and discard half. - Best $O(1)$, Average/Worst $O(\log n)$.

Binary Search — Worked Trace

Find **60** in `[10, 20, 30, 40, 50, 60, 70]` (indices 0–6):

```
low=0, high=6 → mid=3 → A[3]=40 < 60 → search right → low=4  
low=4, high=6 → mid=5 → A[5]=60 = target → FOUND at index 5
```

Only 2 comparisons for 7 elements — that is the power of $O(\log n)$.

★ IBPS Exam Point

- **Binary search requires SORTED data**; $O(\log n)$.
- Linear search works on **unsorted** data; $O(n)$.

⚠ Examiner's Trap

Binary search on **unsorted** data gives **wrong** results. Sorting is a prerequisite. Also

`mid = low + (high - low)/2` avoids overflow (vs `(low+high)/2`).

13.2 Sorting — Key Properties First

- **Stable sort**: equal elements keep their original relative order (Bubble, Insertion, Merge are stable).
- **In-place sort**: uses $O(1)$ extra memory (Bubble, Selection, Insertion, Heap are in-place; Merge is not).
- **Adaptive sort**: faster on nearly-sorted data (Insertion, optimised Bubble).

We sort the sample array **[5, 1, 4, 2, 8]** in ascending order for each method.

13.3 Bubble Sort

Repeatedly compare **adjacent** pairs and swap if out of order. After each pass, the largest remaining element "bubbles" to the end.

Pass-by-Pass Trace on [5, 1, 4, 2, 8]

```
Pass 1: (compare adjacent)
[5,1,4,2,8] → 5>1 swap → [1,5,4,2,8]
           → 5>4 swap → [1,4,5,2,8]
           → 5>2 swap → [1,4,2,5,8]
           → 5<8 no   → [1,4,2,5,8]    (8 now fixed at end)

Pass 2:
[1,4,2,5,8] → 1<4 no; 4>2 swap → [1,2,4,5,8]; 4<5 no → [1,2,4,5,8]

Pass 3:
[1,2,4,5,8] → no swaps → already sorted → STOP
```

- Best **$O(n)$** (already sorted, optimised with a swap flag), Average/Worst **$O(n^2)$** . Stable, in-place.

★ IBPS Exam Point

After **pass k** of bubble sort, the **k largest** elements are in their final positions at the end.

13.4 Selection Sort

Find the **minimum** of the unsorted part and place it at the front.

Pass-by-Pass Trace on [5, 1, 4, 2, 8]

```
Pass 1: min of [5,1,4,2,8] = 1 → swap to front → [1,5,4,2,8]
Pass 2: min of [5,4,2,8]   = 2 → swap          → [1,2,4,5,8]
Pass 3: min of [4,5,8]     = 4 → already placed → [1,2,4,5,8]
Pass 4: min of [5,8]       = 5 → already placed → [1,2,4,5,8]
```

- **$O(n^2)$** in all cases (best, average, worst). In-place, **not stable**. Always does the same number of comparisons.

★ IBPS Exam Point

Selection sort makes the **fewest swaps** (at most $n-1$) but always $O(n^2)$ comparisons. After pass k, the **k smallest** elements are placed at the front.

13.5 Insertion Sort

Build the sorted portion one element at a time by **inserting** each new element into its correct place.

Pass-by-Pass Trace on [5, 1, 4, 2, 8]

```
Start: [5 | 1,4,2,8] (5 is the sorted part)
Insert 1: [1,5 | 4,2,8]
Insert 4: [1,4,5 | 2,8]
Insert 2: [1,2,4,5 | 8]
Insert 8: [1,2,4,5,8] ✓
```

- Best **$O(n)$** (nearly sorted), Average/Worst **$O(n^2)$** . Stable, in-place, **adaptive**.

★ IBPS Exam Point

Insertion sort is **best for small or nearly-sorted arrays** (adaptive, $O(n)$ best case).

13.6 Merge Sort

Divide and conquer: split the array in half, sort each half recursively, then **merge** the two sorted halves.

Worked Trace on [5, 1, 4, 2]

```
Divide: [5,1,4,2] → [5,1] and [4,2]
        [5,1] → [5] [1]      [4,2] → [4] [2]
Merge:  [5]+[1] → [1,5]      [4]+[2] → [2,4]
Merge:  [1,5] + [2,4] → [1,2,4,5] ✓
```

- **$O(n \log n)$** in **all** cases. **Stable**. Needs **$O(n)$ extra space** (not in-place).

★ IBPS Exam Point

Merge sort is **$O(n \log n)$ guaranteed** but needs **$O(n)$ extra memory**. Preferred when stability and guaranteed performance matter (e.g., external sorting of large files).

13.7 Quick Sort

Divide and conquer: pick a **pivot**, **partition** so smaller elements go left and larger go right, then recursively sort each side. Sorting happens **in place**.

One Partition Step (pivot = last element)

Array [3, 1, 4, 2], pivot = 2:

Move elements < 2 to the left of pivot:
 $1 < 2 \rightarrow$ keep left; $3, 4 > 2 \rightarrow$ keep right
 After partition: $[1, 2, 4, 3] \rightarrow$ pivot 2 is now in its FINAL sorted position (index 1)
 Then recursively sort left $[1]$ and right $[4, 3] \rightarrow [3, 4]$
 Final: $[1, 2, 3, 4]$

- Best/Average $O(n \log n)$, Worst $O(n^2)$ (bad pivot, e.g., already-sorted with first/last pivot). **In-place, not stable.** $O(\log n)$ average stack space.

★ IBPS Exam Point

- Quick sort is usually the **fastest in practice** and **in-place**, but its **worst case is $O(n^2)$** .
- After one partition, the **pivot sits in its final sorted position**.

⚠ Examiner's Trap

Quick sort's worst case $O(n^2)$ occurs when the pivot is always the smallest/largest (e.g., sorted input with a poor pivot choice). A **randomised** or median pivot avoids this.

13.8 Heap Sort

Build a **max-heap**, then repeatedly swap the root (maximum) to the end and re-heapify the rest. - $O(n \log n)$ in all cases. **In-place** ($O(1)$ extra). **Not stable**.

★ IBPS Exam Point

Heap sort = $O(n \log n)$ guaranteed **and** in-place (unlike merge sort). But it is **not stable**.

13.9 Sorting Master Comparison (final recap)

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

◆ Memory Trick

Guaranteed $O(n \log n)$: **Merge & Heap**. Worst-case $O(n^2)$ despite good average: **Quick**. Always $O(n^2)$: **Selection**.

13.10 Chapter 13 — Quick Revision Box

- Binary search needs sorted data $\rightarrow O(\log n)$; linear search $O(n)$.
- Bubble/Selection/Insertion = $O(n^2)$ average; Merge/Heap = $O(n \log n)$ guaranteed; Quick avg $O(n \log n)$, worst $O(n^2)$.
- Stable: Bubble, Insertion, Merge. In-place: all except Merge.
- After pass k : bubble $\rightarrow k$ largest at end; selection $\rightarrow k$ smallest at front.
- Quick sort: after partition, pivot is in final position; in-place but worst $O(n^2)$.
- Merge sort: stable, $O(n)$ extra space. Heap sort: in-place, not stable.

(End of Part A — Data Structures & Algorithms)

PART B — OBJECT-ORIENTED PROGRAMMING (OOP)

All code examples use Java as the primary language, since IBPS SO IT output-based OOP questions are overwhelmingly Java-oriented. Python-specific behaviour is clearly labelled where it differs.

CHAPTER 14 — OOP FOUNDATIONS

We start from zero. Before "objects" make any sense, you must know a few basics.

14.1 Prerequisites (zero-knowledge)

- **Program:** a set of instructions that tells a computer what to do.
- **Variable:** a named container that stores a value (e.g., `age = 20`).
- **Function:** a named block of reusable instructions that performs a task and can return a result.
- **Method:** a function that **belongs to a class/object** (in OOP, we call such functions "methods").
- **Method call:** the act of running/invoking a method (e.g., `s.display()`).

★ IBPS Exam Point

A **method is a function defined inside a class**. Every method is a function, but in OOP we specifically say "method" because it belongs to an object/class.

14.2 What is Object-Oriented Programming?

Simple Meaning

OOP is a way of writing programs by modelling **real-world things as "objects"** that bundle **data** (attributes) and **behaviour** (methods) together.

Real-Life Analogy

Think of a **car**. It has **data** (colour, speed, fuel) and **behaviours** (start, accelerate, brake). In OOP, we'd model a `Car` as an object that holds those data values and can perform those actions.

Two Core Concepts: Class and Object

- **Class:** a **blueprint / template** that defines what data and methods an object will have. It does not occupy memory for data by itself.
- **Object:** a **real instance** created from a class. It occupies memory and holds actual values.

Real-Life Analogy

A **class is like an architect's blueprint** for a house. An **object is an actual house** built from that blueprint. One blueprint (class) → many houses (objects).

Java Example

```
// CLASS = blueprint
class Student {
    int rollNo;           // attribute / field / instance variable
    String name;          // attribute

    void display() {      // method (behaviour)
        System.out.println(rollNo + " " + name);
    }
}

// Using the class
Student s1 = new Student(); // OBJECT creation (instance)
s1.rollNo = 1;              // setting data
s1.name = "Asha";
s1.display();               // METHOD CALL → prints: 1 Asha
```

Basic Terminology

- **Attribute / Field / Instance variable:** the data inside an object (`rollNo` , `name`).
- **Method:** the behaviour (`display()`).
- **new keyword:** creates an object (allocates memory) in Java.
- **Instance:** another word for an object created from a class.

★ IBPS Exam Point

- **Class = blueprint (no memory for data on its own); Object = instance (occupies memory).**
- In Java, objects are created with the **new** keyword.

⚠ Examiner's Trap

A class by itself doesn't store data values — only **objects** do. Many objects can be made from **one** class, each with its own copy of instance variables.

◆ Memory Trick

Class = Cookie cutter; Object = the actual cookies. One cutter, many cookies.

14.3 Procedural (POP) vs Object-Oriented (OOP)

Feature	Procedural (POP) — e.g., C	Object-Oriented (OOP) — e.g., Java
Basic unit	Function	Object
Focus	Functions/procedures	Data + behaviour together
Data security	Data is often global, less secure	Data hidden inside objects (encapsulation)
Approach	Top-down	Bottom-up
Reusability	Limited	High (inheritance)
Examples	C, Pascal, FORTRAN	Java, C++, Python, C#

★ IBPS Exam Point

POP is **function-centric** and **top-down**; OOP is **object-centric**, **bottom-up**, with better **data security and reusability**.

14.4 Advantages of OOP

- **Modularity** — code organised into self-contained objects.
- **Reusability** — via inheritance.
- **Security** — via encapsulation (data hiding).
- **Maintainability** — easier to modify and extend.
- **Real-world modelling** — natural mapping of problems to objects.

14.5 Chapter 14 — Quick Revision Box

- OOP models real things as objects bundling data + behaviour.
- Class = blueprint (no data memory alone); Object = instance (has memory), made with `new`.
- Method = function inside a class; attribute = data inside an object.
- POP: function-centric, top-down (C). OOP: object-centric, bottom-up (Java).
- One class → many objects, each with its own instance variables.

CHAPTER 15 — THE FOUR PILLARS OF OOP (OVERVIEW)

All of OOP rests on **four pillars**. This chapter gives the big picture; the next chapters go deep into each.

15.1 The Four Pillars

Pillar	One-line meaning	Real-life analogy
Encapsulation	Bundling data + methods together and hiding internal data	A medicine capsule hides the powder inside
Abstraction	Showing only essential features , hiding complex details	Driving a car without knowing the engine internals
Inheritance	A class acquires properties of another class	A child inherits traits from parents
Polymorphism	One name/interface, many forms of behaviour	The word "run" means different things in different contexts

◆ Memory Trick

"**A PIE**" → **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation. The four pillars = **A PIE**.

★ IBPS Exam Point

The four pillars are **Encapsulation, Abstraction, Inheritance, Polymorphism**. (Remember **A-PIE**.)

⚠ Examiner's Trap

"Class" and "Object" are **fundamental concepts** but are **NOT** counted among the four pillars. Also, some questions list "message passing" — that is a concept, not one of the four core pillars.

15.2 How the Pillars Relate

- **Encapsulation** protects data; **Abstraction** hides complexity — they are related but different (Chapter 16 distinguishes them).
- **Inheritance** enables reuse and sets up **Polymorphism** (overriding).
- **Polymorphism** lets the same call behave differently depending on the object.

15.3 Chapter 15 — Quick Revision Box

- Four pillars = **A PIE**: Abstraction, Polymorphism, Inheritance, Encapsulation.
- Class and Object are core concepts but **not** pillars.
- Encapsulation = bundle + hide data; Abstraction = hide complexity; Inheritance = reuse; Polymorphism = many forms.

CHAPTER 16 — ENCAPSULATION & ABSTRACTION

These two are often confused. Learn each clearly, then the difference.

16.1 Encapsulation

Simple Meaning

Encapsulation = wrapping **data (variables)** and the **methods** that work on them into a single unit (a class), and **restricting direct access** to the data.

Real-Life Analogy

A **medicine capsule** encloses the powder inside a shell. You use it without touching the contents directly. Similarly, a class hides its data and exposes controlled methods.

How It's Done in Java (Data Hiding)

1. Make the fields **private** (cannot be accessed from outside).
2. Provide **public getter and setter** methods to read/change them in a controlled way.

```
class Account {  
    private double balance;           // hidden data  
  
    public double getBalance() {      // getter (controlled read)  
        return balance;  
    }  
    public void setBalance(double amt) { // setter (controlled write)  
        if (amt >= 0) balance = amt;  // validation possible here  
    }  
}
```

Now outside code **cannot** do `account.balance = -500;` directly — it must go through `setBalance`, which can reject invalid values. This is **data hiding**.

★ IBPS Exam Point

Encapsulation = data + methods bundled + data hidden using `private`, accessed via public getters/setters. It provides **data security**.

◆ Memory Trick

Encapsulation = Capsule → data sealed inside, accessed only through approved openings (methods).

16.2 Abstraction

Simple Meaning

Abstraction = showing only the **essential features** (the "what") and hiding the **complex implementation** (the "how").

Real-Life Analogy

When you **drive a car**, you use the steering wheel, accelerator, and brake. You don't need to know how the engine's combustion works. The complex details are **abstracted** away.

How It's Done in Java

- **Abstract classes** (with abstract methods) and **interfaces** (covered in detail in Chapter 19) provide abstraction by declaring **what** must be done without specifying **how**.

```
abstract class Shape {  
    abstract double area();    // WHAT: every shape has an area  
}  
                                // HOW is left to each subclass
```

★ IBPS Exam Point

Abstraction = hiding implementation, exposing only essential behaviour. Achieved via **abstract classes and interfaces**.

16.3 Encapsulation vs Abstraction (key distinction — asked often)

Aspect	Encapsulation	Abstraction
Purpose	Data hiding (protect data)	Complexity hiding (hide implementation)
Focus	How data is protected/wrapped	What an object does, not how
Achieved by	private fields + getters/setters	Abstract classes + interfaces
Level	Implementation level	Design level
Analogy	Capsule sealing the medicine	Car controls hiding the engine

⚠ Examiner's Trap

- **Encapsulation hides DATA; Abstraction hides IMPLEMENTATION (complexity).** Do not swap these.
- Encapsulation is achieved with **access modifiers (private)**; abstraction with **abstract classes/interfaces**. Mixing up the mechanisms is a common wrong answer.

◆ Memory Trick

Encapsulation → **Encloses data**. Abstraction → **Abstracts away details/complexity**.

16.4 Access Modifiers (quick intro; detailed in Chapter 21)

Encapsulation relies on **access modifiers** that control visibility:

Modifier	Accessible from
<code>private</code>	Same class only
<code>default</code> (no keyword)	Same package
<code>protected</code>	Same package + subclasses
<code>public</code>	Everywhere

★ IBPS Exam Point

Visibility order (least → most): **private < default < protected < public**.

16.5 Chapter 16 — Quick Revision Box

- Encapsulation = bundle data + methods, hide data with `private`, expose getters/setters → **data security**.
 - Abstraction = hide implementation, show essentials → via **abstract classes/interfaces**.
 - Encapsulation hides **data**; Abstraction hides **complexity/implementation**.
 - Access modifiers: `private < default < protected < public`.
-

CHAPTER 17 — INHERITANCE

17.1 What is Inheritance?

Simple Meaning

Inheritance lets one class (**child/subclass**) automatically **acquire the fields and methods** of another class (**parent/superclass**), so code can be reused and extended.

Real-Life Analogy

A **child inherits traits** (eye colour, surname) from parents, and can also add their own. In OOP, a subclass inherits from a superclass and can add or change behaviour.

Terminology

- **Superclass / Parent / Base class:** the class being inherited from.
- **Subclass / Child / Derived class:** the class that inherits.
- **extends:** the Java keyword for inheritance.
- **IS-A relationship:** inheritance models "is a kind of" (a Dog **is an** Animal).

Java Example

```
class Animal {                // superclass
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {    // subclass inherits eat()
    void bark() { System.out.println("Barking..."); }
}

Dog d = new Dog();
d.eat(); // inherited from Animal → "Eating..."
d.bark(); // own method           → "Barking..."
```

★ IBPS Exam Point

Inheritance is implemented with **extends** in Java and models an **IS-A** relationship. It promotes **code reusability**.

◆ Memory Trick

Inherit = Inherit from parents → child gets parent's features for free.

17.2 Types of Inheritance

Type	Structure	Supported in Java (classes)?
Single	A → B	Yes
Multilevel	A → B → C	Yes
Hierarchical	A → B, A → C (one parent, many children)	Yes
Multiple	B, C → D (one child, many parents)	NO (not with classes)
Hybrid	Combination involving multiple	NO (with classes)

★ IBPS Exam Point (critical)

Java does NOT support multiple inheritance using classes — a class cannot **extends** two classes. Java achieves multiple inheritance **only through interfaces** (a class can **implements** many interfaces — see Chapter 19).

The Diamond Problem

If class D inherited from both B and C (which both inherit from A), and both B and C override a method, D wouldn't know which version to use — this ambiguity is the **diamond problem**. To avoid it, Java bans multiple inheritance of classes.

⚠ Examiner's Trap

"Java supports multiple inheritance" is **only true via interfaces**, not classes. If a question says "using classes", the answer is **not supported** (to avoid the diamond problem). **C++ does support** multiple inheritance of classes — don't confuse the two languages.

17.3 The `super` Keyword

`super` refers to the **immediate parent class**. It is used to: 1. Call the parent's constructor: `super();` 2. Call a parent method that the child has overridden: `super.method();` 3. Access a parent field hidden by the child.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    void sound() {
        super.sound();           // calls Animal's version
        System.out.println("Bark");
    }
}
// new Dog().sound() → "Some sound" then "Bark"
```

★ IBPS Exam Point

`super` accesses the **immediate parent's** members/constructor. `super()` (constructor call) must be the **first statement** in the child constructor.

17.4 Chapter 17 — Quick Revision Box

- Inheritance = subclass acquires superclass members via `extends`; models **IS-A**; enables reuse.
- Types: single, multilevel, hierarchical (all supported); multiple & hybrid **not** via classes in Java.
- Java multiple inheritance = only via **interfaces**; classes banned to avoid the **diamond problem**.
- C++ allows multiple inheritance of classes; Java does not.
- `super` = immediate parent; `super()` must be first line of child constructor.

CHAPTER 18 — POLYMORPHISM

The most important OOP chapter for **output-based** IBPS questions.

18.1 What is Polymorphism?

Simple Meaning

Polymorphism = "**many forms**". The same method name (or interface) behaves **differently** depending on context or object.

Real-Life Analogy

The word "**cut**" means different things: cut vegetables, cut hair, cut a deal. One word, many behaviours — depending on the context.

Two Types

1. **Compile-time (Static) Polymorphism** → **Method Overloading**.
 2. **Run-time (Dynamic) Polymorphism** → **Method Overriding**.
-

18.2 Method Overloading (Compile-time / Static)

Same method name, different parameter lists (different number or types of parameters), **within the same class**. The compiler decides which version to call based on the arguments — resolved at **compile time**.

```
class Calculator {  
    int add(int a, int b)          { return a + b; }  
    int add(int a, int b, int c)  { return a + b + c; }  
    double add(double a, double b) { return a + b; }  
}  
  
Calculator c = new Calculator();  
c.add(2, 3);           // calls first → 5  
c.add(2, 3, 4);        // calls second → 9  
c.add(2.5, 1.5);       // calls third → 4.0
```

⚠ Examiner's Trap

Overloading is decided **only by the parameter list** (count/type/order). You **cannot** overload by changing **only the return type** — that alone causes a compile error.

★ IBPS Exam Point

Method **overloading** = same name, different parameters, same class, resolved at **compile time** (static binding).

18.3 Method Overriding (Run-time / Dynamic)

A **subclass provides its own version** of a method that already exists in the **superclass**, with the **same signature** (same name + same parameters). The version that runs is decided at **run time** based on the **actual object type** — this is **dynamic method dispatch**.

```
class Animal {  
    void sound() { System.out.println("Animal makes a sound"); }  
}  
class Dog extends Animal {  
    void sound() { System.out.println("Dog barks"); }    // overrides  
}  
  
Animal a = new Dog();    // reference type Animal, object type Dog  
a.sound();               // OUTPUT: "Dog barks" (decided at run time)
```

Why "Dog barks"? Even though the **reference** is of type `Animal`, the **actual object** is a `Dog`. Overridden methods are chosen by the **object's real type at run time**, not the reference type.

★ IBPS Exam Point

Method **overriding** = same signature, subclass redefines superclass method, resolved at **run time** (dynamic binding / dynamic dispatch). The **object's actual type** decides which method runs.

⚠ Examiner's Trap

For **overridden (instance) methods**, the **object type** decides the method → "Dog barks". BUT for **fields (variables)** and **static methods**, the **reference type** decides (fields and statics are NOT polymorphic). This reference-vs-object distinction is one of the most common output-question traps.

18.4 Overloading vs Overriding (must-know table)

Feature	Overloading	Overriding
Where	Same class	Subclass vs superclass
Method signature	Different parameters	Same signature
Binding	Compile-time (static)	Run-time (dynamic)
Polymorphism type	Compile-time	Run-time
Return type	Can differ (with different params)	Same (or covariant)
Inheritance needed?	No	Yes

◆ Memory Trick

Over-LOAD → Lots of parameters (different) in the same class, decided early (compile time).

Over-RIDE → child **rides over** the parent's method, decided late (run time).

18.5 Static Binding vs Dynamic Binding

- **Static (early) binding:** the method call is resolved at **compile time**. Applies to **overloaded**, **static**, **private**, and **final** methods (they can't be overridden).
- **Dynamic (late) binding:** resolved at **run time**. Applies to **overridden** instance methods.

★ IBPS Exam Point

static, **private**, and **final** methods use **static binding** (cannot be overridden → cannot be dynamically dispatched). Normal overridden instance methods use **dynamic binding**.

⚠ Examiner's Trap

A **static** method in a subclass with the same signature is **method hiding**, NOT overriding (covered in Chapter 21). It follows the **reference type**, not the object type.

18.6 Chapter 18 — Quick Revision Box

- Polymorphism = many forms; compile-time (overloading) + run-time (overriding).
- Overloading: same name, different params, same class, compile-time.
- Overriding: same signature, subclass vs superclass, run-time, object type decides.
- Cannot overload by return type alone.
- Fields & static methods use **reference type**; overridden instance methods use **object type**.
- static/private/final → static binding (not overridable).

CHAPTER 19 — ABSTRACT CLASSES & INTERFACES

These are the tools that deliver **abstraction** (Chapter 16) and enable Java's form of **multiple inheritance**.

19.1 Abstract Class

Definition

An **abstract class** is a class declared with the **abstract** keyword that **cannot be instantiated** (you can't do `new` on it). It may contain: - **Abstract methods** (declared without a body — subclasses must implement them), **and/or** - **Concrete methods** (with a body), instance variables, and constructors.

```
abstract class Shape {
    abstract double area();           // abstract method (no body)
    void describe() {                 // concrete method
        System.out.println("I am a shape");
    }
}
class Circle extends Shape {
    double r = 5;
    double area() { return 3.14 * r * r; } // MUST implement area()
}
// new Shape(); // ERROR: cannot instantiate an abstract class
// new Circle(); // OK
```

★ IBPS Exam Point

- An **abstract class cannot be instantiated** but **can have constructors, fields, and both abstract & concrete methods**.
- A subclass **must implement all abstract methods**, otherwise the subclass must itself be declared abstract.

⚠ Examiner's Trap

An abstract class **can have a constructor** (called when a subclass object is created) and **can have non-abstract methods**. "An abstract class can have only abstract methods" is **false**.

19.2 Interface

Definition

An **interface** is a fully abstract "contract" of methods a class agrees to implement, using the **interface** keyword and **implements** in the class.

Key rules (standard Java): - Interface **variables** are implicitly **public static final** (constants). - Before **Java 8**, all interface methods were implicitly **public abstract** (no body). - **Java 8+** added **default** and **static** methods (which *can* have a body). **Java 9+** added **private** interface methods. - A class can **implement multiple interfaces** → this is how Java achieves **multiple inheritance**.

```
interface Drawable {
    int SIDES = 4;           // implicitly public static final
    void draw();             // implicitly public abstract
}
class Square implements Drawable {
    public void draw() { System.out.println("Drawing square"); }
}
```

★ IBPS Exam Point

- Interface variables are **public static final** (constants) by default.
- Interface methods are **public abstract** by default (pre-Java 8); Java 8+ allows **default** and **static** methods with bodies.
- A class uses **implements** for interfaces and can implement **many → multiple inheritance in Java**.

⚠ Examiner's Trap

- You **cannot instantiate** an interface directly.
- Interface fields are **always constants** (final) — you cannot reassign them.
- A class **extends** one class but **implements** many interfaces — don't mix the keywords.

19.3 Abstract Class vs Interface (high-frequency comparison)

Feature	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Instantiation	Not allowed	Not allowed
Methods	Abstract + concrete	Abstract (pre-8); default/static allowed (8+)
Variables	Any kind	<code>public static final</code> (constants) only
Constructor	Yes	No
Multiple inheritance	No (single <code>extends</code>)	Yes (implement many)
Keyword to use	<code>extends</code>	<code>implements</code>
When to use	Share common code + partial abstraction	Pure contract / capability across unrelated classes

◆ Memory Trick

Abstract class = "partial blueprint" (some built, some to-do, has a constructor).

Interface = "pure contract" (only promises, no constructor, enables multiple inheritance).

19.4 Chapter 19 — Quick Revision Box

- Abstract class: `abstract`, cannot instantiate, has abstract + concrete methods, fields, **constructor**; subclass must implement all abstract methods.
- Interface: contract; variables **public static final**; methods public abstract by default (default/static allowed Java 8+); no constructor.
- Class **extends** one class, **implements** many interfaces → Java's multiple inheritance.
- Neither can be instantiated directly.

CHAPTER 20 — CONSTRUCTORS & INITIALIZATION

Constructor behaviour — especially **execution order** — is a favourite source of output-based questions.

20.1 What is a Constructor?

Simple Meaning

A **constructor** is a special method that runs **automatically when an object is created**, used to **initialize** the object's data.

Key Rules (Java)

- The constructor has the **same name as the class**.
- It has **no return type** (not even `void`).
- It is called automatically with `new`.

```
class Student {  
    int roll;  
    Student(int r) {        // constructor  
        roll = r;  
    }  
}  
Student s = new Student(10); // constructor runs, roll = 10
```

★ IBPS Exam Point

A constructor has the **same name as the class** and **no return type**. It is invoked automatically when an object is created with `new`.

⚠ Examiner's Trap

If a constructor is given a return type (e.g., `void Student()`), it becomes an **ordinary method**, not a constructor — a subtle trick.

20.2 Types of Constructors

Type	Description
Default (no-arg)	Takes no parameters. If you write no constructor, Java provides an implicit default one.
Parameterized	Takes arguments to initialize with specific values.
Copy	Creates an object by copying another (Java has no built-in copy constructor like C++; you define one manually).

⚠ Examiner's Trap

The compiler adds a default constructor **only if you define no constructor at all**. If you write a **parameterized** constructor and then try `new Student()` with no args, it **fails to compile** unless you also add a no-arg constructor.

20.3 Constructor Overloading

Like methods, constructors can be **overloaded** (multiple constructors with different parameter lists).

```
class Box {  
    Box()          { System.out.println("No-arg"); }  
    Box(int x)     { System.out.println("One-arg"); }  
    Box(int x, int y){ System.out.println("Two-arg"); }  
}
```

20.4 Constructor Chaining: `this()` and `super()`

- `this()` calls **another constructor in the same class**.
- `super()` calls the **parent class constructor**.
- Whichever is used **must be the first statement** in the constructor. If you write neither, Java **implicitly inserts** `super()` as the first line.

20.5 Execution Order in Inheritance (classic output question)

Rule: the **parent (super) constructor always runs before** the child constructor, because `super()` is called first.

Worked Example 1 — constructors only

```
class A {  
    A() { System.out.println("A constructor"); }  
}  
class B extends A {  
    B() { System.out.println("B constructor"); }  
}  
new B();
```

Output:

```
A constructor  
B constructor
```

Parent first, then child.

Worked Example 2 — full order (static block, instance block, constructor)

This is the complete initialization order and a very common trick question.

```

class A {
    static { System.out.println("A static block"); }
    { System.out.println("A instance block"); }
    A() { System.out.println("A constructor"); }
}
class B extends A {
    static { System.out.println("B static block"); }
    { System.out.println("B instance block"); }
    B() { System.out.println("B constructor"); }
}
new B();

```

Output:

A static block	<- static blocks first (parent, then child), ONCE at class load
B static block	
A instance block	<- then per object: parent instance block
A constructor	<- parent constructor
B instance block	<- child instance block
B constructor	<- child constructor

The order to memorise: 1. **Static** blocks — parent then child (run **once**, when the class is loaded). 2. **Instance** block of parent → **parent constructor**. 3. **Instance** block of child → **child constructor**.

★ IBPS Exam Point

- **Parent constructor executes before child constructor** (`super()` runs first).
- Full order: **static (parent→child, once) → instance block + constructor of parent → instance block + constructor of child**.

⚠ Examiner's Trap

Static blocks run only once (at class loading), **before** any object is created. Instance blocks and constructors run **every time** an object is created. Creating two objects runs static blocks **once** but instance blocks/constructors **twice**.

◆ Memory Trick

"Static once, Super first, Instance-then-Constructor per object."

20.6 Chapter 20 — Quick Revision Box

- Constructor: same name as class, no return type, auto-called by `new`.
- Types: default (auto only if none written), parameterized, copy (manual in Java).
- `this()` = same-class constructor; `super()` = parent constructor; must be first line.
- Parent constructor runs before child.
- Full order: static blocks (parent→child, once) → parent instance block + ctor → child instance block + ctor.

CHAPTER 21 — KEYWORDS & MODIFIERS

21.1 The `static` Keyword

`static` means a member belongs to the **class itself**, not to any individual object — there is **one shared copy** for all objects.

- **Static variable:** shared by all objects (e.g., a counter of how many objects exist).
- **Static method:** called on the class (`ClassName.method()`); **cannot access instance (non-static) members directly** and **cannot use `this`**.
- **Static block:** runs once when the class is loaded.

```
class Counter {
    static int count = 0;    // shared across all objects
    Counter() { count++; }
}
```

★ IBPS Exam Point

A **static** member belongs to the **class**, shared by all objects. A static method **cannot access non-static members directly** and cannot use `this`. `main` is static so it runs without creating an object.

⚠ Examiner's Trap

Calling a non-static (instance) variable/method **directly** from a static method is a **compile error** — you'd need an object reference.

21.2 The `final` Keyword

`final` means "cannot be changed", but its meaning depends on where it's used:

Used on	Meaning
final variable	A constant — value cannot be reassigned
final method	Cannot be overridden by a subclass
final class	Cannot be extended (no subclass) — e.g., <code>String</code> is final

★ IBPS Exam Point

- `final` variable → constant; `final` method → not overridable; `final` class → not inheritable.
- Java's `String` class is **final**.

⚠ Examiner's Trap

A `final` method **cannot be overridden**, but it **can be overloaded**. A `final` class cannot be a superclass. Don't confuse `final` (Java constant / no-override) with `finally` (exception block) or `finalize()` (garbage-collection method) — three different things.

21.3 The `this` Keyword

this refers to the **current object**. Used to distinguish instance variables from parameters with the same name, and to call another constructor via `this()`.

```
class Point {  
    int x;  
    Point(int x) { this.x = x; }    // this.x = instance field; x = parameter  
}
```

21.4 Access Modifiers (full detail)

Modifier	Same class	Same package	Subclass (other package)	Everywhere
<code>private</code>	✓	✗	✗	✗
<code>default</code> (no keyword)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

Visibility (least → most): **private < default < protected < public**.

⚠ Examiner's Trap

`protected` members are accessible in a **subclass even in a different package**, but `default` members are **not** visible outside their package at all. This private/default/protected boundary is frequently tested.

21.5 Method Hiding vs Overriding

- **Overriding** applies to **instance** methods → resolved by the **object's type** at run time (dynamic).
- **Method hiding** happens when a **subclass defines a static method** with the same signature as a parent's static method. Static methods are **not** overridden — the one that runs depends on the **reference type**, decided at **compile time**.

```
class Parent { static void show() { System.out.println("Parent"); } }
class Child extends Parent { static void show() { System.out.println("Child"); } }

Parent p = new Child();
p.show(); // OUTPUT: "Parent" (static → reference type, this is HIDING not overriding)
```

★ IBPS Exam Point

Static methods are **hidden, not overridden**. A hidden static method is chosen by the **reference type** (compile time), unlike overridden instance methods (object type, run time).

21.6 Chapter 21 — Quick Revision Box

- `static` = belongs to class, one shared copy; static method can't use `this` or access instance members directly.
- `final`: variable=constant, method=no override, class=no inherit; `String` is final.
- `this` = current object; distinguishes fields from parameters.
- Access: private < default < protected < public; `protected` reaches subclass across packages, `default` does not.
- Static methods → method **hiding** (reference type, compile time), not overriding.

CHAPTER 22 — RELATIONSHIPS: ASSOCIATION, AGGREGATION, COMPOSITION

22.1 IS-A vs HAS-A

- **IS-A** relationship = **inheritance** (a Dog **is a** Animal) → `extends / implements`.
- **HAS-A** relationship = **association** (a Car **has a** Engine) → one class holds a reference to another.

★ IBPS Exam Point

Inheritance models **IS-A**; association (having another object as a member) models **HAS-A**.

22.2 Association, Aggregation, Composition

These describe **HAS-A** relationships of increasing strength.

Relationship	Meaning	Lifecycle dependency	Analogy
Association	A general "uses-a" link between two independent objects	Independent	A teacher and a student (both exist independently)
Aggregation	A weak "has-a"; the part can exist without the whole	Independent	A department has professors; professors survive if the department closes
Composition	A strong "has-a"; the part cannot exist without the whole	Dependent	A house has rooms; destroy the house and the rooms cease to exist

Key Distinction (asked often)

- **Aggregation:** "has-a", but the contained object has an **independent lifecycle** (weak ownership).
- **Composition:** "part-of", and the contained object's lifecycle is **tied** to the container (strong ownership). If the whole is destroyed, so are the parts.

```
// Composition: Engine is part-of Car and created/owned by it
class Car {
    private Engine engine = new Engine(); // Engine lives & dies with Car
}
```

★ IBPS Exam Point

- **Composition = strong ownership**, part cannot exist without the whole (dependent lifecycle).
- **Aggregation = weak ownership**, part can exist independently.

⚠ Examiner's Trap

Both aggregation and composition are **HAS-A**; the difference is **lifecycle dependency**. Composition is the **stronger** form. Do not equate HAS-A with inheritance (that's IS-A).

◆ Memory Trick

Composition = Cannot exist alone (both start with "C"). Aggregation = "Apart is okay" — can exist apart.

22.3 Chapter 22 — Quick Revision Box

- IS-A = inheritance (extends/implements); HAS-A = association.
- Association (independent) < Aggregation (weak has-a, independent lifecycle) < Composition (strong has-a, dependent lifecycle).
- Composition: part dies with whole. Aggregation: part survives.

CHAPTER 23 — JAVA vs PYTHON OOP DIFFERENCES

IBPS sometimes tests language-specific behaviour. Know where Java and Python differ.

23.1 Key Differences

Feature	Java	Python
Typing	Statically typed (declare types)	Dynamically typed
Access modifiers	Real keywords: <code>private</code> , <code>protected</code> , <code>public</code>	Convention only: <code>_name</code> (protected), <code>__name</code> (private via name-mangling)
Multiple inheritance	Not via classes (only interfaces)	Supported (uses MRO — Method Resolution Order)
Method overloading	Supported (different signatures)	Not truly supported — the last definition wins; use default args
Constructor	Method named after the class	The <code>__init__</code> method
Current object keyword	<code>this</code> (implicit)	<code>self</code> (explicit first parameter)
Everything an object?	Primitives are not objects	Everything is an object
Compilation	Compiled to bytecode, then JVM	Interpreted

★ IBPS Exam Point

- **Python constructor = `__init__`; Java constructor = class-name method.**
- `self` (Python) $\approx$ `this` (Java), but Python passes `self` explicitly as the first parameter.
- **Python supports multiple inheritance** (via MRO); **Java does not** (with classes).
- **Java supports method overloading; Python does not** (last definition overrides earlier ones).

⚠ Examiner's Trap

Python has **no true private** members — `__name` only triggers **name-mangling**, not real access control. Java's `private` is enforced by the compiler. Do not claim Python has strict private/protected keywords.

23.2 Chapter 23 — Quick Revision Box

- Java: static typing, real access modifiers, no class multiple inheritance, overloading yes, `this`, constructor = class name.
- Python: dynamic typing, convention-based access (`_`, `__`), multiple inheritance yes (MRO), overloading no, `self`, constructor = `__init__`.
- Python: everything is an object; no true private (name-mangling only).

(End of Part B — Object-Oriented Programming)

PART C — LAST-MINUTE REVISION MASTER SECTION

Use this section in the final days before the exam. Every point is exam-ready.

C1. DSA LAST-MINUTE REVISION NOTES

- Data = raw facts; Information = processed data. Flow: Data → Data Structure → Algorithm → Output.
 - Algorithm must be **finite, definite, effective**, with input & output; it **must terminate**.
 - ADT = what (logical); Data structure = how (physical).
 - Linear: Array, Stack, Queue, Linked List. Non-linear: Tree, Graph.
 - Array = static + homogeneous + contiguous; access $O(1)$, insert/delete middle $O(n)$.
 - Row-major (C/C++/Java/Python): multiply row diff by **columns**. Column-major (FORTRAN): multiply col diff by **rows**.
 - Linked list: node = data + next; head = entry; last → NULL; insert/delete at head $O(1)$; random access $O(n)$.
 - Stack = LIFO (push/pop/peek $O(1)$); Queue = FIFO (enqueue rear, dequeue front).
 - Recursion & function calls use the **call stack**; DFS uses stack, BFS uses queue.
 - Postfix/prefix need no parentheses; evaluate postfix with a stack.
 - Tree: root at level 0; height in edges; max nodes at level $l = 2^l$; max nodes height $h = 2^{(h+1)} - 1$.
 - BST: left < node < right; **inorder = sorted**; balanced $O(\log n)$, skewed $O(n)$.
 - AVL: balance factor $\in \{-1, 0, +1\}$; LL → right, RR → left, LR → left-then-right, RL → right-then-left.
 - Heap = complete binary tree in array; children at $2i+1, 2i+2$; max-heap root = max; build $O(n)$.
 - Graph: adjacency matrix $O(V^2)$, list $O(V+E)$; tree = connected acyclic, $n-1$ edges.
 - Dijkstra (no negatives), Bellman-Ford (negatives), Prim/Kruskal (MST = $V-1$ edges).
 - Hashing: $O(1)$ average, $O(n)$ worst; separate chaining (linked lists) vs open addressing.
 - Binary search needs sorted data → $O(\log n)$; linear search $O(n)$.
 - Comparison sorting lower bound = $O(n \log n)$. Merge & Heap = guaranteed $O(n \log n)$.
-

C2. OOPs LAST-MINUTE REVISION NOTES

- Class = blueprint (no data memory alone); Object = instance (has memory), created with `new`.
- Four pillars = **A-PIE**: Abstraction, Polymorphism, Inheritance, Encapsulation.

- Encapsulation = bundle + hide data (`private` + getters/setters) → data security.
 - Abstraction = hide implementation (abstract classes/interfaces) → show essentials.
 - Inheritance = `extends` , IS-A, reuse. Java: no multiple inheritance via classes (diamond problem); only via interfaces.
 - Polymorphism: overloading (compile-time, same class, different params) + overriding (run-time, subclass, same signature).
 - Overriding decided by **object type** (run time); fields & static methods decided by **reference type**.
 - Cannot overload by return type alone.
 - Abstract class: not instantiable, has constructors + abstract & concrete methods.
 - Interface: variables public static final; methods public abstract by default (default/static since Java 8); implement many → multiple inheritance.
 - Constructor: same name as class, no return type, auto-called; parent constructor runs before child.
 - Init order: static blocks (parent→child, once) → parent instance block + ctor → child instance block + ctor.
 - `static` = class-level, shared; static method can't use `this`. `final`: constant / no-override / no-inherit.
 - Access: `private` < `default` < `protected` < `public`. `protected` reaches subclass across packages; `default` does not.
 - Static methods → method **hiding** (reference type), not overriding.
 - HAS-A: association < aggregation (weak, independent lifecycle) < composition (strong, dependent lifecycle).
 - Python: `__init__`, `self`, dynamic typing, multiple inheritance (MRO), no true overloading, no true private.
-

C3. MASTER DATA STRUCTURE COMPLEXITY TABLE (average case)

Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Doubly Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Hash Table	—	$O(1)$	$O(1)$	$O(1)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
BST (skewed, worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)$ peek	$O(n)$	$O(\log n)$	$O(\log n)$

* at a known position such as head; $O(n)$ if the position must be searched first.

C4. MASTER SORTING COMPLEXITY TABLE

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

Searching: Linear $O(n)$ (unsorted OK); Binary $O(\log n)$ (needs sorted).

C5. MASTER TREE FORMULA SHEET

(Convention: root at level 0, height = number of edges.)

- Maximum nodes at level $I = 2^I$.

- Maximum nodes in a binary tree of height $h = 2^{(h+1)} - 1$.
- Minimum height with n nodes $\approx \lfloor \log_2 n \rfloor$.
- A binary tree with n nodes has $n - 1$ edges.
- Full binary tree: **leaves = internal nodes + 1**.
- Perfect binary tree of height h : total nodes = $2^{(h+1)} - 1$, leaves = 2^h .
- Complete binary tree with n nodes: height = $\lfloor \log_2 n \rfloor$.
- Heap array indices (0-based): left = $2i+1$, right = $2i+2$, parent = $\lfloor (i-1)/2 \rfloor$.
- MST of a graph with V vertices has $V - 1$ edges.
- Number of binary trees with n nodes = **Catalan number** $C_n = (2n)! / ((n+1)! n!)$.

C6. MASTER OOP CONCEPT COMPARISON TABLE

Concept A	Concept B	Key difference
Class	Object	Blueprint vs instance (memory)
Encapsulation	Abstraction	Hide data vs hide implementation
Overloading	Overriding	Compile-time, same class vs run-time, subclass
Static binding	Dynamic binding	Compile-time vs run-time resolution
Abstract class	Interface	Partial (has ctor, concrete methods) vs pure contract (multiple inheritance)
extends	implements	Inherit class vs implement interface
Method overriding	Method hiding	Instance (object type) vs static (reference type)
Aggregation	Composition	Weak has-a (independent) vs strong has-a (dependent lifecycle)
IS-A	HAS-A	Inheritance vs association
this	super	Current object vs parent object
final	finally / finalize()	Constant/no-override vs exception block / GC method

C7. TOP 100 DSA MUST-REMEMBER FACTS

1. A stack is LIFO; a queue is FIFO.
2. Array access by index is $O(1)$.
3. Array insertion/deletion in the middle is $O(n)$ due to shifting.
4. Arrays are static (fixed size) and homogeneous.

5. Linked list insertion at head is $O(1)$.
6. Linked list random access is $O(n)$.
7. A node in a singly linked list = data + next pointer.
8. The last node of a singly linked list points to NULL.
9. Circular linked list: last node points to the first (no NULL).
10. Doubly linked list has prev and next pointers.
11. Stack operations push/pop/peek are all $O(1)$.
12. Stack overflow = push on a full stack; underflow = pop on empty.
13. Recursion uses the call stack internally.
14. Queue: enqueue at rear, dequeue at front.
15. Circular queue solves linear queue's memory wastage.
16. A deque allows insertion/deletion at both ends.
17. Priority queue serves by priority, not FIFO; usually heap-based.
18. BFS uses a queue; DFS uses a stack (or recursion).
19. Postfix and prefix expressions need no parentheses.
20. Infix $A+B \rightarrow$ postfix $AB+ \rightarrow$ prefix $+AB$.
21. Operator precedence: $^ > * / > + -$.
22. Postfix evaluation uses a stack.
23. In postfix evaluation, the first popped operand is the right operand.
24. Big O = upper bound (worst case).
25. Omega (Ω) = lower bound (best case).
26. Theta (Θ) = tight bound.
27. Growth order: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
28. A loop multiplying/dividing the counter by a constant is $O(\log n)$.
29. Nested loops multiply complexities.
30. Consecutive loops add (keep the largest term).
31. Naive recursive Fibonacci is $O(2^n)$.
32. Factorial recursion is $O(n)$ time, $O(n)$ stack space.
33. Linear search is $O(n)$; best case $O(1)$.
34. Binary search is $O(\log n)$ and requires sorted data.
35. Bubble sort worst/average is $O(n^2)$; best $O(n)$ if optimised.
36. Selection sort is $O(n^2)$ in all cases.
37. Insertion sort is $O(n^2)$ average, $O(n)$ best (nearly sorted).
38. Merge sort is $O(n \log n)$ in all cases.
39. Merge sort needs $O(n)$ extra space and is stable.
40. Quick sort average $O(n \log n)$, worst $O(n^2)$.
41. Quick sort is in-place; merge sort is not.
42. Heap sort is $O(n \log n)$ and in-place but not stable.
43. Stable sorts: bubble, insertion, merge.
44. Comparison-based sorting cannot beat $O(n \log n)$.
45. After k passes of bubble sort, k largest elements are placed at the end.
46. After k passes of selection sort, k smallest elements are placed at the front.
47. Selection sort makes the fewest swaps ($\leq n-1$).
48. In a tree, root has no parent; leaf has no children.
49. Height of a tree = longest path (edges) from root to a leaf.
50. Level of the root = 0 (standard convention).
51. A binary tree node has at most 2 children.

52. Full binary tree: every node has 0 or 2 children.
53. Complete binary tree: filled level by level, last level left-aligned.
54. Perfect binary tree: full and all leaves at the same level.
55. Max nodes at level $l = 2^l$.
56. Max nodes in a tree of height $h = 2^{(h+1)} - 1$.
57. A tree with n nodes has $n - 1$ edges.
58. Full binary tree: leaves = internal nodes + 1.
59. Preorder = Root, Left, Right.
60. Inorder = Left, Root, Right.
61. Postorder = Left, Right, Root.
62. Level-order traversal uses a queue (BFS).
63. Inorder traversal of a BST yields sorted ascending order.
64. BST rule: left subtree < node < right subtree.
65. A skewed BST degrades to $O(n)$ operations.
66. Inserting sorted data into a plain BST creates a skewed tree.
67. AVL tree balance factor $\in \{-1, 0, +1\}$.
68. Balance factor = height(left) – height(right).
69. LL imbalance is fixed by a single right rotation.
70. RR imbalance is fixed by a single left rotation.
71. LR imbalance = left rotation then right rotation.
72. RL imbalance = right rotation then left rotation.
73. AVL guarantees $O(\log n)$ operations.
74. B-tree and B+ tree are used for database indexing.
75. In a B+ tree, all data is stored in linked leaf nodes.
76. A heap is a complete binary tree.
77. Max-heap root is the maximum; min-heap root is the minimum.
78. Heap children of index i are at $2i+1$ and $2i+2$.
79. Heap insert/delete is $O(\log n)$; peek is $O(1)$.
80. Building a heap from an array is $O(n)$.
81. A heap is not a sorted structure and not a BST.
82. Graph $G = (V, E)$: vertices and edges.
83. A tree is a connected acyclic graph.
84. DAG = Directed Acyclic Graph.
85. Adjacency matrix uses $O(V^2)$ space; $O(1)$ edge check.
86. Adjacency list uses $O(V + E)$ space; good for sparse graphs.
87. BFS and DFS are both $O(V + E)$ with adjacency lists.
88. BFS finds the shortest path in an unweighted graph.
89. Dijkstra fails with negative edge weights.
90. Bellman-Ford handles negative edge weights.
91. Prim's and Kruskal's build a Minimum Spanning Tree.
92. An MST has $V - 1$ edges.
93. Hashing gives $O(1)$ average search/insert/delete.
94. Common hash function: key mod table_size.
95. Load factor = number of items / table size.
96. Separate chaining stores collisions in linked lists (open hashing).
97. Open addressing stores collisions in the table itself (closed hashing).
98. Linear probing suffers primary clustering; double hashing spreads best.

99. Hashing worst case is $O(n)$ (all keys collide).
 100. Merge sort is preferred for external sorting of very large files.
-

C8. TOP 100 OOPS MUST-REMEMBER FACTS

1. OOP models real-world entities as objects.
2. A class is a blueprint; an object is an instance.
3. A class alone does not store data values; objects do.
4. Objects are created with the `new` keyword in Java.
5. A method is a function inside a class.
6. Attributes/fields hold an object's data.
7. The four pillars are Abstraction, Polymorphism, Inheritance, Encapsulation (A-PIE).
8. Class and object are core concepts, not pillars.
9. Encapsulation bundles data and methods and hides data.
10. Encapsulation is achieved via `private` fields + getters/setters.
11. Encapsulation provides data security.
12. Abstraction hides implementation and shows essentials.
13. Abstraction is achieved via abstract classes and interfaces.
14. Encapsulation hides data; abstraction hides complexity.
15. Inheritance lets a subclass acquire superclass members.
16. Inheritance uses the `extends` keyword in Java.
17. Inheritance models an IS-A relationship.
18. Inheritance promotes code reusability.
19. Single, multilevel, and hierarchical inheritance are supported in Java.
20. Java does not support multiple inheritance using classes.
21. The diamond problem is why Java bans multiple class inheritance.
22. Java achieves multiple inheritance via interfaces.
23. C++ supports multiple inheritance of classes.
24. `super` refers to the immediate parent class.
25. `super()` must be the first statement in a child constructor.
26. Polymorphism means "many forms".
27. Compile-time polymorphism = method overloading.
28. Run-time polymorphism = method overriding.
29. Overloading: same name, different parameters, same class.
30. Overloading is resolved at compile time (static binding).
31. You cannot overload by changing only the return type.
32. Overriding: same signature, subclass redefines superclass method.
33. Overriding is resolved at run time (dynamic dispatch).
34. Overridden method chosen by the object's actual type.
35. Fields (variables) are not polymorphic — reference type decides.
36. Static methods are not overridden — reference type decides.
37. `static`, `private`, and `final` methods use static binding.
38. An abstract class cannot be instantiated.
39. An abstract class can have abstract and concrete methods.

40. An abstract class can have constructors and fields.
41. A subclass must implement all abstract methods or be abstract.
42. An interface cannot be instantiated.
43. Interface variables are implicitly public static final.
44. Interface methods are public abstract by default (pre-Java 8).
45. Java 8 added default and static methods to interfaces.
46. Java 9 added private methods to interfaces.
47. A class uses `implements` for interfaces.
48. A class can implement multiple interfaces.
49. A class extends one class but implements many interfaces.
50. Abstract class has a constructor; interface does not.
51. A constructor has the same name as the class.
52. A constructor has no return type.
53. A constructor is called automatically on object creation.
54. If no constructor is written, Java adds a default one.
55. Writing a parameterized constructor removes the implicit default.
56. Constructors can be overloaded.
57. `this()` calls another constructor in the same class.
58. `super()` calls the parent constructor.
59. The parent constructor runs before the child constructor.
60. Static blocks run once, at class loading.
61. Init order: static (parent→child) → instance block + ctor (parent) → instance block + ctor (child).
62. Static blocks run before any object is created.
63. `static` members belong to the class, shared by all objects.
64. A static method cannot use `this`.
65. A static method cannot directly access instance members.
66. `main` is static so it runs without an object.
67. A `final` variable is a constant.
68. A `final` method cannot be overridden.
69. A `final` class cannot be extended.
70. Java's String class is final.
71. `final`, `finally`, and `finalize()` are three different things.
72. A final method can still be overloaded.
73. `this` refers to the current object.
74. Access modifiers: private < default < protected < public.
75. `private` is accessible only in the same class.
76. `default` (no keyword) is accessible only in the same package.
77. `protected` is accessible in subclasses across packages.
78. `public` is accessible everywhere.
79. Static method redefinition in a subclass is method hiding.
80. Hidden static methods are chosen by reference type at compile time.
81. IS-A is inheritance; HAS-A is association.
82. Association is a general link between independent objects.
83. Aggregation is a weak has-a with independent lifecycle.
84. Composition is a strong has-a with dependent lifecycle.
85. In composition, the part cannot exist without the whole.

86. Composition is stronger than aggregation.
 87. Message passing is calling methods between objects.
 88. Java is statically typed.
 89. Python is dynamically typed.
 90. Python's constructor is `__init__`.
 91. Python uses `self`; Java uses `this`.
 92. Python passes `self` explicitly as the first parameter.
 93. Python supports multiple inheritance via MRO.
 94. Python does not truly support method overloading.
 95. Python has no true private members (name-mangling only).
 96. In Python, everything is an object.
 97. Java compiles to bytecode run by the JVM.
 98. Overriding requires an inheritance relationship; overloading does not.
 99. An interface provides 100% abstraction (pre-Java 8).
 100. Encapsulation + abstraction together improve security and maintainability.
-

C9. TOP 50 DSA EXAMINER TRAPS

1. Array insertion is $O(1)$ only at the end; middle/front is $O(n)$.
2. A stack and a queue are linear, not non-linear.
3. Linked list access is $O(n)$, not $O(1)$ — no random access.
4. Losing the head pointer loses the whole linked list.
5. Big O is an upper bound, not the exact running time.
6. Do not swap Big O (upper) and Omega (lower).
7. $O(2^n)$ is worse than $O(n^3)$ for large n .
8. $O(\log n)$ is faster than $O(n)$.
9. Naive Fibonacci is $O(2^n)$, not $O(n)$.
10. A loop with a constant bound (e.g., 100 times) is $O(1)$.
11. Row-major multiplies by columns, not rows.
12. Check whether the array lower bound is 0 or 1 in address problems.
13. Queue follows FIFO, not LIFO.
14. In postfix evaluation, first popped = right operand (matters for $-$ and $/$).
15. Prefix and postfix require no parentheses.
16. A priority queue is not strictly FIFO.
17. A deque can act as both stack and queue.
18. Height is measured in edges; a single node has height 0.
19. Level of the root is 0 (unless the question says otherwise).
20. Full binary tree = 0 or 2 children, not "completely filled".
21. Complete $\neq$ perfect $\neq$ full — know each definition.
22. Inorder of a BST is sorted, but not for a general binary tree.
23. Inserting sorted data into a BST makes it skewed ($O(n)$).
24. BST is $O(\log n)$ only when balanced.
25. AVL LL is fixed by a RIGHT rotation (opposite of the name).
26. AVL RR is fixed by a LEFT rotation.

27. Balance factor is left height – right height.
 28. A heap is not sorted and not a BST.
 29. Build-heap is $O(n)$, not $O(n \log n)$.
 30. Max-heap root is the largest, not the smallest.
 31. Heap is a complete binary tree, stored in an array.
 32. Adjacency matrix wastes space for sparse graphs ($O(V^2)$).
 33. Dijkstra fails with negative edge weights.
 34. BFS (not DFS) gives shortest path in an unweighted graph.
 35. A tree is a special graph (connected, acyclic, $n-1$ edges).
 36. An MST has exactly $V-1$ edges.
 37. "Open hashing" = separate chaining (confusingly named).
 38. "Closed hashing" = open addressing.
 39. Hashing worst case is $O(n)$, not $O(1)$.
 40. Linear probing causes primary clustering.
 41. Binary search needs sorted data; else results are wrong.
 42. Selection sort is always $O(n^2)$, even if the array is sorted.
 43. Quick sort worst case is $O(n^2)$, not $O(n \log n)$.
 44. Merge sort needs $O(n)$ extra space (not in-place).
 45. Heap sort is in-place but not stable.
 46. Bubble/insertion best case is $O(n)$ (nearly sorted); selection is not adaptive.
 47. Comparison sorting cannot be faster than $O(n \log n)$.
 48. The number of swaps $\neq$ the number of comparisons (selection has few swaps).
 49. Traversal visits every node once; searching may stop early.
 50. A doubly linked list uses extra memory for the prev pointer.
-

C10. TOP 50 OOPs EXAMINER TRAPS

1. A class does not store data; only objects do.
2. Class and object are not counted among the four pillars.
3. Encapsulation hides data; abstraction hides implementation (don't swap).
4. Encapsulation uses access modifiers; abstraction uses abstract classes/interfaces.
5. Java does not support multiple inheritance via classes.
6. "Java supports multiple inheritance" is true only via interfaces.
7. C++ allows multiple class inheritance; Java does not.
8. You cannot overload by return type alone.
9. Overriding needs the same signature; overloading needs different parameters.
10. Overriding is run-time; overloading is compile-time.
11. Overridden methods use the object type, not the reference type.
12. Fields (variables) are chosen by reference type, not object type.
13. Static methods are hidden, not overridden.
14. Hidden static methods follow the reference type.
15. An abstract class can have concrete methods and a constructor.
16. "An abstract class has only abstract methods" is false.
17. An abstract class cannot be instantiated.

18. An interface cannot be instantiated.
 19. Interface variables are always public static final (constants).
 20. Interface fields cannot be reassigned.
 21. A class extends one class but implements many interfaces.
 22. Do not mix `extends` and `implements` keywords.
 23. A constructor with a return type becomes an ordinary method.
 24. The default constructor is added only if no constructor is written.
 25. A parameterized constructor removes the implicit default.
 26. The parent constructor runs before the child constructor.
 27. `super()` / `this()` must be the first statement in a constructor.
 28. Static blocks run once, before any object creation.
 29. Instance blocks run every time an object is created.
 30. Creating two objects runs static blocks once but constructors twice.
 31. A static method cannot use `this`.
 32. A static method cannot access instance members directly.
 33. `final` variable = constant; `final` method = no override; `final` class = no inherit.
 34. `final`, `finally`, and `finalize()` are different concepts.
 35. A final method can still be overloaded.
 36. Java's String class is final and immutable.
 37. `protected` reaches subclasses across packages.
 38. `default` members are not visible outside the package.
 39. Visibility order: private < default < protected < public.
 40. IS-A is inheritance; HAS-A is association — don't equate them.
 41. Aggregation and composition are both HAS-A; lifecycle differs.
 42. Composition is the stronger (dependent-lifecycle) relationship.
 43. Python has no true private members (name-mangling only).
 44. Python's constructor is `__init__`, not the class name.
 45. Python uses `self`, passed explicitly; Java's `this` is implicit.
 46. Python supports multiple inheritance; Java (classes) does not.
 47. Python does not truly support method overloading (last def wins).
 48. In Python, everything is an object.
 49. An interface gives 100% abstraction only before Java 8 (defaults changed this).
 50. Message passing is a concept, not one of the four pillars.
-

C11. 30 MOST CONFUSING DSA CONCEPTS

1. **Array vs Linked List** — fast access vs fast insertion at ends.
2. **Stack vs Queue** — LIFO vs FIFO.
3. **Big O vs Omega vs Theta** — upper vs lower vs tight bound.
4. **$O(\log n)$ vs $O(n)$** — logarithmic is much faster.
5. **$O(n^2)$ vs $O(2^n)$** — quadratic vs exponential (exponential far worse).
6. **Best vs Worst case** — least vs most operations.
7. **Time vs Space complexity** — speed vs memory.
8. **Infix vs Prefix vs Postfix** — operator position.

9. **Linear vs Circular queue** — wasted slots vs reused slots.
 10. **Singly vs Doubly vs Circular linked list** — pointer arrangement.
 11. **Full vs Complete vs Perfect binary tree** — three distinct definitions.
 12. **Height vs Depth vs Level** — measured from leaf/root differently.
 13. **Preorder vs Inorder vs Postorder** — position of root.
 14. **BST vs Heap** — sorted-ish ordering vs parent-child order only.
 15. **Balanced vs Skewed BST** — $O(\log n)$ vs $O(n)$.
 16. **AVL LL/RR/LR/RL** — which rotation fixes which case.
 17. **Min-heap vs Max-heap** — smallest vs largest at root.
 18. **B-tree vs B+ tree** — data in all nodes vs only leaves.
 19. **DFS vs BFS** — stack (deep) vs queue (broad).
 20. **Adjacency matrix vs list** — dense vs sparse graphs.
 21. **Directed vs Undirected graph** — edge direction.
 22. **Cyclic vs Acyclic graph** — presence of cycles.
 23. **Dijkstra vs Bellman-Ford** — negatives not allowed vs allowed.
 24. **Prim's vs Kruskal's** — grow tree vs pick smallest edges.
 25. **Separate chaining vs Open addressing** — linked lists vs in-table probing.
 26. **Linear vs Quadratic vs Double hashing** — probing strategies.
 27. **Linear vs Binary search** — unsorted OK vs sorted required.
 28. **Stable vs Unstable sort** — equal elements keep order or not.
 29. **In-place vs Not in-place** — $O(1)$ extra vs extra array.
 30. **Quick vs Merge sort** — in-place/worst $O(n^2)$ vs stable/ $O(n)$ space.
-

C12. 30 MOST CONFUSING OOP CONCEPTS

1. **Class vs Object** — blueprint vs instance.
2. **Encapsulation vs Abstraction** — hide data vs hide implementation.
3. **Overloading vs Overriding** — compile-time/same-class vs run-time/subclass.
4. **Static vs Dynamic binding** — compile-time vs run-time resolution.
5. **Overriding vs Hiding** — instance (object type) vs static (reference type).
6. **Abstract class vs Interface** — partial vs pure contract.
7. **extends vs implements** — class vs interface.
8. **IS-A vs HAS-A** — inheritance vs association.
9. **Association vs Aggregation vs Composition** — link strength / lifecycle.
10. **Aggregation vs Composition** — independent vs dependent lifecycle.
11. **this vs super** — current object vs parent.
12. **this() vs super()** — same-class ctor vs parent ctor.
13. **Instance vs Static members** — per-object vs shared.
14. **Instance block vs Static block** — per-object vs once at load.
15. **final vs finally vs finalize()** — constant vs exception block vs GC.
16. **final method vs final class** — no override vs no inherit.
17. **Default vs Parameterized constructor** — no-arg vs with args.
18. **Constructor vs Method** — no return type & class name vs normal.
19. **private vs default vs protected vs public** — visibility levels.

20. **protected vs default** — subclass-across-package vs same-package only.
 21. **Compile-time vs Run-time polymorphism** — overloading vs overriding.
 22. **Single vs Multiple inheritance** — one vs many parents.
 23. **Multiple inheritance in Java vs C++** — interfaces-only vs classes allowed.
 24. **Field access vs Method call polymorphism** — reference vs object type.
 25. **Abstract method vs Concrete method** — no body vs has body.
 26. **Interface (pre-8) vs (8+)** — pure abstract vs default/static methods.
 27. **Java this vs Python self** — implicit vs explicit.
 28. **Java constructor vs Python init** — class name vs **init**.
 29. **Static typing vs Dynamic typing** — Java vs Python.
 30. **Method hiding vs Method overloading** — static redefinition vs different params.
-

(End of Part C — Last-Minute Revision Master Section)

PART D — 200 IBPS SO IT MCQs

Instructions: Each question has one correct option. The complete Answer Key with detailed explanations appears in Part E. Do not scroll to answers until you have attempted the set.

SECTION 1 — DSA (EASY) | Q1-Q55

- Q1.** Which data structure follows the LIFO principle? A) Queue B) Stack C) Tree D) Graph
- Q2.** Which data structure follows the FIFO principle? A) Stack B) Tree C) Queue D) Heap
- Q3.** What is the time complexity of accessing an array element by its index? A) $O(n)$ B) $O(\log n)$ C) $O(1)$ D) $O(n^2)$
- Q4.** In a singly linked list, the link field of the last node points to: A) The head node B) Itself C) NULL D) The first node
- Q5.** Which of the following is a non-linear data structure? A) Array B) Stack C) Queue D) Tree
- Q6.** The operation of inserting an element into a stack is called: A) Enqueue B) Push C) Append D) Add
- Q7.** Removing an element from a stack is performed by which operation? A) Pop B) Dequeue C) Delete D) Extract
- Q8.** In a queue, insertion (enqueue) takes place at the: A) Front B) Rear C) Middle D) Top
- Q9.** Which data structure is internally used to implement recursion? A) Queue B) Stack C) Graph D) Array
- Q10.** Big O notation primarily represents the: A) Best case only B) Average case only C) Upper bound (worst case) D) Lower bound (best case)
- Q11.** Which data structure is used to implement Breadth-First Search (BFS)? A) Stack B) Queue C) Tree D) Heap
- Q12.** Which data structure is used to implement Depth-First Search (DFS)? A) Queue B) Stack C) Heap D) Priority Queue
- Q13.** The time complexity of binary search is: A) $O(n)$ B) $O(1)$ C) $O(\log n)$ D) $O(n^2)$
- Q14.** Binary search can be correctly applied only when the data is: A) Random B) Sorted C) Reversed D) Unsorted
- Q15.** The worst-case time complexity of linear search is: A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
- Q16.** In a binary tree, the maximum number of children a node can have is: A) 1 B) 2 C) 3 D) Unlimited
- Q17.** A node in a tree that has no children is called a: A) Root B) Leaf C) Parent D) Internal node
- Q18.** The topmost node of a tree is called the: A) Leaf B) Child C) Root D) Sibling
- Q19.** The inorder traversal of a Binary Search Tree produces the elements in: A) Random order B) Reverse order C) Sorted ascending order D) Unsorted order

- Q20.** Which sorting algorithm has $O(n \log n)$ time complexity in all cases (best, average, worst)? A) Bubble Sort B) Selection Sort C) Merge Sort D) Insertion Sort
- Q21.** Stack overflow occurs when we attempt to: A) Pop from an empty stack B) Push onto a full stack C) Peek an empty stack D) Traverse the stack
- Q22.** Stack underflow occurs when we attempt to: A) Push onto a full stack B) Pop from an empty stack C) Peek a full stack D) Insert an element
- Q23.** LIFO stands for: A) Last In First Out B) Least In First Out C) Last In Final Out D) Linear In First Out
- Q24.** FIFO stands for: A) First In First Out B) Final In First Out C) Fast In Fast Out D) First In Final Out
- Q25.** Which of the following is a linear data structure? A) Tree B) Graph C) Array D) Heap
- Q26.** A tree with n nodes has exactly how many edges? A) n B) $n - 1$ C) $n + 1$ D) $2n$
- Q27.** Which data structure stores its elements in contiguous memory locations? A) Linked list B) Array C) Tree D) Graph
- Q28.** In a queue, deletion (dequeue) happens at the: A) Rear B) Front C) Top D) Middle
- Q29.** The peek (or top) operation in a stack: A) Removes the top element B) Adds an element C) Views the top element without removing it D) Clears the stack
- Q30.** Which pointer provides the entry point to access an entire linked list? A) Tail B) Head C) Middle D) Current
- Q31.** A node in a singly linked list contains data and: A) An index B) A pointer/link to the next node C) Only a value D) An array
- Q32.** How many pointers does each node of a doubly linked list contain? A) 1 B) 2 C) 3 D) 0
- Q33.** Which of these is the fastest (best) average-case time complexity? A) $O(n)$ B) $O(\log n)$ C) $O(1)$ D) $O(n \log n)$
- Q34.** Among the following, which represents the smallest growth rate? A) $O(n)$ B) $O(1)$ C) $O(n^2)$ D) $O(\log n)$
- Q35.** Among the following, which represents the largest growth rate? A) $O(n^2)$ B) $O(n!)$ C) $O(2^n)$ D) $O(n \log n)$
- Q36.** The postfix form of the infix expression $A + B$ is: A) $+AB$ B) $AB+$ C) $A+B$ D) $BA+$
- Q37.** The prefix form of the infix expression $A + B$ is: A) $AB+$ B) $+AB$ C) $A+B$ D) $BA+$
- Q38.** Which notation does not require parentheses to preserve the order of operations? A) Infix B) Postfix C) Only infix D) Algebraic
- Q39.** The time complexity of selection sort in all cases is: A) $O(n)$ B) $O(n \log n)$ C) $O(n^2)$ D) $O(\log n)$
- Q40.** Which of the following sorting algorithms is stable? A) Selection Sort B) Quick Sort C) Heap Sort D) Merge Sort
- Q41.** The auxiliary space complexity of merge sort is: A) $O(1)$ B) $O(n)$ C) $O(\log n)$ D) $O(n^2)$
- Q42.** Which sorting algorithm is in-place but has a worst-case time of $O(n^2)$? A) Merge Sort B) Quick Sort C) Heap Sort D) Counting Sort

- Q43.** A complete binary tree is filled: A) Randomly B) Level by level, left to right C) Right to left D) Bottom up
- Q44.** The balance factor of every node in an AVL tree must lie in the set: A) $\{0, 1, 2\}$ B) $\{-1, 0, 1\}$ C) $\{-2, 0, 2\}$ D) $\{1, 2, 3\}$
- Q45.** In a max-heap, the root node always contains the: A) Smallest element B) Largest element C) Median element D) A random element
- Q46.** A heap is which type of tree? A) Binary Search Tree B) Complete binary tree C) Skewed tree D) AVL tree
- Q47.** The space complexity of representing a graph using an adjacency matrix is: A) $O(V)$ B) $O(E)$ C) $O(V^2)$ D) $O(V + E)$
- Q48.** A Minimum Spanning Tree (MST) of a graph with V vertices has how many edges? A) V B) $V - 1$ C) E D) $E - 1$
- Q49.** The average-case time complexity of search in hashing is: A) $O(n)$ B) $O(\log n)$ C) $O(1)$ D) $O(n^2)$
- Q50.** A collision in hashing occurs when: A) Two different keys map to the same index B) A slot is empty C) The table is full D) A key is deleted
- Q51.** Which tree traversal uses a queue? A) Inorder B) Preorder C) Level-order D) Postorder
- Q52.** The order of visiting nodes in preorder traversal is: A) Left, Root, Right B) Root, Left, Right C) Left, Right, Root D) Right, Left, Root
- Q53.** Which of the following is a dynamic data structure? A) Array B) Linked list C) Static array D) Matrix
- Q54.** In a binary tree (root at level 0), the maximum number of nodes at level 3 is: A) 4 B) 8 C) 16 D) 3
- Q55.** Which searching technique works on unsorted data? A) Binary search B) Linear search C) Both require sorted data D) Ternary search
-

SECTION 2 — OOPs (EASY) | Q56-Q100

- Q56.** OOP stands for: A) Object Oriented Programming B) Order Of Programs C) Object Ordered Process D) Operational Object Programming
- Q57.** A blueprint or template used for creating objects is called a: A) Object B) Class C) Method D) Function
- Q58.** An instance created from a class is called a(n): A) Method B) Object C) Variable D) Package
- Q59.** How many fundamental pillars does OOP have? A) 2 B) 3 C) 4 D) 5
- Q60.** Which of the following is NOT a pillar of OOP? A) Encapsulation B) Inheritance C) Compilation D) Polymorphism
- Q61.** Wrapping data and the methods that operate on it into a single unit is called: A) Inheritance B) Encapsulation C) Polymorphism D) Abstraction

- Q62.** Hiding internal implementation details and showing only essential features is called: A) Encapsulation B) Abstraction C) Inheritance D) Overloading
- Q63.** The mechanism by which one class acquires the properties of another class is: A) Polymorphism B) Encapsulation C) Inheritance D) Abstraction
- Q64.** The term "many forms" describes which OOP concept? A) Inheritance B) Polymorphism C) Encapsulation D) Abstraction
- Q65.** Which keyword is used to implement inheritance in Java? A) implement B) inherits C) extends D) derive
- Q66.** In Java, an object is created using which keyword? A) create B) new C) object D) make
- Q67.** A constructor has the same name as the: A) Method B) Class C) Package D) Variable
- Q68.** What is the return type of a constructor? A) void B) int C) No return type D) String
- Q69.** Which keyword refers to the current object? A) super B) this C) self D) current
- Q70.** Which keyword is used to refer to the immediate parent class? A) this B) base C) super D) parent
- Q71.** Defining multiple methods with the same name but different parameter lists is called: A) Overriding B) Overloading C) Hiding D) Inheritance
- Q72.** When a subclass provides its own implementation of a superclass method, it is called: A) Overloading B) Overriding C) Hiding D) Nesting
- Q73.** Method overloading is resolved at: A) Run time B) Compile time C) Link time D) Load time
- Q74.** Method overriding is resolved at: A) Compile time B) Run time C) Load time D) Never
- Q75.** Encapsulation is typically achieved by: A) Making fields public B) Using private fields with public getters and setters C) Using static members D) Using final classes
- Q76.** Which access modifier is the most restrictive? A) public B) protected C) private D) default
- Q77.** Which access modifier allows access from everywhere? A) private B) default C) public D) protected
- Q78.** A class that cannot be instantiated directly is a(n): A) Final class B) Abstract class C) Static class D) Public class
- Q79.** Variables declared inside an interface are by default: A) private B) public static final C) protected D) static only
- Q80.** Java achieves multiple inheritance through: A) Classes B) Interfaces C) Packages D) Methods
- Q81.** A variable declared as final becomes a: A) Constant B) Method C) Loop D) Object
- Q82.** A final method cannot be: A) Called B) Overridden C) Declared D) Used
- Q83.** A final class cannot be: A) Instantiated B) Extended C) Compiled D) Used
- Q84.** A static member belongs to the: A) Object B) Class C) Method D) Package
- Q85.** Which method serves as the entry point of a Java program? A) start() B) main() C) run() D) begin()
- Q86.** The main method in Java is declared as: A) private B) static C) final D) abstract

- Q87.** An IS-A relationship is represented by: A) Association B) Inheritance C) Composition D) Aggregation
- Q88.** A HAS-A relationship is represented by: A) Inheritance B) Association C) Overriding D) Overloading
- Q89.** Which is the strongest form of relationship between objects? A) Association B) Aggregation C) Composition D) Dependency
- Q90.** The constructor method in Python is named: A) init B) **__init__** C) constructor D) new
- Q91.** In Python, the keyword used to refer to the current object is: A) this B) self C) me D) cls
- Q92.** Data hiding is primarily a feature of: A) Inheritance B) Encapsulation C) Polymorphism D) Overloading
- Q93.** Which mnemonic is commonly used to remember the four pillars of OOP? A) A-PIE B) POID C) CADE D) OOPS
- Q94.** A real-world entity that has state (data) and behaviour (methods) is best modelled as a(n): A) Class B) Object C) Method D) Loop
- Q95.** Abstraction in Java is achieved using: A) Loops B) Abstract classes and interfaces C) Arrays D) Static blocks
- Q96.** A constructor is called: A) Manually every time B) Automatically when an object is created C) Never D) When an object is deleted
- Q97.** Which of the following cannot have a constructor? A) Abstract class B) Interface C) Normal class D) Subclass
- Q98.** Two classes having a parent-child relationship demonstrate: A) Association B) Inheritance C) Hashing D) Recursion
- Q99.** Code reusability in OOP is achieved mainly through: A) Encapsulation B) Inheritance C) Abstraction D) Polymorphism
- Q100.** Which of the following is NOT a valid access modifier in Java? A) public B) private C) protected D) friend

SECTION 3 — DSA (MODERATE TO HARD) | Q101-Q155

Q101. What is the time complexity of the following code?

```
for i = 1 to n:
    for j = 1 to n:
        statement
```

A) $O(n)$ B) $O(n^2)$ C) $O(\log n)$ D) $O(n \log n)$

Q102. What is the time complexity of the following loop?

```
i = 1
while i < n:
    i = i * 2
```

A) $O(n)$ B) $O(n^2)$ C) $O(\log n)$ D) $O(1)$

Q103. What is the time complexity of the following code?

```
for i = 1 to n:
    for j = 1 to i:
        statement
```

A) $O(n)$ B) $O(n^2)$ C) $O(\log n)$ D) $O(n \log n)$

Q104. Two separate (consecutive) loops each running n times give a combined complexity of: A) $O(n^2)$ B) $O(n)$ C) Exactly $O(2n)$ and cannot be simplified D) $O(\log n)$

Q105. The time complexity of the naive recursive Fibonacci algorithm is: A) $O(n)$ B) $O(\log n)$ C) $O(2^n)$ D) $O(n^2)$

Q106. The postfix expression for the infix $(A + B) * C$ is: A) $AB+C$ B) $ABC+$ C) $A+BC$ D) $+ABC$

Q107. The postfix expression for the infix $A + B * C$ is: A) $AB+C$ B) $ABC+$ C) $ABC+ D) A+BC$

Q108. The prefix expression for the infix $A * B + C$ is: A) $+ABC$ B) $+ABC$ C) $ABC+$ D) $ABC+$

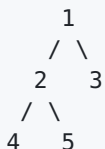
Q109. Evaluate the postfix expression: $5\ 6\ 2\ +\ *$ A) 40 B) 32 C) 16 D) 56

Q110. Evaluate the postfix expression: $8\ 2\ /\ 3\ -$ A) 1 B) 2 C) 5 D) -1

Q111. A stack undergoes: push 1, push 2, push 3, pop, push 4, pop. What element is now on top? A) 1 B) 2 C) 3 D) 4

Q112. A queue undergoes: enqueue 1, enqueue 2, enqueue 3, dequeue, enqueue 4, dequeue. What is now at the front? A) 2 B) 3 C) 4 D) 1

Q113. For the tree below, the preorder traversal is:



A) 1 2 4 5 3 B) 4 2 5 1 3 C) 4 5 2 3 1 D) 1 2 3 4 5

Q114. For the same tree as Q113, the inorder traversal is: A) 1 2 4 5 3 B) 4 2 5 1 3 C) 4 5 2 3 1 D) 2 4 5 1 3

Q115. For the same tree as Q113, the postorder traversal is: A) 1 2 4 5 3 B) 4 2 5 1 3 C) 4 5 2 3 1 D) 4 5 2 1 3

Q116. Elements 50, 30, 70, 20, 40 are inserted (in this order) into an initially empty BST. The left child of the root is: A) 20 B) 30 C) 40 D) 70

Q117. For the BST formed in Q116, the inorder traversal yields: A) 20 30 40 50 70 B) 50 30 70 20 40 C) 20 40 30 70 50 D) 50 30 20 40 70

Q118. Values 10, 20, 30 are inserted into an empty AVL tree. Which rotation is required to rebalance? A) Single left rotation B) Single right rotation C) Left-Right rotation D) Right-Left rotation

Q119. Values 30, 20, 10 are inserted into an empty AVL tree. Which rotation is required? A) Single left rotation B) Single right rotation C) Left-Right rotation D) Right-Left rotation

- Q120.** The balance factor of a node in an AVL tree is calculated as: A) $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$ B) $\text{height}(\text{right subtree}) - \text{height}(\text{left subtree})$ C) $\text{height}(\text{left}) + \text{height}(\text{right})$ D) $\text{height}(\text{left}) \times \text{height}(\text{right})$
- Q121.** The maximum number of nodes in a binary tree of height 3 (root at level 0) is: A) 7 B) 8 C) 15 D) 16
- Q122.** The number of leaf nodes in a perfect binary tree of height 3 is: A) 4 B) 8 C) 15 D) 16
- Q123.** Given the max-heap array [90, 80, 70, 40, 60], the left child of the element at index 0 is: A) 80 B) 70 C) 40 D) 60
- Q124.** In a heap stored as a 0-indexed array, the parent of the element at index 4 is at index: A) 0 B) 1 C) 2 D) 3
- Q125.** How many comparisons does binary search make to find 70 in [10, 20, 30, 40, 50, 60, 70]? A) 1 B) 2 C) 3 D) 7
- Q126.** After the first complete pass of bubble sort (ascending) on [5, 1, 4, 2, 8], the array becomes: A) [1, 4, 2, 5, 8] B) [1, 2, 4, 5, 8] C) [1, 4, 5, 2, 8] D) [5, 1, 4, 2, 8]
- Q127.** After the first pass of selection sort (ascending) on [64, 25, 12, 22, 11], the array becomes: A) [11, 25, 12, 22, 64] B) [11, 12, 22, 25, 64] C) [25, 64, 12, 22, 11] D) [11, 64, 12, 22, 25]
- Q128.** The maximum number of swaps performed by selection sort on n elements is: A) n B) $n - 1$ C) n^2 D) $n \log n$
- Q129.** Which sorting algorithm is most efficient for a nearly-sorted array? A) Selection Sort B) Insertion Sort C) Merge Sort D) Heap Sort
- Q130.** The worst case $O(n^2)$ of quick sort typically occurs when: A) The data is random B) The array is already sorted and the pivot is poorly chosen C) The array is reversed only D) Never — quick sort is always $O(n \log n)$
- Q131.** Merge sort works by dividing the array into: A) Two halves B) Three parts C) n parts directly D) It does not divide the array
- Q132.** The time complexity to build a heap from n unsorted elements is: A) $O(n)$ B) $O(n \log n)$ C) $O(\log n)$ D) $O(n^2)$
- Q133.** Which data structure is explicitly used in the iterative implementation of DFS? A) Queue B) Stack C) Heap D) Priority queue
- Q134.** The space complexity of representing a graph using an adjacency list is: A) $O(V^2)$ B) $O(V + E)$ C) $O(E^2)$ D) $O(V)$
- Q135.** Which shortest-path algorithm fails when a graph has negative edge weights? A) Bellman-Ford B) Dijkstra C) Floyd-Warshall D) Prim's
- Q136.** Which of the following algorithms builds a Minimum Spanning Tree? A) Dijkstra's B) Kruskal's C) BFS D) DFS
- Q137.** Using the hash function $h(\text{key}) = \text{key} \% 10$, the key 25 is stored at index: A) 2 B) 5 C) 25 D) 0
- Q138.** In a hash table of size 10 with $h(\text{key}) = \text{key} \% 10$ and linear probing, key 15 is inserted while index 5 is already occupied. Where is 15 placed? A) Index 5 B) Index 6 C) Index 15 D) Index 0

- Q139.** The load factor of a hash table is defined as: A) table size / number of items B) number of items / table size C) number of items $\times$ table size D) collisions / number of items
- Q140.** Which collision-resolution technique stores colliding elements in linked lists? A) Linear probing B) Separate chaining C) Double hashing D) Quadratic probing
- Q141.** The time complexity of inserting a node at the end of a singly linked list (without a tail pointer) is: A) $O(1)$ B) $O(n)$ C) $O(\log n)$ D) $O(n^2)$
- Q142.** The time complexity of deleting the head node of a singly linked list is: A) $O(1)$ B) $O(n)$ C) $O(\log n)$ D) $O(n^2)$
- Q143.** What is the time complexity of three loops nested inside one another, each running n times? A) $O(n)$ B) $O(n^2)$ C) $O(n^3)$ D) $O(2^n)$
- Q144.** What is the time complexity of the loop `for (i = 1; i <= n; i = i + 2)`? A) $O(\log n)$ B) $O(n)$ C) $O(n^2)$ D) $O(1)$
- Q145.** A circular queue is preferred over a linear array queue because it solves the problem of: A) Slow speed B) Memory (slot) wastage C) Overflow only D) Sorting
- Q146.** The address of $A[5]$ given base address 1000, element size 4 bytes, and lower bound 0 is: A) 1020 B) 1024 C) 1005 D) 1040
- Q147.** For a 2D array $A[3][4]$ stored in row-major order with base address 2000 and element size 2 bytes ($LB = 0$), the address of $A[1][2]$ is: A) 2010 B) 2012 C) 2014 D) 2022
- Q148.** To obtain the elements of a BST in descending order, which traversal is used? A) Inorder B) Preorder C) Reverse inorder (Right, Root, Left) D) Postorder
- Q149.** An expression tree, when traversed to produce a postfix expression, is traversed in: A) Preorder B) Inorder C) Postorder D) Level-order
- Q150.** The number of comparisons in the worst case of linear search for n elements is: A) n B) $\log n$ C) $n / 2$ D) n^2
- Q151.** In bubble sort, after k complete passes, how many elements are guaranteed to be in their final sorted positions? A) k B) $n - k$ C) 1 D) n
- Q152.** The worst-case time complexity of quick sort is: A) $O(n \log n)$ B) $O(n^2)$ C) $O(n)$ D) $O(\log n)$
- Q153.** A connected graph with no cycles is called a: A) Forest B) Tree C) Complete graph D) Cyclic graph
- Q154.** BFS finds the shortest path in a(n): A) Weighted graph B) Unweighted graph C) Graph with negative weights D) Directed graph only
- Q155.** Which data structure is most efficiently used to implement a priority queue? A) Array B) Heap C) Stack D) Linked list

SECTION 4 — OOPs (MODERATE TO HARD) | Q156-Q200

- Q156.** What is the output?

```
class A { A(){ System.out.print("A"); } }
class B extends A { B(){ System.out.print("B"); } }
// in main:
new B();
```

A) AB B) BA C) B D) A

Q157. What is the output?

```
class Animal { void sound(){ System.out.print("Animal"); } }
class Dog extends Animal { void sound(){ System.out.print("Dog"); } }
// in main:
Animal a = new Dog();
a.sound();
```

A) Animal B) Dog C) AnimalDog D) Compile error

Q158. What is the output?

```
class P { static void show(){ System.out.print("P"); } }
class C extends P { static void show(){ System.out.print("C"); } }
// in main:
P obj = new C();
obj.show();
```

A) P B) C C) PC D) Compile error

Q159. Which of the following pairs represents valid method overloading? A) `int f(int a)` and `int f(int a, int b)` B) `int f(int a)` and `double f(int a)` C) `void f()` and `void f()` D) `int f(int a)` and `int g(int a)`

Q160. What is the output?

```
class Test {
    int x = 10;
    void show(){ System.out.print(x); }
}
// in main:
Test t = new Test();
t.x = 20;
t.show();
```

A) 10 B) 20 C) 0 D) Compile error

Q161. What is the output?

```
class A {
    A(){ System.out.print("1"); }
    A(int x){ System.out.print("2"); }
}
// in main:
new A();
new A(5);
```

A) 12 B) 21 C) 1 D) 2

Q162. What is the output?

```
class A {
    A(){ this(5); System.out.print("A"); }
    A(int x){ System.out.print("B"); }
}
// in main:
new A();
```

A) AB B) BA C) A D) B

Q163. A final method can be: A) Overridden B) Overloaded C) Both overridden and overloaded D) Neither

Q164. Which of the following causes a compile-time error? A) Overriding a method with the same signature B) Overloading a method with different parameters C) Overloading by changing only the return type D) Inheriting a class using extends

Q165. What happens with the following code?

```
abstract class Shape { abstract void draw(); }
// in main:
Shape s = new Shape();
```

A) Runs successfully B) Compile-time error (cannot instantiate abstract class) C) Prints null D) Prints "Shape"

Q166. In Java, a class can directly extend how many classes? A) 1 B) 2 C) Unlimited D) 0

Q167. In Java, a class can implement how many interfaces? A) Exactly 1 B) Exactly 2 C) Only one via extends D) Multiple

Q168. What is the output?

```
class A {
    static int count = 0;
    A(){ count++; }
}
// in main:
new A(); new A(); new A();
System.out.print(A.count);
```

A) 1 B) 3 C) 0 D) Compile error

Q169. What is the output?

```
class A { int x = 5; }
class B extends A { int x = 10; }
// in main:
A obj = new B();
System.out.print(obj.x);
```

A) 5 B) 10 C) 15 D) Compile error

Q170. Which keyword prevents a class from being inherited? A) static B) final C) abstract D) private

Q171. Which statement is true about an interface before Java 8? A) It can have concrete methods B) All its methods are abstract C) It can be instantiated D) It has constructors

Q172. What is the output?

```
class A { void display(){ System.out.print("A"); } }
class B extends A {
    void display(){ super.display(); System.out.print("B"); }
}
// in main:
new B().display();
```

A) AB B) BA C) A D) B

Q173. When overriding a method, the access modifier of the overriding method: A) Must be more restrictive B) Cannot be more restrictive (same or less restrictive) C) Must be private D) Must be static

Q174. Which of the following supports run-time polymorphism? A) Method overloading B) Method overriding C) Constructor overloading D) Static methods

Q175. What is the output?

```
class A { A(){ System.out.print("A"); } }
class B extends A { B(){ super(); System.out.print("B"); } }
// in main:
new B();
```

A) AB B) BA C) B D) A

Q176. If a subclass does not implement all abstract methods of its abstract parent, then: A) It runs fine B) The subclass must itself be declared abstract C) It compiles with a warning D) The methods are ignored

Q177. Which relationship has a dependent lifecycle, where the part cannot exist without the whole? A) Association B) Aggregation C) Composition D) Inheritance

Q178. What is the output?

```
class Test {
    static { System.out.print("S"); }
    Test(){ System.out.print("C"); }
}
// in main:
new Test();
new Test();
```

A) SCC B) SCSC C) SSC D) CC

Q179. Which access modifier makes a member accessible in a subclass of a different package but not from unrelated classes everywhere? A) private B) default C) protected D) public

Q180. A method that cannot be overridden is declared using: A) static B) final C) abstract D) public

Q181. What is the output?

```
class A {
    void m(int x){ System.out.print("int"); }
    void m(double x){ System.out.print("double"); }
}
// in main:
new A().m(5);
```

A) int B) double C) Compile error D) intdouble

Q182. Which keyword explicitly prevents a method from being overridden? A) final B) static C) abstract D) void

Q183. By default (before Java 8), an interface method has which modifiers? A) private abstract B) public abstract C) protected D) public final

Q184. What is the output?

```
class A {
    A(){ System.out.print("A"); }
    A(int x){ this(); System.out.print("B"); }
}
// in main:
new A(5);
```

A) AB B) BA C) A D) B

Q185. Which statement is NOT true about a static method? A) It belongs to the class B) It can access instance variables directly C) It can be called without creating an object D) It cannot use the `this` keyword

Q186. In code, composition is represented by: A) The extends keyword B) An object holding another object as a field that it creates and owns C) The implements keyword D) A static variable

Q187. What is the output?

```
class A {
    int x = 100;
    void show(){ System.out.print(this.x); }
}
// in main:
new A().show();
```

A) 100 B) 0 C) Compile error D) null

Q188. Which feature allows a Java class to inherit from multiple sources? A) Multiple classes B) Multiple interfaces C) Abstract classes only D) It is not possible at all

Q189. Encapsulation is best demonstrated by: A) public variables B) private variables with public getter/setter methods C) static methods D) abstract classes

Q190. What is the output?

```
class Base { Base(){ System.out.print("Base"); } }
class Derived extends Base { Derived(){ System.out.print("Derived"); } }
// in main:
Base b = new Derived();
```

A) BaseDerived B) DerivedBase C) Base D) Derived

Q191. Which concept allows the same method name to behave differently depending on the object? A) Encapsulation B) Polymorphism C) Abstraction D) Inheritance

Q192. A subclass object is assigned to a superclass reference. Calling an overridden method executes the: A) Superclass version B) Subclass version C) Both versions D) Neither (error)

Q193. Which statement about constructors is true? A) They can be inherited B) They cannot be inherited C) They have a return type D) They are static

Q194. What is the output?

```
interface I { int VALUE = 5; }
class A implements I {
    void show(){ System.out.print(VALUE); }
}
// in main:
new A().show();
```

A) 5 B) 0 C) Compile error D) null

Q195. Which of the following languages is dynamically typed? A) Java B) C++ C) Python D) C

Q196. When a subclass defines a static method with the same signature as the parent's static method, it is called: A) Overriding B) Overloading C) Method hiding D) Abstraction

Q197. What is the output?

```
class A { void greet(){ System.out.print("Hello"); } }
class B extends A { void greet(){ System.out.print("Hi"); } }
// in main:
A a = new A();
a.greet();
```

A) Hello B) Hi C) HelloHi D) Compile error

Q198. Before Java 8, which is NOT allowed for an interface method? A) Being public B) Being abstract C) Having a body (concrete implementation) D) Being public and abstract

Q199. In a constructor, the `super` keyword is used to: A) Call the current class constructor B) Call the parent class constructor C) Create an object D) End the constructor

Q200. What is the output?

```
class A { static int x = 5; }
class B extends A {}
// in main:
B.x = 10;
System.out.print(A.x);
```

A) 5 B) 10 C) 0 D) Compile error

(End of Part D — 200 MCQs)

PART E — DETAILED ANSWER KEY & EXPLANATIONS

Each answer includes the reasoning, why other options fail, a remember-point, and (where relevant) the examiner's trap. Tracing, complexity, traversal, AVL, and sorting answers show the working.

ANSWERS Q1-Q55 (DSA — Easy)

Q1. Correct Answer: B) Stack A stack follows Last In, First Out (LIFO): the last element pushed is the first popped. Why others are wrong: A) Queue is FIFO; C) Tree is hierarchical; D) Graph models vertices/edges. ★ Remember: Stack = LIFO. △ Trap: Don't confuse with Queue (FIFO). Difficulty: Easy | Topic: Stack | Type: PYQ Pattern

Q2. Correct Answer: C) Queue A queue follows First In, First Out (FIFO): the first element added is the first removed. Why others: A) Stack is LIFO; B) Tree/D) Heap are non-linear. ★ Remember: Queue = FIFO. Difficulty: Easy | Topic: Queue | Type: PYQ Pattern

Q3. Correct Answer: C) O(1) Arrays use contiguous memory, so the address of any index is computed directly — constant time. Why others: Search is $O(n)$, not access; $\log n$ applies to binary search; n^2 is unrelated. ★ Remember: Array access/update by index = $O(1)$. △ Trap: Access is $O(1)$, but insert/delete in the middle is $O(n)$. Difficulty: Easy | Topic: Array access | Type: PYQ Pattern

Q4. Correct Answer: C) NULL The last node's link field holds NULL, marking the end of a singly linked list. Why others: Head/first node would make it circular; itself is invalid. ★ Remember: Last node → NULL (except in circular lists). Difficulty: Easy | Topic: Linked list | Type: Original Practice

Q5. Correct Answer: D) Tree Trees are hierarchical (non-linear). Arrays, stacks, and queues are linear. ★ Remember: Non-linear = Tree, Graph. Difficulty: Easy | Topic: Classification | Type: PYQ Pattern

Q6. Correct Answer: B) Push Inserting into a stack is "push"; removing is "pop". Why others: Enqueue is a queue term; append/add are generic. ★ Remember: Stack = push/pop/peek. Difficulty: Easy | Topic: Stack operations | Type: Original Practice

Q7. Correct Answer: A) Pop Removing the top element of a stack is "pop". Why others: Dequeue is for queues. ★ Remember: Pop removes the most recently pushed element. Difficulty: Easy | Topic: Stack operations | Type: Original Practice

Q8. Correct Answer: B) Rear In a queue, insertion (enqueue) is at the rear; deletion (dequeue) is at the front. ★ Remember: Enqueue → rear, Dequeue → front. △ Trap: Don't swap the ends. Difficulty: Easy | Topic: Queue operations | Type: PYQ Pattern

Q9. Correct Answer: B) Stack Recursive calls are managed by the call stack (LIFO) — most recent call returns first. ★ Remember: Recursion uses the call stack. Difficulty: Easy | Topic: Recursion/stack | Type: PYQ Pattern

- Q10. Correct Answer: C) Upper bound (worst case)** Big O expresses the maximum growth — an upper bound. Why others: Lower bound is Omega; best/average are separate cases. ★ Remember: Big O = upper bound. ⚠ Trap: Big O is not the exact time. Difficulty: Easy | Topic: Big O | Type: PYQ Pattern
- Q11. Correct Answer: B) Queue** BFS explores level by level using a queue. ★ Remember: BFS → Queue; DFS → Stack. Difficulty: Easy | Topic: BFS | Type: PYQ Pattern
- Q12. Correct Answer: B) Stack** DFS dives deep along a branch, backtracking via a stack (or recursion). ★ Remember: DFS → Stack. Difficulty: Easy | Topic: DFS | Type: PYQ Pattern
- Q13. Correct Answer: C) $O(\log n)$** Binary search halves the search space each step → logarithmic. ★ Remember: Binary search = $O(\log n)$. ⚠ Trap: Requires sorted data. Difficulty: Easy | Topic: Binary search | Type: PYQ Pattern
- Q14. Correct Answer: B) Sorted** Binary search relies on order to discard half the data each comparison. ★ Remember: No sorting → binary search gives wrong results. Difficulty: Easy | Topic: Binary search | Type: PYQ Pattern
- Q15. Correct Answer: C) $O(n)$** Worst case scans all n elements (target last or absent). Why others: $O(1)$ is best case only. ★ Remember: Linear search worst = $O(n)$. Difficulty: Easy | Topic: Linear search | Type: Original Practice
- Q16. Correct Answer: B) 2** A binary tree node has at most two children (left and right). ★ Remember: "Binary" = 2 children max. Difficulty: Easy | Topic: Binary tree | Type: PYQ Pattern
- Q17. Correct Answer: B) Leaf** A leaf (external node) has no children. Why others: Root has no parent; internal nodes have children. ★ Remember: Leaf = no children. Difficulty: Easy | Topic: Tree terminology | Type: Original Practice
- Q18. Correct Answer: C) Root** The root is the topmost node with no parent. ★ Remember: Root = entry point of the tree. Difficulty: Easy | Topic: Tree terminology | Type: Original Practice
- Q19. Correct Answer: C) Sorted ascending order** Inorder (Left, Root, Right) on a BST visits values in increasing order. ★ Remember: BST inorder = sorted ascending. ⚠ Trap: True only for BST, not general binary trees. Difficulty: Easy | Topic: BST inorder | Type: PYQ Pattern
- Q20. Correct Answer: C) Merge Sort** Merge sort is $O(n \log n)$ in best, average, and worst cases. Why others: Bubble/Selection/Insertion are $O(n^2)$ average. ★ Remember: Guaranteed $O(n \log n)$: Merge and Heap. Difficulty: Easy | Topic: Sorting | Type: PYQ Pattern
- Q21. Correct Answer: B) Push onto a full stack** Overflow = pushing when the (array-based) stack is full. ★ Remember: Overflow = push full; Underflow = pop empty. Difficulty: Easy | Topic: Stack overflow | Type: Original Practice
- Q22. Correct Answer: B) Pop from an empty stack** Underflow = popping when the stack is empty. ★ Remember: Underflow = pop empty. Difficulty: Easy | Topic: Stack underflow | Type: Original Practice
- Q23. Correct Answer: A) Last In First Out** LIFO = Last In, First Out (the stack principle). Difficulty: Easy | Topic: LIFO | Type: Original Practice
- Q24. Correct Answer: A) First In First Out** FIFO = First In, First Out (the queue principle). Difficulty: Easy | Topic: FIFO | Type: Original Practice

- Q25. Correct Answer: C) Array** Arrays are linear (sequential). Trees, graphs, heaps are non-linear. ★ Remember: Linear = Array, Stack, Queue, Linked List. Difficulty: Easy | Topic: Classification | Type: PYQ Pattern
- Q26. Correct Answer: B) $n - 1$** A tree with n nodes always has exactly $n - 1$ edges. ★ Remember: n nodes $\rightarrow n - 1$ edges. Difficulty: Easy | Topic: Tree edges | Type: PYQ Pattern
- Q27. Correct Answer: B) Array** Arrays store elements in contiguous memory; linked lists/trees/graphs use scattered nodes with pointers. ★ Remember: Array = contiguous memory. Difficulty: Easy | Topic: Array memory | Type: Original Practice
- Q28. Correct Answer: B) Front** Dequeue removes from the front of a queue. ★ Remember: Delete at front, insert at rear. Difficulty: Easy | Topic: Queue operations | Type: PYQ Pattern
- Q29. Correct Answer: C) Views the top element without removing it** Peek/top returns the top value but leaves the stack unchanged. ★ Remember: Peek reads; pop removes. Difficulty: Easy | Topic: Stack peek | Type: Original Practice
- Q30. Correct Answer: B) Head** The head pointer is the entry point; losing it loses the whole list. ★ Remember: Head = access to the entire linked list. Difficulty: Easy | Topic: Linked list | Type: Original Practice
- Q31. Correct Answer: B) A pointer/link to the next node** A singly linked list node = data + link (next). ★ Remember: Node = data + next. Difficulty: Easy | Topic: Linked list node | Type: Original Practice
- Q32. Correct Answer: B) 2** A doubly linked list node has two pointers: prev and next. ★ Remember: Doubly = prev + next (extra memory). Difficulty: Easy | Topic: Doubly linked list | Type: Original Practice
- Q33. Correct Answer: C) $O(1)$** $O(1)$ (constant) is the fastest — same time regardless of input size. ★ Remember: Growth: $1 < \log n < n < n \log n < n^2 \dots$ Difficulty: Easy | Topic: Complexity order | Type: Original Practice
- Q34. Correct Answer: B) $O(1)$** Among the options, constant time grows the slowest. ★ Remember: Smallest growth = $O(1)$. Difficulty: Easy | Topic: Growth order | Type: PYQ Pattern
- Q35. Correct Answer: B) $O(n!)$** Factorial grows fastest: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$. ⚠ Trap: 2^n is large but $n!$ is larger. ★ Remember: $n!$ is the worst on the standard scale. Difficulty: Easy | Topic: Growth order | Type: PYQ Pattern
- Q36. Correct Answer: B) AB+** Postfix places the operator after operands: $A + B \rightarrow AB+$. ★ Remember: Postfix = operator last. Difficulty: Easy | Topic: Postfix | Type: PYQ Pattern
- Q37. Correct Answer: B) +AB** Prefix places the operator before operands: $A + B \rightarrow +AB$. ★ Remember: Prefix = operator first. Difficulty: Easy | Topic: Prefix | Type: PYQ Pattern
- Q38. Correct Answer: B) Postfix** Postfix (and prefix) need no parentheses; infix requires them for precedence. ★ Remember: Postfix/prefix are parenthesis-free. Difficulty: Easy | Topic: Notation | Type: Original Practice
- Q39. Correct Answer: C) $O(n^2)$** Selection sort always makes $\sim n^2/2$ comparisons — $O(n^2)$ in every case. ★ Remember: Selection sort is $O(n^2)$ always (not adaptive). Difficulty: Easy | Topic: Selection sort | Type: PYQ Pattern

- Q40. Correct Answer: D) Merge Sort** Merge sort is stable (equal elements keep order). Selection, Quick, Heap are not stable. ★ Remember: Stable = Bubble, Insertion, Merge. Difficulty: Easy | Topic: Stable sort | Type: PYQ Pattern
- Q41. Correct Answer: B) $O(n)$** Merge sort needs an auxiliary array of size $n \rightarrow O(n)$ extra space. ★ Remember: Merge sort is not in-place ($O(n)$ space). Difficulty: Easy | Topic: Merge sort space | Type: PYQ Pattern
- Q42. Correct Answer: B) Quick Sort** Quick sort is in-place but has $O(n^2)$ worst case (bad pivot). ★ Remember: Quick = in-place, worst $O(n^2)$. Difficulty: Easy | Topic: Sorting properties | Type: PYQ Pattern
- Q43. Correct Answer: B) Level by level, left to right** A complete binary tree fills each level fully (last level left-aligned). ★ Remember: Complete = filled top-down, left-to-right. Difficulty: Easy | Topic: Complete tree | Type: Original Practice
- Q44. Correct Answer: B) $\{-1, 0, 1\}$** AVL balance factor must be $-1, 0$, or $+1$; otherwise a rotation occurs. ★ Remember: $|BF| \leq 1$ for AVL. Difficulty: Easy | Topic: AVL balance | Type: PYQ Pattern
- Q45. Correct Answer: B) Largest element** Max-heap: every parent $\geq$ children, so the root is the maximum. ★ Remember: Max-heap root = max; min-heap root = min. Difficulty: Easy | Topic: Max-heap | Type: PYQ Pattern
- Q46. Correct Answer: B) Complete binary tree** A heap is a complete binary tree satisfying the heap property. ⚠ Trap: A heap is not a BST. ★ Remember: Heap = complete binary tree in an array. Difficulty: Easy | Topic: Heap | Type: PYQ Pattern
- Q47. Correct Answer: C) $O(V^2)$** An adjacency matrix is $V \times V \rightarrow O(V^2)$ space. ★ Remember: Matrix $O(V^2)$; list $O(V+E)$. Difficulty: Easy | Topic: Adjacency matrix | Type: PYQ Pattern
- Q48. Correct Answer: B) $V - 1$** An MST connects all V vertices with exactly $V - 1$ edges. ★ Remember: MST edges = $V - 1$. Difficulty: Easy | Topic: MST edges | Type: PYQ Pattern
- Q49. Correct Answer: C) $O(1)$** Hashing gives average $O(1)$ search via direct index computation. ⚠ Trap: Worst case is $O(n)$ (all collide). ★ Remember: Hashing average = $O(1)$. Difficulty: Easy | Topic: Hashing | Type: PYQ Pattern
- Q50. Correct Answer: A) Two different keys map to the same index** That is the definition of a collision. ★ Remember: Collision $\rightarrow$ resolved by chaining or open addressing. Difficulty: Easy | Topic: Collision | Type: Original Practice
- Q51. Correct Answer: C) Level-order** Level-order (BFS) traversal uses a queue to visit nodes level by level. ★ Remember: Level-order = queue-based BFS. Difficulty: Easy | Topic: Traversal | Type: PYQ Pattern
- Q52. Correct Answer: B) Root, Left, Right** Preorder visits the root first, then left, then right subtree. ★ Remember: Pre = Root first. Difficulty: Easy | Topic: Preorder | Type: PYQ Pattern
- Q53. Correct Answer: B) Linked list** A linked list grows/shrinks at run time $\rightarrow$ dynamic. Arrays are static. ★ Remember: Dynamic = linked list. Difficulty: Easy | Topic: Dynamic DS | Type: Original Practice
- Q54. Correct Answer: B) 8** Max nodes at level $l = 2^l$. Level 3 $\rightarrow 2^3 = 8$. ★ Remember: Max nodes at level $l = 2^l$. Difficulty: Easy | Topic: Tree formula | Type: PYQ Pattern

Q55. Correct Answer: B) Linear search Linear search works on unsorted data; binary search needs sorted data. ★ Remember: Unsorted → linear search. Difficulty: Easy | Topic: Searching | Type: Original Practice

ANSWERS Q56-Q100 (OOPs — Easy)

Q56. Correct Answer: A) Object Oriented Programming OOP models software as interacting objects that bundle data and behaviour. ★ Remember: OOP = Object Oriented Programming. Difficulty: Easy | Topic: OOP basics | Type: Original Practice

Q57. Correct Answer: B) Class A class is the blueprint/template; objects are built from it. Why others: An object is an instance, not a blueprint. ★ Remember: Class = blueprint; Object = instance. Difficulty: Easy | Topic: Class/Object | Type: PYQ Pattern

Q58. Correct Answer: B) Object An instance created from a class is an object (occupies memory). ★ Remember: One class → many objects. Difficulty: Easy | Topic: Class/Object | Type: PYQ Pattern

Q59. Correct Answer: C) 4 The four pillars: Abstraction, Polymorphism, Inheritance, Encapsulation. ★ Remember: A-PIE = four pillars. Difficulty: Easy | Topic: Pillars | Type: PYQ Pattern

Q60. Correct Answer: C) Compilation Compilation is a build step, not an OOP pillar. ⚠ Trap: Class/object are core concepts, not pillars either. ★ Remember: Pillars = A-PIE only. Difficulty: Easy | Topic: Pillars | Type: PYQ Pattern

Q61. Correct Answer: B) Encapsulation Encapsulation bundles data + methods and hides the data. ★ Remember: Encapsulation = bundle + hide (data security). Difficulty: Easy | Topic: Encapsulation | Type: PYQ Pattern

Q62. Correct Answer: B) Abstraction Abstraction hides implementation, exposing only essentials. ⚠ Trap: Encapsulation hides data; abstraction hides complexity. ★ Remember: Abstraction = hide "how", show "what". Difficulty: Easy | Topic: Abstraction | Type: PYQ Pattern

Q63. Correct Answer: C) Inheritance Inheritance lets one class acquire another's members. ★ Remember: Inheritance = reuse via extends (IS-A). Difficulty: Easy | Topic: Inheritance | Type: PYQ Pattern

Q64. Correct Answer: B) Polymorphism "Many forms" = polymorphism (overloading + overriding). ★ Remember: Polymorphism = one name, many behaviours. Difficulty: Easy | Topic: Polymorphism | Type: PYQ Pattern

Q65. Correct Answer: C) extends Java uses `extends` for class inheritance. Why others: `implements` is for interfaces. ★ Remember: `extends` = inherit class; `implements` = interface. Difficulty: Easy | Topic: Inheritance keyword | Type: PYQ Pattern

Q66. Correct Answer: B) new `new` allocates memory and creates an object in Java. ★ Remember: Objects are created with `new`. Difficulty: Easy | Topic: Object creation | Type: Original Practice

Q67. Correct Answer: B) Class A constructor shares the class's name and has no return type. ★ Remember: Constructor name = class name. Difficulty: Easy | Topic: Constructor | Type: PYQ Pattern

Q68. Correct Answer: C) No return type Constructors have no return type (not even void). ⚠ Trap: Adding a return type makes it an ordinary method. ★ Remember: Constructor = no return type. Difficulty: Easy | Topic: Constructor | Type: PYQ Pattern

Q69. Correct Answer: B) this `this` refers to the current object. Why others: `self` is Python; `super` is the parent. ★ Remember: `this` = current object. Difficulty: Easy | Topic: `this` keyword | Type: Original Practice

Q70. Correct Answer: C) super `super` refers to the immediate parent class. ★ Remember: `super` = parent; `this` = current. Difficulty: Easy | Topic: `super` keyword | Type: Original Practice

Q71. Correct Answer: B) Overloading Same name + different parameters = overloading (compile-time). ★ Remember: Overload = different parameters, same class. Difficulty: Easy | Topic: Overloading | Type: PYQ Pattern

Q72. Correct Answer: B) Overriding Subclass redefining a superclass method (same signature) = overriding. ★ Remember: Override = subclass, same signature, run-time. Difficulty: Easy | Topic: Overriding | Type: PYQ Pattern

Q73. Correct Answer: B) Compile time Overloading is resolved by the compiler using argument types. ★ Remember: Overloading = static (compile-time) binding. Difficulty: Easy | Topic: Overloading binding | Type: PYQ Pattern

Q74. Correct Answer: B) Run time Overriding is resolved at run time by the object's actual type. ★ Remember: Overriding = dynamic (run-time) binding. Difficulty: Easy | Topic: Overriding binding | Type: PYQ Pattern

Q75. Correct Answer: B) Using private fields with public getters and setters This is the standard encapsulation/data-hiding technique. Why others: Public fields break encapsulation. ★ Remember: private data + public getters/setters. Difficulty: Easy | Topic: Encapsulation | Type: PYQ Pattern

Q76. Correct Answer: C) private `private` is the most restrictive (same class only). ★ Remember: `private` < `default` < `protected` < `public`. Difficulty: Easy | Topic: Access modifiers | Type: PYQ Pattern

Q77. Correct Answer: C) public `public` allows access from everywhere. ★ Remember: `public` = widest access. Difficulty: Easy | Topic: Access modifiers | Type: PYQ Pattern

Q78. Correct Answer: B) Abstract class An abstract class cannot be instantiated directly. ★ Remember: Abstract class = no direct object (subclass required). Difficulty: Easy | Topic: Abstract class | Type: PYQ Pattern

Q79. Correct Answer: B) public static final Interface variables are implicitly public static final (constants). ⚠ Trap: You cannot reassign interface variables. ★ Remember: Interface fields = constants. Difficulty: Easy | Topic: Interface variables | Type: PYQ Pattern

Q80. Correct Answer: B) Interfaces Java achieves multiple inheritance only via interfaces (not classes). ⚠ Trap: Classes can't multiple-inherit (diamond problem). ★ Remember: Multiple inheritance in Java = interfaces. Difficulty: Easy | Topic: Multiple inheritance | Type: PYQ Pattern

Q81. Correct Answer: A) Constant A `final` variable cannot be reassigned — a constant. ★ Remember: final variable = constant. Difficulty: Easy | Topic: final variable | Type: Original Practice

Q82. Correct Answer: B) Overridden A `final` method cannot be overridden (but can be overloaded). ★ Remember: final method = no override. Difficulty: Easy | Topic: final method | Type: PYQ Pattern

Q83. Correct Answer: B) Extended A `final` class cannot be subclassed (e.g., `String`). ★ Remember: final class = no inheritance. Difficulty: Easy | Topic: final class | Type: PYQ Pattern

Q84. Correct Answer: B) Class A static member belongs to the class and is shared by all objects. ★ Remember: static = class-level, one shared copy. Difficulty: Easy | Topic: static | Type: PYQ Pattern

Q85. Correct Answer: B) main() `main()` is the entry point of a Java program. ★ Remember: Execution begins at `main()`. Difficulty: Easy | Topic: Java entry | Type: Original Practice

Q86. Correct Answer: B) static `main` is static so the JVM can call it without creating an object. ★ Remember: main is public static void. Difficulty: Easy | Topic: main static | Type: PYQ Pattern

Q87. Correct Answer: B) Inheritance IS-A ("a Dog is an Animal") is modelled by inheritance. ★ Remember: IS-A = inheritance. Difficulty: Easy | Topic: IS-A | Type: PYQ Pattern

Q88. Correct Answer: B) Association HAS-A ("a Car has an Engine") is association. ★ Remember: HAS-A = association. Difficulty: Easy | Topic: HAS-A | Type: PYQ Pattern

Q89. Correct Answer: C) Composition Composition is the strongest relationship (dependent lifecycle). ★ Remember: Composition > Aggregation > Association. Difficulty: Easy | Topic: Relationships | Type: PYQ Pattern

Q90. Correct Answer: B) init Python's constructor method is `__init__`. ★ Remember: Python constructor = `__init__`. Difficulty: Easy | Topic: Python constructor | Type: PYQ Pattern

Q91. Correct Answer: B) self Python uses `self` (Java's `this`), passed explicitly. ★ Remember: `self` (Python) ≈ `this` (Java). Difficulty: Easy | Topic: Python self | Type: PYQ Pattern

Q92. Correct Answer: B) Encapsulation Data hiding (via `private`) is the core of encapsulation. ★ Remember: Data hiding = encapsulation. Difficulty: Easy | Topic: Data hiding | Type: Original Practice

Q93. Correct Answer: A) A-PIE A-PIE = Abstraction, Polymorphism, Inheritance, Encapsulation. ★ Remember: Four pillars = A-PIE. Difficulty: Easy | Topic: Pillars mnemonic | Type: Original Practice

Q94. Correct Answer: B) Object An object has state (fields) and behaviour (methods). ★ Remember: Object = state + behaviour. Difficulty: Easy | Topic: Object concept | Type: Original Practice

Q95. Correct Answer: B) Abstract classes and interfaces Java delivers abstraction through abstract classes and interfaces. ★ Remember: Abstraction tools = abstract class + interface. Difficulty: Easy | Topic: Abstraction | Type: PYQ Pattern

Q96. Correct Answer: B) Automatically when an object is created Constructors run automatically on object creation (with `new`). ★ Remember: Constructor is auto-invoked by `new`. Difficulty: Easy | Topic: Constructor | Type: Original Practice

Q97. Correct Answer: B) Interface An interface cannot have a constructor (it can't be instantiated). Δ Trap: An abstract class CAN have a constructor. $\star$ Remember: Interface = no constructor. Difficulty: Easy | Topic: Interface | Type: PYQ Pattern

Q98. Correct Answer: B) Inheritance A parent-child class relation is inheritance. $\star$ Remember: Parent-child = inheritance (IS-A). Difficulty: Easy | Topic: Inheritance | Type: Original Practice

Q99. Correct Answer: B) Inheritance Reusability comes mainly from inheriting existing code. $\star$ Remember: Reusability = inheritance. Difficulty: Easy | Topic: Reusability | Type: PYQ Pattern

Q100. Correct Answer: D) friend `friend` is a C++ keyword, not a Java access modifier. Java has private, default, protected, public. Δ Trap: Don't mix C++ (`friend`) with Java modifiers. $\star$ Remember: Java modifiers = private/default/protected/public. Difficulty: Easy | Topic: Access modifiers | Type: PYQ Pattern

ANSWERS Q101-Q155 (DSA — Moderate to Hard)

Q101. Correct Answer: B) $O(n^2)$ Outer loop runs n times; for each, inner loop runs n times $\rightarrow n \times n = n^2$. $\star$ Remember: Nested loops multiply. Difficulty: Moderate | Topic: Nested loops | Type: PYQ Pattern

Q102. Correct Answer: C) $O(\log n)$ i doubles each step: 1, 2, 4, ... $n \rightarrow$ about $\log_2 n$ iterations. $\star$ Remember: Counter $\times$ constant $\rightarrow O(\log n)$. Difficulty: Moderate | Topic: Doubling loop | Type: PYQ Pattern

Q103. Correct Answer: B) $O(n^2)$ Inner runs $1+2+\dots+n = n(n+1)/2$ times $\rightarrow$ dominant term $n^2 \rightarrow O(n^2)$. $\star$ Remember: Triangular nested loop is still $O(n^2)$. Difficulty: Moderate | Topic: Triangular loop | Type: PYQ Pattern

Q104. Correct Answer: B) $O(n)$ $n + n = 2n$; drop the constant $\rightarrow O(n)$. Consecutive loops add. $\star$ Remember: Add consecutive loops, keep the largest term. Difficulty: Moderate | Topic: Consecutive loops | Type: Original Practice

Q105. Correct Answer: C) $O(2^n)$ Each call spawns two calls ($\text{fib}(n-1) + \text{fib}(n-2)$) $\rightarrow$ exponential growth. Δ Trap: Iterative Fibonacci is $O(n)$; the naive recursion is $O(2^n)$. $\star$ Remember: Naive Fibonacci = $O(2^n)$. Difficulty: Moderate | Topic: Fibonacci recursion | Type: PYQ Pattern

Q106. Correct Answer: A) $AB+C$ $(A+B)$ first $\rightarrow AB+$; then $C \rightarrow AB+C^*$. $\star$ Remember: Brackets force the inner operation first. Difficulty: Moderate | Topic: Infix $\rightarrow$ Postfix | Type: PYQ Pattern

Q107. Correct Answer: B) ABC^*+ $*$ has higher precedence: $BC \rightarrow BC$; then $A+ \rightarrow ABC^*+$. $\star$ Remember: Higher-precedence operator converts first. Difficulty: Moderate | Topic: Infix $\rightarrow$ Postfix | Type: PYQ Pattern

Q108. Correct Answer: B) $+*ABC$ $AB \rightarrow AB$; then $+C$ wraps around $\rightarrow +*ABC$. $\star$ Remember: Prefix operator precedes its operands. Difficulty: Moderate | Topic: Infix $\rightarrow$ Prefix | Type: PYQ Pattern

Q109. Correct Answer: A) 40 Push 5,6,2. $+$ $\rightarrow$ pop 2,6 $\rightarrow$ 8. $*$ $\rightarrow$ pop 8,5 $\rightarrow$ $5 \times 8 = 40$. ★
Remember: Operand $\rightarrow$ push; operator $\rightarrow$ pop two, compute. Difficulty: Moderate | Topic: Postfix evaluation | Type: PYQ Pattern

Q110. Correct Answer: A) 1 Push 8,2. $/$ $\rightarrow$ first popped 2 is right, 8 is left $\rightarrow$ $8/2 = 4$. Push 3. $-$ $\rightarrow$ $4-3 = 1$. ⚠ Trap: First popped is the RIGHT operand (matters for $-$ and $/$). Difficulty: Moderate | Topic: Postfix evaluation | Type: PYQ Pattern

Q111. Correct Answer: B) 2 Push 1,2,3 $\rightarrow$ [1,2,3]. Pop $\rightarrow$ 3 out $\rightarrow$ [1,2]. Push 4 $\rightarrow$ [1,2,4]. Pop $\rightarrow$ 4 out $\rightarrow$ [1,2]. Top = 2. ★ Remember: Trace push/pop as LIFO. Difficulty: Moderate | Topic: Stack trace | Type: Original Practice

Q112. Correct Answer: B) 3 Enqueue 1,2,3 $\rightarrow$ front 1. Dequeue $\rightarrow$ 1 out. Enqueue 4 $\rightarrow$ [2,3,4]. Dequeue $\rightarrow$ 2 out. Front = 3. ★ Remember: Queue removes from the front (FIFO). Difficulty: Moderate | Topic: Queue trace | Type: Original Practice

Q113. Correct Answer: A) 1 2 4 5 3 Preorder = Root, Left, Right: $1 \rightarrow (2 \rightarrow 4 \rightarrow 5) \rightarrow 3$. ★
Remember: Pre = Root first. Difficulty: Moderate | Topic: Preorder | Type: PYQ Pattern

Q114. Correct Answer: B) 4 2 5 1 3 Inorder = Left, Root, Right: $(4 \ 2 \ 5) \rightarrow 1 \rightarrow 3$. ★
Remember: In = Root in the middle. Difficulty: Moderate | Topic: Inorder | Type: PYQ Pattern

Q115. Correct Answer: C) 4 5 2 3 1 Postorder = Left, Right, Root: $(4 \ 5 \ 2) \rightarrow (3) \rightarrow 1$. ★
Remember: Post = Root last. Difficulty: Moderate | Topic: Postorder | Type: PYQ Pattern

Q116. Correct Answer: B) 30 Insert 50 (root); $30 < 50 \rightarrow$ left child. 70 $\rightarrow$ right; 20 $\rightarrow$ left of 30; 40 $\rightarrow$ right of 30. Root's left child = 30. ★ Remember: BST inserts left if smaller, right if larger. Difficulty: Moderate | Topic: BST insert | Type: PYQ Pattern

Q117. Correct Answer: A) 20 30 40 50 70 Inorder of the BST gives ascending order. ★
Remember: BST inorder = sorted. Difficulty: Moderate | Topic: BST inorder | Type: PYQ Pattern

Q118. Correct Answer: A) Single left rotation 10 $\rightarrow$ 20 $\rightarrow$ 30 is a right-right (RR) imbalance at 10 (BF = -2) $\rightarrow$ single left rotation. ⚠ Trap: RR is fixed by a LEFT rotation (opposite of the name). Difficulty: Moderate | Topic: AVL RR | Type: PYQ Pattern

Q119. Correct Answer: B) Single right rotation 30 $\rightarrow$ 20 $\rightarrow$ 10 is a left-left (LL) imbalance (BF = $+2$) $\rightarrow$ single right rotation. ⚠ Trap: LL is fixed by a RIGHT rotation. Difficulty: Moderate | Topic: AVL LL | Type: PYQ Pattern

Q120. Correct Answer: A) height(left) – height(right) Balance factor = left subtree height minus right subtree height. ★ Remember: BF = left – right (must be $-1, 0, +1$). Difficulty: Moderate | Topic: AVL balance factor | Type: PYQ Pattern

Q121. Correct Answer: C) 15 Max nodes for height $h = 2^{(h+1)} - 1 = 2^4 - 1 = 15$. ★
Remember: Max nodes = $2^{(h+1)} - 1$. Difficulty: Moderate | Topic: Tree formula | Type: PYQ Pattern

Q122. Correct Answer: B) 8 Leaves of a perfect tree of height $h = 2^h = 2^3 = 8$. ★
Remember: Perfect tree leaves = 2^h . Difficulty: Moderate | Topic: Tree formula | Type: PYQ Pattern

Q123. Correct Answer: A) 80 Left child of index 0 is at $2 \times 0 + 1 =$ index 1 $\rightarrow$ value 80. ★
Remember: Left child = $2i+1$. Difficulty: Moderate | Topic: Heap indexing | Type: PYQ Pattern

Q124. Correct Answer: B) 1 Parent of index $i = \lfloor (i-1)/2 \rfloor = \lfloor 3/2 \rfloor = 1$. ★ Remember: Parent = $\lfloor (i-1)/2 \rfloor$. Difficulty: Moderate | Topic: Heap indexing | Type: PYQ Pattern

- Q125. Correct Answer: C) 3** $low=0, high=6, mid=3(40)<70 \rightarrow low=4;$
 $mid=5(60)<70 \rightarrow low=6; mid=6(70)$ found. 3 comparisons. ★ Remember: Binary search $\approx \log_2 n$ comparisons. Difficulty: Moderate | Topic: Binary search trace | Type: PYQ Pattern
- Q126. Correct Answer: A) [1, 4, 2, 5, 8]** Pass 1 swaps: $5>1 \rightarrow [1, 5, 4, 2, 8]; 5>4 \rightarrow [1, 4, 5, 2, 8];$
 $5>2 \rightarrow [1, 4, 2, 5, 8]; 5<8$ stop. 8 fixed. ★ Remember: After pass 1, the largest bubbles to the end. Difficulty: Moderate | Topic: Bubble pass | Type: PYQ Pattern
- Q127. Correct Answer: A) [11, 25, 12, 22, 64]** Minimum of the array is 11; swap it to the front. Rest stays. ★ Remember: Selection sort places the minimum first each pass. Difficulty: Moderate | Topic: Selection pass | Type: PYQ Pattern
- Q128. Correct Answer: B) $n - 1$** Selection sort performs at most $n-1$ swaps (one per pass). ★ Remember: Selection = fewest swaps ($\leq n-1$). Difficulty: Moderate | Topic: Selection swaps | Type: PYQ Pattern
- Q129. Correct Answer: B) Insertion Sort** Insertion sort is adaptive $\rightarrow O(n)$ on nearly-sorted data. ★ Remember: Nearly sorted $\rightarrow$ insertion sort. Difficulty: Moderate | Topic: Adaptive sort | Type: PYQ Pattern
- Q130. Correct Answer: B) The array is already sorted and the pivot is poorly chosen** A smallest/largest pivot creates unbalanced partitions $\rightarrow O(n^2)$. ★ Remember: Bad pivot on sorted data = quick sort worst case. Difficulty: Moderate | Topic: Quick sort worst | Type: PYQ Pattern
- Q131. Correct Answer: A) Two halves** Merge sort recursively splits the array into two halves. ★ Remember: Divide-and-conquer into halves. Difficulty: Moderate | Topic: Merge sort | Type: Original Practice
- Q132. Correct Answer: A) $O(n)$** Building a heap via bottom-up heapify is $O(n)$, not $O(n \log n)$. ⚠ Trap: Build-heap is $O(n)$. Difficulty: Moderate | Topic: Build heap | Type: PYQ Pattern
- Q133. Correct Answer: B) Stack** Iterative DFS uses an explicit stack (recursive DFS uses the call stack). ★ Remember: DFS $\rightarrow$ stack. Difficulty: Moderate | Topic: DFS structure | Type: PYQ Pattern
- Q134. Correct Answer: B) $O(V + E)$** An adjacency list stores each vertex plus its edges $\rightarrow O(V + E)$. ★ Remember: List $O(V+E)$; matrix $O(V^2)$. Difficulty: Moderate | Topic: Adjacency list | Type: PYQ Pattern
- Q135. Correct Answer: B) Dijkstra** Dijkstra assumes non-negative weights; negatives break it. Use Bellman-Ford. ⚠ Trap: Dijkstra fails with negative edges. Difficulty: Moderate | Topic: Shortest path | Type: PYQ Pattern
- Q136. Correct Answer: B) Kruskal's** Kruskal's (and Prim's) build a Minimum Spanning Tree. ★ Remember: MST $\rightarrow$ Prim's / Kruskal's. Difficulty: Moderate | Topic: MST | Type: PYQ Pattern
- Q137. Correct Answer: B) 5** $h(25) = 25 \% 10 = 5$. ★ Remember: key mod table_size. Difficulty: Moderate | Topic: Hashing | Type: Original Practice
- Q138. Correct Answer: B) Index 6** $h(15) = 5$ (occupied) $\rightarrow$ linear probing tries 6 (free) $\rightarrow$ store at 6. ★ Remember: Linear probing $\rightarrow$ next slot. Difficulty: Moderate | Topic: Linear probing | Type: PYQ Pattern

Q139. Correct Answer: B) number of items / table size Load factor $\alpha = \text{items} / \text{slots}$; higher $\alpha \rightarrow$ more collisions. ★ Remember: $\alpha = \text{items} / \text{table size}$. Difficulty: Moderate | Topic: Load factor | Type: PYQ Pattern

Q140. Correct Answer: B) Separate chaining Separate chaining keeps a linked list of colliding keys per slot. ★ Remember: Chaining = linked lists (open hashing). Difficulty: Moderate | Topic: Collision resolution | Type: PYQ Pattern

Q141. Correct Answer: B) $O(n)$ Without a tail pointer, you must traverse to the last node $\rightarrow O(n)$. ★ Remember: Insert at end = $O(n)$ ($O(1)$ with a tail pointer). Difficulty: Moderate | Topic: Linked list insert | Type: PYQ Pattern

Q142. Correct Answer: A) $O(1)$ Deleting the head just moves the head pointer to the next node. ★ Remember: Head deletion = $O(1)$. Difficulty: Moderate | Topic: Linked list delete | Type: PYQ Pattern

Q143. Correct Answer: C) $O(n^3)$ Three nested loops each $n \rightarrow n \times n \times n = n^3$. ★ Remember: k nested loops of $n \rightarrow O(n^k)$. Difficulty: Moderate | Topic: Triple nested | Type: PYQ Pattern

Q144. Correct Answer: B) $O(n)$ Step of 2 runs $n/2$ times; drop the constant $\rightarrow O(n)$. ⚠ Trap: A step increment does not make it logarithmic (that needs $\times/\div$). Difficulty: Moderate | Topic: Loop complexity | Type: PYQ Pattern

Q145. Correct Answer: B) Memory (slot) wastage A circular queue reuses freed front slots that a linear queue wastes. ★ Remember: Circular queue fixes linear-queue wastage. Difficulty: Moderate | Topic: Circular queue | Type: PYQ Pattern

Q146. Correct Answer: A) 1020 Address = Base + $(i - LB) \times w = 1000 + (5 - 0) \times 4 = 1000 + 20 = 1020$. ★ Remember: 1D addr = $B + (i - LB) \times w$. Difficulty: Moderate | Topic: Address calculation | Type: PYQ Pattern

Q147. Correct Answer: B) 2012 Row-major: $B + [(i - LBr) \times N + (j - LBc)] \times w = 2000 + [(1 \times 4) + 2] \times 2 = 2000 + 6 \times 2 = 2012$. ($N = 4$ columns.) ⚠ Trap: Row-major multiplies the row index by the number of columns. Difficulty: Moderate | Topic: 2D address calc | Type: PYQ Pattern

Q148. Correct Answer: C) Reverse inorder (Right, Root, Left) Reverse inorder of a BST gives descending order. ★ Remember: Reverse inorder = descending. Difficulty: Moderate | Topic: BST traversal | Type: PYQ Pattern

Q149. Correct Answer: C) Postorder Postorder traversal of an expression tree yields the postfix expression. ★ Remember: Expression tree $\rightarrow$ postorder = postfix. Difficulty: Moderate | Topic: Expression tree | Type: PYQ Pattern

Q150. Correct Answer: A) n Worst-case linear search checks all n elements $\rightarrow n$ comparisons. ★ Remember: Linear search worst = n comparisons. Difficulty: Moderate | Topic: Linear search | Type: Original Practice

Q151. Correct Answer: A) k After k passes of bubble sort, the k largest elements are finalised at the end. ★ Remember: Bubble pass $k \rightarrow k$ largest placed. Difficulty: Moderate | Topic: Bubble passes | Type: PYQ Pattern

Q152. Correct Answer: B) $O(n^2)$ Quick sort's worst case (bad pivot) is $O(n^2)$, though average is $O(n \log n)$. ★ Remember: Quick worst = $O(n^2)$. Difficulty: Moderate | Topic: Quick sort | Type: PYQ Pattern

Q153. Correct Answer: B) Tree A connected acyclic graph is, by definition, a tree. ★ Remember: Tree = connected + acyclic + (n−1 edges). Difficulty: Moderate | Topic: Graph/tree | Type: PYQ Pattern

Q154. Correct Answer: B) Unweighted graph BFS finds the shortest path (fewest edges) in unweighted graphs. ⚠ Trap: For weighted graphs use Dijkstra, not BFS. Difficulty: Moderate | Topic: BFS shortest path | Type: PYQ Pattern

Q155. Correct Answer: B) Heap A heap gives $O(\log n)$ insert and $O(1)$ peek — ideal for a priority queue. ★ Remember: Priority queue → heap. Difficulty: Moderate | Topic: Priority queue | Type: PYQ Pattern

ANSWERS Q156-Q200 (OOPs — Moderate to Hard)

Q156. Correct Answer: A) AB Creating a B object implicitly calls `super()` first → A's constructor prints "A", then B's prints "B". ★ Remember: Parent constructor runs before child. Difficulty: Moderate | Topic: Constructor order | Type: PYQ Pattern

Q157. Correct Answer: B) Dog Reference is `Animal` but the object is `Dog`. Overridden methods use the object's actual type at run time → "Dog". ★ Remember: Overriding = object type decides (dynamic dispatch). Difficulty: Moderate | Topic: Dynamic dispatch | Type: PYQ Pattern

Q158. Correct Answer: A) P `show()` is static → method hiding, not overriding. Static calls follow the **reference type** (P) at compile time → "P". ⚠ Trap: Static methods are hidden (reference type), not overridden. Difficulty: Hard | Topic: Static method hiding | Type: PYQ Pattern

Q159. Correct Answer: A) f(int a) and f(int a, int b) Different parameter counts = valid overloading. Option B differs only by return type (illegal); C is a duplicate; D uses different names. ⚠ Trap: Cannot overload by return type alone. Difficulty: Moderate | Topic: Overloading validity | Type: PYQ Pattern

Q160. Correct Answer: B) 20 `t.x` is set to 20 before `show()` runs, so it prints the updated value 20. ★ Remember: Instance fields hold per-object values. Difficulty: Moderate | Topic: Field update | Type: Original Practice

Q161. Correct Answer: A) 12 `new A()` calls the no-arg constructor → "1"; `new A(5)` calls the parameterized one → "2". Output: 12. ★ Remember: Overloaded constructors are chosen by arguments. Difficulty: Moderate | Topic: Constructor overload | Type: Original Practice

Q162. Correct Answer: B) BA `A()` calls `this(5)` first → `A(int)` prints "B"; then control returns and prints "A". Output: BA. ⚠ Trap: `this()` runs before the rest of the constructor body. Difficulty: Hard | Topic: Constructor chaining | Type: PYQ Pattern

Q163. Correct Answer: B) Overloaded A `final` method cannot be overridden, but it CAN be overloaded (different parameters). ★ Remember: final blocks overriding, not overloading. Difficulty: Moderate | Topic: final method | Type: PYQ Pattern

Q164. Correct Answer: C) Overloading by changing only the return type Two methods differing only by return type cause a compile error. ⚠ Trap: Return type alone is not enough to overload. Difficulty: Moderate | Topic: Overloading rule | Type: PYQ Pattern

Q165. Correct Answer: B) Compile-time error (cannot instantiate abstract class)

`new Shape()` on an abstract class is illegal. ★ Remember: Abstract classes need a concrete subclass. Difficulty: Moderate | Topic: Abstract instantiation | Type: PYQ Pattern

Q166. Correct Answer: A) 1 A Java class can extend only one class (single inheritance of classes). ★ Remember: One `extends` per class. Difficulty: Moderate | Topic: Single inheritance | Type: PYQ Pattern

Q167. Correct Answer: D) Multiple A class can implement multiple interfaces → Java's multiple inheritance. ★ Remember: implement many interfaces. Difficulty: Moderate | Topic: Interfaces | Type: PYQ Pattern

Q168. Correct Answer: B) 3 `count` is static (shared). Three objects increment it to 3. ★ Remember: static variable is shared across all objects. Difficulty: Moderate | Topic: Static variable | Type: PYQ Pattern

Q169. Correct Answer: A) 5 `obj` is typed `A`. Fields are resolved by the **reference type**, not the object → `A's x = 5`. ⚠ Trap: Fields are NOT polymorphic (unlike overridden methods). Difficulty: Hard | Topic: Field vs method resolution | Type: PYQ Pattern

Q170. Correct Answer: B) final A `final` class cannot be extended. ★ Remember: final class = no inheritance. Difficulty: Moderate | Topic: final class | Type: PYQ Pattern

Q171. Correct Answer: B) All its methods are abstract Before Java 8, interface methods were all public abstract (no bodies). ★ Remember: Pre-Java 8 interface = 100% abstract. Difficulty: Moderate | Topic: Interface pre-8 | Type: PYQ Pattern

Q172. Correct Answer: A) AB `super.display()` prints "A" first, then "B". Output: AB. ★ Remember: `super.method()` calls the parent version. Difficulty: Moderate | Topic: super method | Type: PYQ Pattern

Q173. Correct Answer: B) Cannot be more restrictive (same or less restrictive) An overriding method may not reduce visibility (e.g., `public` → `private` is illegal). ⚠ Trap: You can widen access, not narrow it. Difficulty: Hard | Topic: Overriding rule | Type: PYQ Pattern

Q174. Correct Answer: B) Method overriding Run-time polymorphism is achieved through overriding (dynamic dispatch). ★ Remember: Overriding = run-time polymorphism. Difficulty: Moderate | Topic: Runtime polymorphism | Type: PYQ Pattern

Q175. Correct Answer: A) AB Explicit `super()` runs A's constructor ("A") first, then B's ("B"). ★ Remember: `super()` must be first; parent runs first. Difficulty: Moderate | Topic: `super()` constructor | Type: PYQ Pattern

Q176. Correct Answer: B) The subclass must itself be declared abstract

Unimplemented abstract methods force the subclass to be abstract. ★ Remember: Implement all abstract methods, or stay abstract. Difficulty: Moderate | Topic: Abstract methods | Type: PYQ Pattern

Q177. Correct Answer: C) Composition Composition has a dependent lifecycle — the part dies with the whole. ★ Remember: Composition = cannot exist alone. Difficulty: Moderate | Topic: Relationships | Type: PYQ Pattern

Q178. Correct Answer: A) SCC Static block runs once at class load → "S". Each `new Test()` runs the constructor → "C", "C". Output: SCC. ⚠ Trap: Static block runs once, constructors run per object. Difficulty: Hard | Topic: Static block order | Type: PYQ Pattern

Q179. Correct Answer: C) protected `protected` is visible in subclasses across packages but not to unrelated classes everywhere. ⚠ Trap: `default` is package-only; `protected` reaches cross-package subclasses. Difficulty: Moderate | Topic: Access modifiers | Type: PYQ Pattern

Q180. Correct Answer: B) final A `final` method cannot be overridden. ★ Remember: final method = no override. Difficulty: Moderate | Topic: final method | Type: PYQ Pattern

Q181. Correct Answer: A) int The literal 5 is an int, so `m(int)` is chosen → "int". ★ Remember: Overload resolution matches argument type. Difficulty: Moderate | Topic: Overload resolution | Type: PYQ Pattern

Q182. Correct Answer: A) final `final` explicitly prevents a method from being overridden. ★ Remember: final = no override. Difficulty: Moderate | Topic: Prevent override | Type: PYQ Pattern

Q183. Correct Answer: B) public abstract Interface methods are implicitly public abstract (pre-Java 8). ★ Remember: Interface method default = public abstract. Difficulty: Moderate | Topic: Interface method | Type: PYQ Pattern

Q184. Correct Answer: A) AB `A(5)` calls `this()` → `A()` prints "A"; then the body prints "B". Output: AB. ★ Remember: `this()` runs first, then the remaining body. Difficulty: Hard | Topic: Constructor chaining | Type: PYQ Pattern

Q185. Correct Answer: B) It can access instance variables directly This statement is FALSE — a static method cannot access instance members directly (no object context). ⚠ Trap: Static methods have no `this`, so no direct instance access. Difficulty: Moderate | Topic: Static method | Type: PYQ Pattern

Q186. Correct Answer: B) An object holding another object as a field that it creates and owns Composition = strong ownership; the container creates and controls the part. ★ Remember: Composition = owned member object. Difficulty: Moderate | Topic: Composition | Type: PYQ Pattern

Q187. Correct Answer: A) 100 `this.x` refers to the current object's field = 100. ★ Remember: `this.field` accesses the current object's data. Difficulty: Moderate | Topic: this keyword | Type: Original Practice

Q188. Correct Answer: B) Multiple interfaces Java inherits from multiple sources only via multiple interfaces. ★ Remember: Multiple inheritance = interfaces. Difficulty: Moderate | Topic: Multiple inheritance | Type: PYQ Pattern

Q189. Correct Answer: B) private variables with public getter/setter methods This is textbook encapsulation (data hiding). ★ Remember: private data + public accessors. Difficulty: Moderate | Topic: Encapsulation | Type: PYQ Pattern

Q190. Correct Answer: A) BaseDerived `new Derived()` calls Base's constructor ("Base") then Derived's ("Derived"). ★ Remember: Parent constructor first. Difficulty: Moderate | Topic: Constructor order | Type: PYQ Pattern

Q191. Correct Answer: B) Polymorphism Same method name behaving differently by object = polymorphism. ★ Remember: Polymorphism = many forms. Difficulty: Moderate | Topic: Polymorphism | Type: PYQ Pattern

Q192. Correct Answer: B) Subclass version An overridden method runs the subclass version based on the actual object (dynamic dispatch). ★ Remember: Object type decides the overridden method. Difficulty: Moderate | Topic: Dynamic dispatch | Type: PYQ Pattern

Q193. Correct Answer: B) They cannot be inherited Constructors are not inherited (though `super()` can call the parent's). ★ Remember: Constructors are not inherited. Difficulty: Moderate | Topic: Constructors | Type: PYQ Pattern

Q194. Correct Answer: A) 5 Interface variable `VALUE` is a public static final constant = 5. ★ Remember: Interface fields are constants. Difficulty: Moderate | Topic: Interface constant | Type: PYQ Pattern

Q195. Correct Answer: C) Python Python is dynamically typed; Java, C++, and C are statically typed. ★ Remember: Python = dynamic typing. Difficulty: Moderate | Topic: Typing | Type: PYQ Pattern

Q196. Correct Answer: C) Method hiding A subclass static method with the parent's signature hides it (not overriding). ⚠ Trap: Static → hiding, not overriding. Difficulty: Hard | Topic: Static hiding | Type: PYQ Pattern

Q197. Correct Answer: A) Hello `a` refers to an actual `A` object, so `a.greet()` runs `A`'s method → "Hello". ★ Remember: The real object type determines the method. Difficulty: Moderate | Topic: Object type | Type: PYQ Pattern

Q198. Correct Answer: C) Having a body (concrete implementation) Before Java 8, interface methods could not have bodies (all abstract). ★ Remember: Pre-Java 8 interfaces = no method bodies. Difficulty: Moderate | Topic: Interface pre-8 | Type: PYQ Pattern

Q199. Correct Answer: B) Call the parent class constructor `super(...)` invokes the parent constructor from the child constructor. ★ Remember: `super()` = parent constructor; `this()` = same class. Difficulty: Moderate | Topic: `super` | Type: PYQ Pattern

Q200. Correct Answer: B) 10 `x` is static, shared between `A` and `B`. `B.x = 10` updates the single shared variable, so `A.x` prints 10. ⚠ Trap: A static variable has one copy shared across the class hierarchy. Difficulty: Hard | Topic: Static shared variable | Type: PYQ Pattern

(End of Part E — Detailed Answer Key & Explanations)

PART F — SOURCES AND REFERENCES

Technical concepts were researched from standard references and official documentation, then rewritten in original, beginner-friendly language. Exam-structure and trend data were drawn from public exam-analysis sources. Key references consulted:

Source	Used for	URL	Accessed
Oliveboard — IBPS SO IT Officer Syllabus & Exam Pattern	2026 exam pattern, topic-wise weightage	oliveboard.in/blog/ibps-so-it-officer-syllabus-and-exam-pattern	20 July 2026
Adda247 — IBPS SO IT Officer Syllabus 2026	Professional Knowledge subject list	adda247.com/jobs/ibps-so-it-officer-syllabus	20 July 2026
PracticeMock / BankersAdda — IBPS SO Syllabus 2026	DSA + OOP + language coverage confirmation	practicemock.com/blog/ibps-so-syllabus	20 July 2026
Oracle Java Language Documentation	Java OOP semantics (overriding, static, final, interfaces)	docs.oracle.com/javase	20 July 2026
Python Language Documentation	Python OOP behaviour (<code>__init__</code> , <code>self</code> , MRO)	docs.python.org/3	20 July 2026
Standard DSA references (CLRS-style asymptotic conventions)	Complexity values, sorting/searching, tree & graph properties	Standard academic references	20 July 2026

Authenticity note: No question in this book is labelled a Verified previous-year question, because reliable question-level public evidence for IBPS SO IT DSA/OOP items is not available. Questions are honestly labelled as PYQ Pattern or Original Practice.

End of Book — Ways to Officer: Dpksanu